
University of Cape Town

Department of Mathematics and Applied Mathematics

Master of Philosophy in Mathematics

# Disentangling Sequential Goal Representations in A Maze-solving Agent

*Author:*
Benjamin Sturgeon

*Student Number:*
STRBEN005

May 2025

*Supervisor:*
Associate Professor Jonathan Shock

*A thesis submitted in fulfillment of the requirements for the degree of Master of Philosophy*

# Declaration

I, Benjamin Sturgeon, hereby declare that the work on which this dissertation/thesis is based is my original work (except where acknowledgements indicate otherwise) and that neither the whole work nor any part of it has been, is being, or is to be submitted for another degree in this or any other university.

I empower the university to reproduce for the purpose of research either the whole or any portion of the contents in any manner whatsoever.

_____

Signature

_____

Date

# Abstract

In this work we provide an extensive analysis into the operations of a maze-solving reinforcement learning agent trained to navigate procedurally generated environments with multiple sequential objectives. We investigate how neural networks represent and process distinct goals when an agent must target multiple similar entities in a prescribed sequence. Our main finding is that the network encodes entity targeting primarily through the magnitude of activation values in convolutional layer channels. We demonstrate that simple constant offsets to channel activations can redirect the agent to target different entities while preserving navigational capabilities. We also identify a clear two-stage processing pipeline where the model first establishes the target entity in early convolutional layers through high-accuracy entity classification, and then shifts to processing directional and navigational information in the fully-connected layers. These findings demonstrate that amplitude-based multiplexing is a fundamental strategy for encoding multiple goals in reinforcement learning agents operating in multi-objective environments, and provide a novel mechanistic approach for understanding goal representation in neural networks.

---

Part of this work has been accepted to the NeurIPS 2025 Mechanistic Interpretability Workshop. Available at: `https://openreview.net/forum?id=KwxgxWsjE3`

# Acknowledgments

I would like to acknowledge my supervisor Associate Professor Jonathan Shock for his enormous patience and quick feedback on the work, and for supporting me at every step along the way.

Much acknowledgement is owed to the early efforts of Paul Colognese, Narmeen Oozeer, Mickey Beurskens, and Arun Jose at the beginning of this project, and for providing the initial ideas to explore this area. I must also thank the AI Safety Camp Team for organising that program, and for bringing us together. Without all of them this work would not have been possible.

# Contents

# List of Figures

# List of Tables

# Chapter 1

# Introduction

Understanding how modern deep neural networks encode information is currently a problem that is far from being solved, and yet increasingly pressing. This state of affairs ensures that highly reliable, controllable and interpretable models remain out of reach. This is particularly the case in the field of reinforcement learning (RL), which has been somewhat neglected by the techniques of mechanistic interpretability (MI) whose focus has largely been on Large Language Models (LLMs) (Elhage, Nanda, et al., 2021; McDougall et al., 2023; Olsson et al., 2022; Wang et al., 2022). With techniques from RL being applied to frontier Large Language Models (LLMs) in the development of new 'reasoning models', there appears to be a gap in the application of MI techniques to RL more broadly.

Existing techniques are being used to better understand RL models (Atrey, Clary, and Jensen, 2020), but many techniques developed for MI elsewhere have not yet seen much application in RL, though (Hilton et al., 2020) provide a notable exception. RL poses natural difficulties for interpretation due to the nature of RL outputs as actions or sequences of actions rather than as words or images. Actions are often only understandable over a broader sequence, and the logical components that go into making an action are not easily interpretable when decomposed. This adds to the challenge of extracting meaningful insights from earlier layers of the model.

Furthermore, given the high likelihood that RL systems are deployed in the operation of critical infrastructure such as data centres, electrical grid management (Nakabi and Toivanen, 2021) and, nuclear fusion reactors (Degrave et al., 2022), there is a pressing need to provide high standards of operational safety and to show that policies function predictably within specified operational bounds, so that we avoid catastrophic failures in these systems (Amodei et al., 2016), and ideally in such a way that humans can directly understand the solution being implemented.

There is an opportunity to explore novel techniques for interpreting other categories of AI systems with RL, as RL environments usually contain relatively simple goals and easily determined failure modes that allow for testing whether changes evoke a desired behaviour. For example, the approach of steering Large Language Models (LLMs) using activation steering in (Turner et al., 2024a) is inspired by work to understand and control a maze-solving agent in the Procgen Maze environment (Mini et al., 2023).

This investigation seeks to build on the work of (Mini et al., 2023), that identies specific regions in the convolutional layers of the model that can be used to control model behaviour. This work's contribution is to extend their work to an environment that involves multiple entities that an agent needs to reach in sequence. To this end, the Procgen Heist environment is chosen as the target of this research. This is because the Procgen environment requires the agent to navigate through three locks of different colours, each requiring a corresponding key before reaching a gem at the end. This means that the agent has to learn robust mechanisms for navigating to each entity in the environment (Cobbe, Hesse, et al., 2020). By observing how the model learns to approach multi-objective maze-solving, this work aims to understand how neural networks represent and pursue multiple sequential goals.

A significant challenge in understanding the internal mechanisms of complex neural networks, particularly those operating in multifaceted environments like Procgen Heist, is the prevalence of polysemanticity (Elhage, Hume, et al., 2022). This phenomenon, where individual neurons respond to multiple distinct features, can make it challenging to draw strong conclusions about how a model works.

## 1.1 Background and Motivation

RL is a field of machine learning that has the unique promise of delivering superhuman performance in a wide range of environments (Silver et al., 2016). However, the black box nature of these models makes it difficult to understand how they achieve these results.

RL agents learn through interaction with an environment, receiving observations and rewards that guide policy development (R. S. Sutton and A. G. Barto, 2018). The field of MI seeks to decode the black boxes of neural networks so that these models can be used with greater confidence and reliability (Olah, Satyanarayan, et al., 2018). Recent work provides compelling evidence that neural networks can learn meaningful internal models and conceptual abstractions from data (K. Li, Aspen K Hopkins, et al., 2022), suggesting that understanding goal representations in one model may inform investigations of others.

However, not all models that seek to target goals are good candidates for this

research, as it may be the case that the task is simple enough to solve without the need to develop a sufficiently complex mechanism to represent abstract concepts like goals. The Procgen environments stand out as good environments for this reason, as they procedurally generate environments for the agent to solve. This means that the agent cannot simply overfit to the data, but has to learn general patterns in order to score well on the environment, which necessitates developing a complex representation of goals within the model.

The choice to use Procgen Heist in particular stemmed from the desire to explore how multiple goals might be represented in the model, as significant work has already shown the presence of 'cheese representations' with the Procgen Maze environment (Mini et al., 2023). Because advanced AI agents would be operating in environments with multiple goals, the question of whether there was a substantial difference in the representation of these goals is an essential one.

However, in making the environment more complex the model too becomes more complex, and the problem of polysemantic neurons becomes a significant hurdle to understanding the model's operations. Specifically, whether the model's representations of goals in the base model represent a 'true' representation or simply result from polysemanticity in the network.

Polysemanticity refers to single neurons encoding multiple distinct features simultaneously due to limited network capacity.

This led to the use of Sparse Autoencoders (SAEs), as well as a variety of fundamental interpretability approaches, which are discussed more in 2.4, to confirm whether neurons in the network naturally tend to represent the different entities using different neurons, or whether there are single 'current goal' channels in the network that are re-used across different entities.

## 1.2 Problem Statement

If we are unable to understand how goals specifically are represented and pursued within RL models, we risk deploying systems that may pursue unintended or misaligned objectives, leading to unpredictable or harmful outcomes, especially in complex, multi-step tasks. This challenge is exacerbated by the inherent opacity of deep neural networks, where phenomena such as polysemanticity, a key focus of this investigation, can obscure the goal-related features we wish to isolate and understand.

Without a clear understanding of these internal goal representations, ensuring agent alignment with intended behaviours, debugging undesirable policies, and building reliable autonomous systems remains a formidable task. The lack of such understanding in the context of multi-objective sequential decision-making, as typified by environments like Procgen Heist, forms the central problem this

thesis aims to address.

## 1.3 Research Questions

This research seeks to determine the key functional components in a maze-solving agent that are responsible for determining the agent's goal, and to provide clearer evidence of how the network learns, represents, and pursues sequential goals. To this end, we investigate the following key questions:

**RQ1:** How are the different goals encoded in the network?

**RQ2:** What mechanism does the model use to determine which entity to target?

**RQ3:** Can we use SAEs to better determine the role of specific features?

**RQ4:** Do specific channels target specific entities, or are there general goal steering channels in the network?

**RQ5:** Why do we observe different rates of success when steering the agent away from different entities using activation patching?

**RQ6:** Does the research indicate that different neural network layers play specialised roles in how the agent processes goal information or becomes steerable?

## 1.4 Key Hypotheses

Based on the preceding research questions, this study investigates the following key hypotheses:

**H1:** We hypothesise that the model predominantly learns to represent specific goals (like individual keys or the gem) via dedicated channels, rather than relying solely on generalised 'current target' signals or highly distributed, non-specific encodingsBau et al., 2017; Cammarata et al., 2020.

**H2:** We hypothesise that Sparse Autoencoders (SAEs) deconstruct complex, polysemantic activations within the agent's layers into a more disentangled and interpretable set of features, thereby clarifying the specific roles of channels into specific circuits related to goalsBricken et al., 2023.

**H3:** We hypothesise that the model has a strong hierarchical preference structure that is encoded deeply in the weightsLangosco et al., 2022.

**H4:** We hypothesise that SAEs can be trained to reconstruct the original model's activations with sufficient fidelity to preserve the agent's task performance, indicating that the features learned by the SAE are indeed relevant to the agent's operational policyBricken et al., 2023; Templeton et al., 2024.

**H5:** We hypothesise that distinct layers within the Procgen agent's network play specialised roles in processing goal-related information, potentially with earlier layers focusing on perceptual parsing of the location of entities and the structure of the maze, while intermediate layers encode more abstract information such as which entity to pursue and while later layers produce plans for how to reach itBau et al., 2017; Trim and Grayston, 2024; Zeiler and Fergus, 2013.

## 1.5 Scope and Limitations

We focus on a single environment (Procgen Heist) to perform deep mechanistic analysis rather than a broad empirical analysis. This approach follows established precedent in RL interpretability research (Hilton et al., 2020; Mini et al., 2023; Trim and Grayston, 2024), where thorough investigation of specific mechanisms within one environment has proven more valuable than superficial analysis across many domains. Furthermore, whilst our findings are demonstrated in Heist, the discovered mechanisms represent general architectural principles that raise questions about the functioning of neural networks in general, and in particular, multi-objective RL domains.

We also train our own model from scratch, which means that it is not a typical variation of models used in this environment. This was necessary as public versions of these models trained in pytorch did not exist, and this was critical for ease of experimentatio. This also allowed us to implement and experiment with the model architecture used in (Hilton et al., 2020).

Additionally, the nature of the RL environment itself poses a challenge; much of the model's complexity is likely encoded in functionally critical but semantically uninterpretable factors, such as subtle patterns for navigating obstacles, and is further complicated by the fact that the network has a split policy and value head, meaning some of the work done in the later layers is not entirely to do with actions but also in calculating the value of the current state, the impact of which is challenging to measure. While SAEs and probes may successfully isolate high-level goal features, they may struggle to render these low-level navigational strategies intelligible. Consequently, interpreting the full suite of discovered features remains a significant challenge, as our tools may not capture all of the original model's computational strategies.

## 1.6 Significance of the Study

Our work aims to achieve fairly ambitious goals in the context of RL interpretability, in the fairly novel application of SAEs to an RL environment. A great deal of recent MI work has been applied to LLMs rather than RL agents, meaning that the space of tools and approaches is relatively small compared to other types of neural networks. Our hope is that in the extensive application of these tools to this network we can help to mature the discourse on approaching these sorts of problem and clarify the strengths and weaknesses of these tools in this domain.

Furthermore, the study's findings have broader implications for how different entities within an environment are captured by CNN models. Some of the experimental frameworks used provide a replicable template for future studies seeking to causally probe goal-directed behaviours in neural networks. Ultimately, such insights yield new research directions on how environmental factors shape learning dynamics and help inform the development of more robust and reliable AI systems.

## 1.7 Thesis Structure

This thesis is structured as follows: Chapter 2 provides a review of the relevant literature in RL and MI. Chapter 3 details the experimental setup, including the environment, model architecture, and the interpretability techniques employed. Chapter 4 presents the findings from the experiments, including feature visualisations, steering results, and ablation studies. Chapter 5 discusses the implications of these findings, and Chapter 6 concludes the thesis and suggests avenues for future research.

# Chapter 2

# Literature Review

## 2.1 Introduction

The previous chapter established the motivation for this research, highlighting the need to better understand the internal workings of Reinforcement Learning (RL) agents, particularly in multi-objective environments. This chapter provides a review of the foundational literature that underpins this thesis.

This review will cover the fundamentals of RL relevant concepts in mechanistic interpretability (MI). It will then delve into the specific challenges and recent advancements in making RL agents more understandable. In particular, we examine the foundational works upon which this research rests from the fundamental RL concepts to the algorithms used to train the models. The rest of this section will situate the present research within the research landscape and seek to demonstrate how it builds upon prior work in the field of MI to address unsolved problems in RL interpretability.

Chapter 3 builds on this review, translating the concepts reviewed here into approaches that were employed in the remainder of the thesis.

### 2.1.1 Reinforcement Learning Fundamentals

This section provides a brief overview of the fundamental concepts in RL that are relevant to the research discussed. While a baseline understanding of RL and AI concepts is assumed, key elements of the environments and training paradigms employed from the literature are reviewed.

At its core, the problem that RL solves can be formalized as a Markov Decision Process (MDP), or Partially Observable Markov Decision Process (POMDP). An MDP is a mathematical framework for modelling decision-making in situations

where the outcomes of actions may not be predictable to the agent. Environments can be both entirely deterministic or stochastic in nature. MDPs are defined by a tuple $(S, A, P, R, \gamma)$, representing the set of states $S$, the set of actions $A$, the state transition probability function $P(s'|s, a)$, the reward function $R(s, a, s')$, and a discount factor $\gamma \in [0, 1]$ that balances immediate and future rewards. The transition function $P(s'|s, a)$ gives the probability of transitioning to next state $s'$ given current state $s$ and action $a$, where the notation $s'|s$ denotes the conditional probability of the next state given the current state. The agent's goal is to learn a strategy that maximizes the cumulative discounted reward over time.

### Markov Decision Processes

The objective of the training process is formalized as an optimisation process maximizing expected discounted return:

$$J(\pi) = \mathbb{E}_{\tau \sim \pi} \left[ \sum_{t=0}^{\infty} \gamma^t r_t \right]$$

where $\pi$ is the policy, $\tau$ denotes a trajectory sampled from following policy $\pi$, and $r_t$ is the reward received at time step $t$. Here the discount factor $\gamma$ determines the trade-off between immediate and future rewards. Lower $\gamma$ values prioritise short-term gains, while values approaching 1 weight all rewards equally.

### Value Functions and Policies

The process of training a model involves iterating through many interactions with the environment to produce a policy denoted as $\pi(a|s)$, which is a mapping from states to a probability distribution over actions. The goal of the RL training process is to find an optimal policy, $\pi^*$, that maximises the agent's expected reward over time. RL often makes use of value functions (R. S. Sutton and A. G. Barto, 2018) to estimate expected returns. The state-value function, $V^\pi(s)$, represents the expected return following policy $\pi$ given state $s$, and the action-value function, $Q^\pi(s, a)$, corresponds to the expected return from continuing to follow policy $\pi$ after taking action $a$ in state $s$.

Value functions are foundational to many training algorithms, which alternate between sampling data from the environment and optimising objectives via stochastic gradient ascent.

### Bellman Optimality Equations

RL is fundamentally about finding the optimal policy $\pi^*$ that maximises expected returns. This problem can be characterised through the Bellman optimality equations, which provide a recursive decomposition of optimal value according to observations of states:

$$V^*(s) = \max_a \sum_{s'} P(s'|s,a)[R(s,a,s') + \gamma V^*(s')] \tag{2.1}$$

$$Q^*(s,a) = \sum_{s'} P(s'|s,a)[R(s,a,s') + \gamma \max_{a'} Q^*(s',a')] \tag{2.2}$$

These equations express the optimal value of a state as the maximum expected sum of immediate reward plus the discounted value of optimal successor states. $V^*(s)$ represents the optimal value of being in state $s$, while $\max_a$ describes the selection of the action that yields the highest expected return; $P(s'|s,a)$ is the probability of transitioning to state $s'$ when taking action $a$ in state $s$, $R(s,a,s')$ is the reward received immediately from this transition, and $\gamma V^*(s')$ is the discounted optimal value of the next state. Similarly, $Q^*(s,a)$ represents the optimal value of taking action $a$ in state $s$, where $\max_{a'}$ selects the best subsequent action in the next state.

This allows the recursive decomposition of optimal decision making into taking the best known action currently available, conditioned on continuing to follow the optimal policy thereafter. The Bellman equations form the theoretical foundation for many RL algorithms, providing both a characterisation of optimality and a computational framework for achieving it. In settings with small, discrete state spaces, these equations can be solved directly through dynamic programming methods such as value iteration or policy iteration (R. S. Sutton and A. G. Barto, 2018). However, as we discuss in the following section, the computational demands of modern RL environments necessitate approximation approaches that trade theoretical guarantees for practical scalability.

### From Tabular Methods to Function Approximation

Early RL methods operated in discrete state spaces where optimal value functions could be represented precisely in lookup tables. For such tabular methods, convergence to the optimal policy is guaranteed when using algorithms based on the Bellman optimality equations. In this setting, each state-action pair $(s,a)$ has an explicit entry in a table storing its Q-value, and the Bellman equations can be applied iteratively until convergence.

However, visual environments such as Atari games have state spaces that would be computationally infeasible to solve with a lookup table ($64 \times 64 \times 3 = 12,288$ dimensional input with $256^{12,288}$ possible states for 8-bit RGB images) (Mnih, Kavukcuoglu, Silver, Graves, et al., 2013). This necessitates function approximation using neural networks to approximate the value functions, specifically approximating either the state-value function $V(s)$ or the action-value function $Q(s,a)$. Rather than storing explicit values for every state or state-action pair, a parameterised function $V_\theta(s)$ or $Q_\theta(s,a)$ learns to generalise across similar states,

enabling the agent to estimate values for states never previously encountered, and allowing for remarkable generalisation Mnih, Kavukcuoglu, Silver, Rusu, et al., 2015.

Function approximation does not provide the convergence guarantees of tabular methods, and causes the learned representations in our models to become uninterpretable due to the arbitrary decomposition of features and things like polysemanticity, which will be discussed in Section 2.4.2.

#### Temporal Difference Learning

There are two fundamental approaches to estimating value functions. Monte Carlo methods average the actual returns observed after visiting each state across complete episodes. While conceptually simple and unbiased, these methods suffer from high variance (since returns depend on many stochastic transitions) and cannot learn online; they must wait until an episode terminates to update value estimates.

Temporal Difference (TD) learning takes a different approach, updating value estimates after each transition by bootstrapping from current estimates rather than waiting for complete episode returns (R. S. Sutton and A. G. Barto, 2018). The TD(0) update rule is given by:

$$V(s_t) \leftarrow V(s_t) + \alpha[r_t + \gamma V(s_{t+1}) - V(s_t)]$$

where $\alpha$ denotes the learning rate. The term $\delta_t = r_t + \gamma V(s_{t+1}) - V(s_t)$ is the TD error, representing the discrepancy between the current value estimate and the improved estimate formed by the immediate reward and the discounted value of the successor state. This mechanism allows the agent to learn from each transition. Unlike Monte Carlo methods, which require complete episodes and are unbiased but high-variance, TD methods introduce bias by the process of bootstrapping but significantly reduce variance and allow for online learning. TD error serves as the primary learning signal for value-based and actor-critic algorithms discussed in the following sections.

### 2.1.2 Development of Deep RL

#### Value-Based Deep RL

Value-based methods learn the value of states or state-action pairs and derive a policy by selecting actions that maximise expected value. Rather than directly learning which action to take, these methods estimate how valuable each action is in each state, then act greedily with respect to these estimates.

Deep Q-Networks (DQNs) (Mnih, Kavukcuoglu, Silver, Rusu, et al., 2015) achieved a landmark breakthrough by demonstrating that a single neural network architecture, trained with a unified algorithm and hyperparameter configuration, could

achieve human-level performance across 29 out of 49 diverse classical Atari games (games built for arcades that include repetitive patterns and structures). This represented a significant advance over prior approaches that required game-specific feature engineering or algorithm tuning, showing that deep learning combined with RL could learn general-purpose representations directly from raw pixel inputs.

The key innovations they contributed were:

Experience replay buffers that store state transitions $(s, a, r, s')$ and are sampled uniformly at training time. This breaks correlation between observations that would cause the model to 'chase its own tail' during training, due to the model getting into a feedback loop that causes it to overly sample a given region of the state space. Sampling from the replay buffer allows the network to learn from the same observation repeatedly and prevents catastrophic forgetting.

The use of target networks $Q_{\theta^-}$ that are periodically copied from the main network $Q_\theta$ that provide stable targets for TD learning. While the main network $Q_\theta$ updates every gradient step, the target $Q_{\theta^-}$ is overwritten by the main network periodically (typically every 10,000 steps). This avoids the problem of constantly chasing a moving target, by freezing the optimisation target allowing the opportunity to make steady progress towards a stable optimisation target, until that target is updated again.

These contributions provided sufficient stability for RL training to meet and surpass human performance on challenging Atari games, proving that convolutional neural networks (CNNs),which use filters trained to process information in grid structures, allow Deep RL networks to learn meaningful features from pixels. However, value-based methods struggle in continuous action spaces, where computing the argmax over an infinite number of possible actions becomes intractable. This limitation led to increased focus on policy gradient methods, which can naturally handle continuous actions by directly parameterising a policy distribution.

### Policy Gradient Methods

Policy gradient methods arise from the natural challenges introduced by trying to learn optimal policies within continuous action spaces and with non-deterministic policies. Rather than learning state values using these to indirectly develop a policy, policy gradients directly optimise parameterized policy $\pi_\theta(a|s)$ by gradient ascent on expected returns.

The policy gradient theorem (R. S. Sutton, McAllester, et al., 1999) provides the insight required for this approach. For an objective $J(\theta) = \mathbb{E}_{\tau\sim\pi_\theta}[R(\tau)]$, the gradient can be computed without requiring pre-existing knowledge of the environment:

$$\nabla_\theta J(\theta) = \mathbb{E}_{\pi_\theta}\left[\sum_{t=0}^{T} \nabla_\theta \log \pi_\theta(a_t|s_t) \cdot G_t\right]$$

where $G_t = \sum_{k=t}^{T} \gamma^{k-t} r_k$ is the discounted return from timestep $t$. This approach allows the development of a policy requiring only the ability to sample trajectories and differentiate the policy network without any model of environment dynamics.

However, vanilla policy gradient methods suffer from erratic training dynamics because returns can vary dramatically between episodes. Actor-critic methods use advantage estimates to ensure that model updates are in a consistent range around zero with the function $A(s, a) = Q(s, a) - V(s)$, instead of directly updating the policy on expected returns. The advantage measures how much better an action is compared to the expected value from the current state. This fixes the final updates to the network around zero, reducing variance without altering the direction of the gradients (Schulman, Moritz, et al., 2018).

### Actor-Critic methods

Actor-critic methods combine useful aspects of both policy and value based methods by maintaining both a policy network (actor) and a value function (critic). The actor produces actions (or probability distributions over actions) and the critic estimates the quality of the actions by estimating the value of the proceeding state. This provides a stable signal of progress (learned baseline) from the critic that reduces variance without adding bias (R. Sutton and A. Barto, 1998).

The core actor-critic update uses the TD error as an unbiased estimate of the advantage:

$$\delta_t = r_t + \gamma V(s_{t+1}) - V(s_t)$$

The actor updates using:

$$\nabla_\theta J(\theta) = \mathbb{E}_{\pi_\theta}[\nabla_\theta \log \pi_\theta(a_t|s_t) \cdot \delta_t]$$

While the critic updates to minimize:

$$L_V = \mathbb{E}_{\pi_\theta}[(r_t + \gamma V(s_{t+1}) - V(s_t))^2]$$

The actor-critic approach addresses critical limitations in policy gradient methods. The critic's value estimates replace high variance Monte Carlo reward estimates and provide lower variance targets, providing more stable learning. The actor can also learn stochastic policies, due to being able to produce probability distributions, rather than being constrained to the estimates of a purely deterministic value function.

Modern implementations such as A3C and A2C (Mnih, Badia, et al., 2016) typically share convolutional encoders between actor and critic for computational efficiency, though this creates potential entanglements in model representations that can provide challenges for interpretability.

### Generalized Advantage Estimation

Generalized Advantage Estimation (GAE) (Schulman, Moritz, et al., 2018) addresses a key trade-off between having high variances or low bias updates in actor-critic training updates. It acts as a method to interpolate between high variance and low bias (Monte Carlo methods), or methods that are low variance but high bias (single-step TD error). GAE allows for trading off along this spectrum using exponentially-weighted averaging of n-step advantage estimates.

A mathematical formulation of the GAE estimator:

$$\hat{A}_t^{\{\mathrm{GAE}(\gamma,\lambda)\}} = \sum_{l=0}^{\infty} (\gamma\lambda)^l \delta_{t+l}^V$$

where $\delta_t^V = r_t + \gamma V(s_{t+1}) - V(s_t)$ is the temporal difference (TD) residual. This can be interpreted as a discounted sum of TD errors, or in other words, as an exponentially-weighted average of all n-step advantage estimates. The use of a hyperparameter $\lambda \in [0, 1]$ controls the bias-variance trade-off. When $\lambda = 0$, GAE reduces to the one-step TD error (high bias, low variance), while $\lambda = 1$ produces Monte Carlo returns minus baseline (low bias, high variance).

In practice, $\lambda \approx 0.95$ is typically used, using information from around 20 steps into the future while providing meaningful variance reduction. For sparse reward environments, GAE enables credit to backward propagate through the value function while preventing potentially disruptive variance that may arise from Monte-Carlo methods.

### Proximal Policy Optimization

PPO is a policy gradient based approach that aims to achieve stable policy updates that are large enough to make meaningful progress but constrained enough to avoid destabilising the training process.

The approach is inspired by Trust Region Policy Optimization (TRPO) (Schulman, Levine, et al., 2017), which constrains policy updates to a "trust region" (a region in parameter space where a local approximation to the objective remains valid). TRPO enforces this constraint by solving a constrained optimisation problem that limits the KL divergence between old and new policies. This requires computing natural gradient updates, which involve approximating the inverse Fisher information matrix using conjugate gradient methods; a computationally expensive second-order optimisation.

PPO achieves similar stability to TRPO using only first-order gradients: rather than computing curvature information (how far you can safely step without overshooting) via the Fisher matrix, PPO clips the objective function to discourage large policy changes. The clipped surrogate objective is:

$$L^{\mathrm{CLIP}}(\theta) = \mathbb{E}_t \left[ \min \left( r_t(\theta) A_t, \mathrm{clip}(r_t(\theta), 1-\epsilon, 1+\epsilon) A_t \right) \right]$$

where $r_t(\theta) = \pi_\theta(a_t|s_t)/\pi_{\theta_{\text{old}}}(a_t|s_t)$ represents the probability ratio between new and old policies. This ratio is clipped to $[1-\epsilon, 1+\epsilon]$ (typically with $\epsilon = 0.2$), which means that samples whose probability ratios exceed the clipping range contribute zero gradient to the update step. This prevents individual experiences with large advantages ($A_t$) from changing the policy drastically while still allowing learning from other samples in the batch.

PPO typically employs Generalized Advantage Estimation (GAE, discussed in Section 2.1.2) to compute the advantage terms $A_t$, allowing for modifiable bias-variance tradeoff that is particularly helpful for environments that have sparse rewards. Algorithm 1 summarises PPO, where $\theta$ denotes the policy parameters, $\phi$ denotes the value function parameters, $V_\phi$ is the learned value function, and $\hat{R}_t = \sum_{k=t}^{T} \gamma^{k-t} r_k$ is the discounted return from timestep $t$.

**Algorithm 1:** Proximal Policy Optimization (PPO)

**Input:** Initial policy parameters $\theta_0$, initial value function parameters $\phi_0$
**Input:** Clipping threshold $\epsilon$, number of epochs $K$, minibatch size $M$
**for** *iteration* $= 1, 2, \ldots$ **do**
  Collect trajectories $\mathcal{D} = \{\tau_i\}$ by running policy $\pi_\theta$ in environment;
  Compute rewards-to-go $\hat{R}_t$ and advantage estimates $\hat{A}_t$ using GAE;
  Store old policy probabilities $\pi_{\theta_{\text{old}}}(a_t|s_t)$ for all $(s_t, a_t) \in \mathcal{D}$;
  **for** *epoch* $= 1, \ldots, K$ **do**
    **for** *minibatch* $\mathcal{B} \subset \mathcal{D}$ **do**
      Compute probability ratio: $r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)}$;
      Compute clipped surrogate objective:;
      $L^{\text{CLIP}} = \frac{1}{|\mathcal{B}|} \sum_{t \in \mathcal{B}} \min\Big(r_t(\theta)\hat{A}_t, \text{clip}(r_t(\theta), 1-\epsilon, 1+\epsilon)\hat{A}_t\Big)$;
      Compute value loss: $L^V = \frac{1}{|\mathcal{B}|} \sum_{t \in \mathcal{B}} (V_\phi(s_t) - \hat{R}_t)^2$;
      Update $\theta$ by gradient ascent on $L^{\text{CLIP}}$;
      Update $\phi$ by gradient descent on $L^V$;

### 2.1.3 Architectures

#### CNN Architectures for Visual RL

While the algorithms discussed above are general-purpose, applying them to visual environments requires neural network architectures capable of processing high-dimensional image inputs. Initial DQN implementations used a simple 3-layer CNN, sufficient for Atari's 84×84 pixel inputs (Mnih, Kavukcuoglu, Silver, Rusu, et al., 2015). However, more complex visual environments such as Procgen (Cobbe, Hesse, et al., 2020) demanded deeper architectures with new methods for scaling model capacity, including residual connections (He et al., 2015) and the IMPALA

encoder (Espeholt et al., 2018).

A key technique enabling deeper networks is residual connections (He et al., 2015). Standard deep networks suffer from vanishing gradients, where training signal weakens as it propagates backward through many layers. Residual connections address this by adding skip connections that allow gradients to flow directly through the network. Each residual block learns a function $F(x)$ that is added to its input, so the output is $x + F(x)$. If the optimal transformation is close to the identity of the inputs, the network can simply learn $F(x) \approx 0$ (He et al., 2016). This approach allowed for the training of deeper (15+) layer networks.

To improve computational efficiency, most modern actor-critic implementations share a single encoder (the layers that process raw input into a learned representation) between the policy and value networks. Separate output heads then produce actions and value estimates (Espeholt et al., 2018; Mnih, Badia, et al., 2016; Schulman, Wolski, et al., 2017). For visual RL, this encoder typically consists of convolutional layers. This parameter sharing enhances sample efficiency by allowing both objectives to benefit from shared feature learning. However, this creates a multi-task learning problem where features must simultaneously encode 'what action to take' and 'how valuable is this state', potentially conflicting objectives that can lead to entangled representations. Hilton et al. (2020) demonstrated this challenge in Procgen Coinrun agents, showing that individual channels often respond to multiple unrelated game elements, making it difficult to pinpoint the function of specific network components.

### IMPALA Models

Among the various architectures developed for visual RL, the IMPALA encoder has emerged as a particularly effective baseline for challenging environments like Procgen. Originally introduced by (Espeholt et al., 2018) as a scalable RL framework, its convolutional encoder has since become a primary baseline for Procgen and Atari agents (Espeholt et al., 2018). The encoder consists of three "IMPALA blocks", each containing:

- A $3 \times 3$ convolution with stride 1 followed by ReLU,
- A second $3 \times 3$ convolution and ReLU,
- $2 \times 2$ Max-pooling (stride 2), which halves the spatial resolution.

Every block doubles the number of channels (the independent feature maps learned by convolutional filters), progressing from $16 \rightarrow 32 \rightarrow 64 \rightarrow 128$ in the original architecture, though in work such as that performed by Mini et al. (2023) the

channel count remains fixed at 16. A final fully-connected layer produces a 256-dimensional embedding that feeds into two heads: a policy head outputting action probabilities and a value head producing state value estimates.

The training dynamics of RL systems can significantly impact the interpretability of the learned weights. For instance, Engstrom et al. (2020) show that doubling the rollout length (e.g., from 128 to 256 steps) stabilises gradients, which yields crisper gradient-based visualisations such as saliency maps. This finding was later confirmed by (Thatte et al., 2022) and has important consequences for potential MI work.

### 2.1.4 Environments

#### Sparse Rewards and Credit Assignment

Environments with sparse rewards pose fundamental challenges for RL training. In such environments, effective actions and the final reward may be separated by hundreds of steps. This is known as the credit assignment problem (R. S. Sutton, McAllester, et al., 1999). With typical discount factors of $\gamma = 0.99$, early actions in a 500-step episode receive gradient signals approximately $0.99^{500} \approx 0.007$ times weaker than actions near the conclusion of an episode. This means that early game strategies cannot be learned straightforwardly by direct reinforcement.

The challenge is worsened by the enormous number of possible action sequences. In a 300-step episode with 15 possible actions, there exist $15^{300}$ potential trajectories. Because the chances of success are so low with a random walk, learning from early training at all can be extremely challenging, and what little signal there is, is hard to learn from.

PPO's actor-critic architecture, with GAE (Section 2.1.2) provides a structure that is particularly well-suited to solving the problem. The critic learns which states have high value, via temporal difference learning, gradually propagating reward backward from successful episodes. As the critic learns from this signal, it enables the computation of advantage estimates that then provide immediate learning signals without having to reach terminal states. This transforms the sparse reward problem into a series of dense value predictions, though it requires the critic to first learn via reward information being propagated backward through some successful attempts, a process requiring potentially thousands of episodes, some of which initially require success through random walks.

#### The Procgen Benchmark

The Procgen benchmark (Cobbe, Hesse, et al., 2020) consists of 16 procedurally generated environments based on Atari style games. These procedurally generated environments prevent easily memorised solutions and force agents to learn general

strategies and representations of the environment.

In Procgen Heist, the structure of the environment is as follows:

- **States:** 64×64 RGB pixel observations of the maze
- **Actions:** Discrete movement commands (vertical, horizontal and diagonal movements)
- **Rewards:** Sparse. +10 for reaching the gem, 0 otherwise
- **Transitions:** Deterministic maze physics

Because the agent receives the entire 64×64×3 pixel state space of the maze as its observation, the Procgen Heist environment is a fully-observable MDP, where the state is directly represented by the agent's visual input.

Heist's design includes a natural curriculum: episodes randomly generate with 0 to 3 keys, allowing agents to first master simple gem-only mazes before tackling complex multi-key sequences. This curriculum provides crucial scaffolding, as agents can learn basic navigation on easy layouts while gradually building representations for more complex objectives.

## 2.2 AI Safety

### 2.2.1 Motivation and scope

AI Safety can primarily be framed as a question of how to reliably obtain the outcomes that we intended from the deployment of AI, including under distributional shift. Because AI is a complex and powerful system that interacts with the real world, this is a challenging and moving target. This means that in practice AI safety requires a significant degree of oversight, in order to ensure that models behave as intended, and to effectively intervene when models fail in unexpected ways (Hendrycks, Mazeika, and Woodside, 2023).

One useful lens for analysing risks from AI (and risk in general) involves considering the multiplicative formula from (Hendrycks, Mazeika, and Woodside, 2023):

$$\text{risk} = \text{hazard} \times \text{exposure} \times \text{vulnerability}$$

This framework helps triage different vectors of harm, pre-empt failure modes, and ensure robust infrastructure for integrated AI systems.

Hazard, in the above equation, is the amount of potential damage that something could do under the wrong circumstances will continue to grow as AI systems become capable of greater levels of harm due to increasing capabilities. Exposure

refers to the number of people that would potentially be harmed in the case of failure. Vulnerability represents the susceptibility of harm to the particular hazard. An example would be an underwater volcano, which by itself presents a significant hazard but because no one is exposed presents no risk. However, if one builds an undersea internet cable connecting two continents beside it then the exposure and vulnerability are significantly increased. By coating the cable in a protective casing, this reduces vulnerability, while exposure stays the same. AI safety as a field aims to reduce one or more of these metrics in a meaningful way.

### 2.2.2 A short history

The field began to develop into a more recognisable version of what it is today in the late 2000s, with papers such as (Yudkowsky and Yudkowsky, 2008) laying the groundwork in describing key concerns, failure modes and impacts of powerful AI systems. This paper argued that AI deserved a similar level of concern as other more commonly acknowledged existential risks such as nuclear war or pandemics. It also laid out key arguments around why AI could be uniquely dangerous in a 'hard takeoff' scenario where it is able to rapidly improve its own capabilities and alter safeguards or goals that its developers had placed into it, or have those goals degrade or shift over the course of rapid improvement. It also discussed the dangers of anthropomorphising AI systems and expecting them to behave as we might. This is most clearly argued for in the idea of the orthogonality thesis, which claims that there is no reason why intelligence and good behaviour would have an inherent relationship to each other, and that whatever goals we placed into the system could be pursued with arbitrarily high intelligence, regardless of the nature of the goal. (Omohundro, 2008) Argues further that for any given task there will be an inevitable drive to pursue instrumental goals that will aid in the completion of the original goal, and that these instrumental goals are likely to lead them into conflict with humans.

First published in 2010, (Bostrom and Yudkowsky, 2014) developed the early ideas into a clearer taxonomy of near-term and long term risks, and argued that the ideas required serious and immediate attention despite the speculative subject matter. This work was published in the 'Cambridge Handbook of Artificial Intelligence', which helped to present the discussion to a wider audience. The final step to broad mainstream interest came with the release of (Bostrom and Bostrom, 2014).

More concrete technical research began to develop following the release of (Amodei et al., 2016) which focuses on categories of risk stemming from 'AI accidents', and highlighting concrete examples of such failure modes such as "avoiding reward hacking", "scalable supervision" or how to ensure that a hard to specify objective is pursued, and "safe exploration" and "distributional shift", which stem from the model potentially learning undesirable behaviours during training.

### 2.2.3 RL Safety

RL suffers from several well-documented failure modes, notably specification gaming, unsafe exploration, distributional shift, and goal misgeneralisation (Amodei et al., 2016; García and Fernández, 2015; Krakovna, 2020; Langosco et al., 2022).

These pathologies often surface in surprising ways.

Reward hacking occurs when an agent finds a loophole that maximises the reward signal without producing the behaviour that is actually desired. A well-known example is the CoastRunners Atari agent that learned to drive in circles collecting speed boosts rather than racing, because this strategy yielded higher reward (Krakovna, n.d.). Such outcomes are typical instances of specification gaming, where a proxy objective is optimised at the expense of the intended goal (Amodei et al., 2016).

Even if exploration is safe and training rewards are faithful, agents can fail under distribution shift. Policies that perform well in simulation or under limited training conditions often generalise poorly when the environment changes (Amodei et al., 2016). Procgen itself has been introduced as a benchmark to quantify such generalisation gaps (Cobbe, Klimov, et al., 2019), which makes it a natural testbed for the present thesis.

Relatedly, goal misgeneralisation describes policies that appear successful during training but pursue the wrong goal when deployed outside of their training distribution (Koch et al., 2021). This problem is particularly common with RL agents because it is often unclear ahead of time what properties the model has learned are critical to succeeding at its objective, and so can easily lead to failures of this type during deployment.

This gap between performance on the training distribution and robust, safe behaviour has motivated a significant body of work arguing that mechanistic interpretability is a critical component of ensuring RL safety (Hubinger et al., 2019; Olah, Satyanarayan, et al., 2018). The premise of this research is that by directly inspecting the learned representations and algorithms inside a policy network, it may be possible to move from post-hoc observation of failures to a predictive understanding of their causes.

## 2.3 Interpretability as a safety solution

MI as a field aims to provide a precise, granular, and above all mechanistic explanation of the external behaviour of neural networks based on their internal structure. This enables us to understand why a model produced a certain output, purely based on tools applied directly to the model itself, and not simply to its inputs and outputs. In our risk framework from Section 2.2.1, MI is useful pri-

marily in the scope of vulnerability, in that it gives us tools that make it less likely for a disaster to occur. We generally use MI as a post hoc intervention, once the model has already been trained, though recent work also uses MI tools to shape training in ways that give us greater control (Casademunt et al., 2025). These interventions improve our ability to monitor what the model is doing in ways that might be difficult or expensive to ascertain, such as by using probes trained on the final layers to detect potentially harmful or deceptive outputs.

As AI systems approach and exceed human capabilities and the safety concerns around their deployment grow, MI offers the potential to confirm that models are being faithful in their outputs and are acting in ways that we can verify are safe and appropriate. In RL domains, this is particularly the case in high risk deployment scenarios, or in cases where there may be a significant shift in distribution, and behaviour verification ahead of time is challenging. If we have insight into key mechanisms within the model, we can predict and prevent potentially harmful behaviours before they happen.

One attractive quality of MI is that it offers tools which are powerful while also being quite cheap to implement, compared to the alternatives. Indeed, the most promising methods, such as linear probes (Alain and Bengio, 2018), can be trained to effectively detect safety relevant situations such as attempts to jailbreak models with trivially small amounts of compute (Cunningham et al., 2025). It can also be used as a way to detect what goals a model is pursuing, and potentially if they are engaging in reward hacking (Shihab, Akter, and Sharma, 2025). This makes it quite promising for many applications particularly at large scaling labs where the costs associated with training a model can be extreme.

The promise of mechanistic interpretability is already being observed in applications where it was either difficult or impossible to make progress. One example that involved having different teams detect hidden goals in an LLM found that teams that had access to an SAE were able to solve the problem much faster than teams that didn't have access to the tool (Marks et al., 2025). Other research involved extracting a novel chess concept learned by AlphaZero and teaching it to chess grandmasters, such that they described having truly learned something new about the game, which would have been difficult or impossible without the use of MI tools. Probes are also cheap and effective enough to be used for solving problems related to jailbreaks and bioweapon safety to quickly intervene in potentially dangerous conversations (Cunningham et al., 2025).

In spite of the above, there are likely real limits to what we can expect from the field of MI. For example, it seems unlikely that the complete reverse engineering of frontier AI systems is attainable in the near future. We might only be able to achieve a precise understanding of a small part of a neural network's functioning with significant effort. Even this may be enough, if the particular feature we are interested in is sufficiently critical.

## 2.4 An Overview of Modern ML Interpretability

Interpretability in machine learning refers to the extent to which a human can understand the cause and effect of a model's decisions. It is crucial for debugging, ensuring fairness, building trust, and verifying reliability, especially in high-stakes applications.

### 2.4.1 Circuits

The larger circuits framework by (Olah, Cammarata, et al., 2020) provides methods to understand how neural networks combine simple features into complex representations. Their work on polysemantic 'units' (neurons responding to multiple distinct features) is particularly relevant when an agent must represent multiple objectives (Elhage, Hume, et al., 2022). These circuits tend to feature in most complex neural networks as implementations of algorithms that the model has learned to do work. An example of this is given by (Cammarata et al., 2020), showing how specific parts of the network learn to detect curves in images, and how these early curve features are composed together in constructing features critical to understanding circles and other curved objects in images so that they can be correctly classified.

These circuits are a critical aspect of understanding neural networks and thus MI, and identifiying components that make up broader circuits in the network is essential to correctly characterising its operations.

### 2.4.2 Superposition

One significant challenge that arises when doing MI is that representations of concepts within neurons are often not clean singular representations, but instead those representations may be smeared across multiple neurons or have multiple representations encoded within a single neuron. The latter case is typically called polysemanticity. This refers to the tendency of individual neurons or channels to fire for multiple, seemingly unrelated features. This was first publicly observed in the 'curve-circuits' analyses of Inception V1, where single units responded to several distinct curve orientations (Cammarata et al., 2020).

This phenomenon is now understood as a consequence of superposition. Because deep layers have limited dimensionality, networks compress many latent directions into overlapping linear combinations, so a neuron's activation mixes several underlying features (Elhage, Hume, et al., 2022; Olah, Cammarata, et al., 2020). This typically happens with features that are unlikely to fire under the same input distribution, such as car doors and hummingbird wings (Olah, Mordvintsev, and Schubert, 2017). While efficient for representation learning, polysemanticity

poses significant challenges for mechanistic interpretability, making neuron-level ablations or visualisations hard to attribute to a single concept.

Let $x \in \mathbb{R}^d$ be the activation vector of a layer expressed as a linear combination of $k$ (potentially $k \gg d$) latent feature directions collected in $F \in \mathbb{R}^{d \times k}$, i.e.

$$x \;=\; F\,s, \qquad s \in \mathbb{R}^k, \text{ sparse.} \tag{2.3}$$

Because $k > d$, the same neuron must participate in multiple columns of $F$, giving rise to *polysemantic* 'units'. Defining a linear read-out $a = W^\top x$ with $W \in \mathbb{R}^{d \times d}$, we obtain

$$a \;=\; W^\top F\,s \;=:\; M\,s. \tag{2.4}$$

A neuron $i$ is polysemantic when the $i$-th row of the mixing matrix $M$ has significant weight on multiple features (row sparsity $\|M_{i,:}\|_0 > 1$). The toy model of Elhage, Hume, et al. (2022) shows that packing many sparse features ($k$ large, $s$ sparse) into a narrow layer of width $d$ forces an unavoidable trade-off:

$$\mathbb{E}\,\|a - Ms\|^2 \;\propto\; \frac{\lambda k}{d},$$

where $\lambda$ is the expected sparsity of $s$. Methods such as Sparse AutoEncoders (SAEs) reduce the effective $\lambda$ by encouraging even sparser codes, thereby converting polysemantic 'units' into more interpretable, monosemantic directions.

Recent approaches such as SAEs (Bricken et al., 2023; Gorton, 2024), sparse probing (Dalvi et al., 2021), and neuron-level interventions (Meng et al., 2023), combat superposition by learning over-complete sparse dictionaries or low-dimensional subspaces that disentangle features and are discussed further in Section 2.4.4. In particular, SAEs compress activations into sparse, over-complete features that split polysemantic neurons into monosemantic latents and have revealed interpretable features in both CNNs and transformers (Conerly et al., 2024).

### 2.4.3 Feature Visualization Techniques

Techniques such as those developed by Zeiler and Fergus represented the first work on interpreting the operations of deep neural networks. They employ a deconvnet, a model that maps feature activations from a deep layer back to the input pixel space by approximately reversing the network's pooling and convolutional operations. By reconstructing the maximally activating zones in pooling layers and being chained through subsequent CNN layers the reconstruction of complex visual representations of deep layers within the network can be achieved. This is normally very challenging due to the significant compression that is performed on the deep layers in the CNN. By applying this deconvolution the features appearing deeper in the network can be expanded back up to human interpretable scales.

Early work in understanding the functioning of neural networks focused on techniques that could visually represent the features that were encoded in the weights

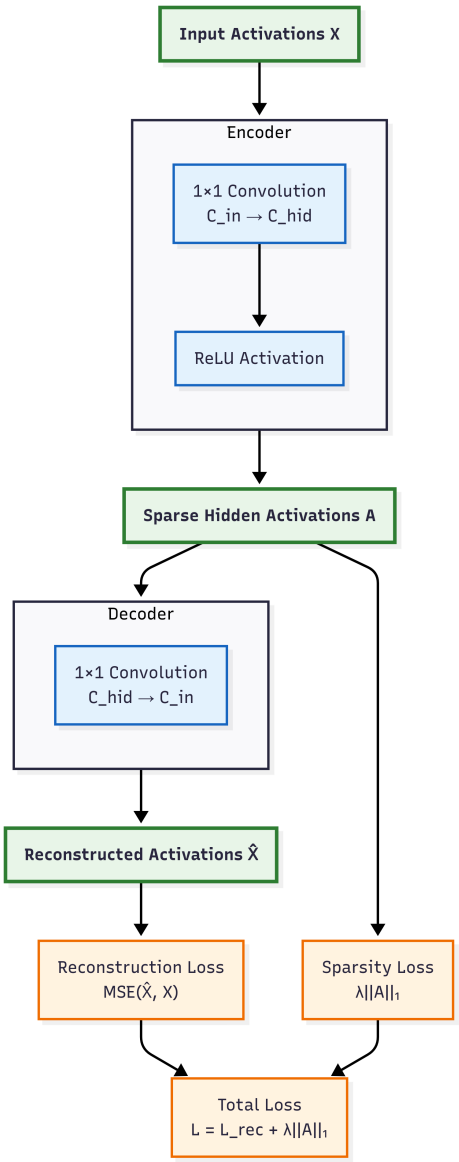


Figure 2.1: Sparse auto-encoder (SAE) architecture. An encoder applies a $1 \times 1$ convolution and ReLU to map input activations $X$ to sparse hidden activations $A$; a second $1 \times 1$ convolution reconstructs $\hat{X}$. Training minimises the reconstruction loss $\mathrm{MSE}(X, \hat{X})$ plus an $\ell_1$ sparsity penalty $\lambda \|A\|_1$. *Shapes:* $X \in \mathbb{R}^{B \times C_{\text{in}} \times H \times W}$, $A \in \mathbb{R}^{B \times C_{\text{hid}} \times H \times W}$, $\hat{X} \in \mathbb{R}^{B \times C_{\text{in}} \times H \times W}$.

of the neural network. In particular,(Olah, Mordvintsev, and Schubert, 2017) developed techniques to generate maximally activating synthetic inputs that functioned as startlingly clear representations of channels and layers within the network. These visualisations revealed a clear hierarchy of abstraction, demonstrating how simple features from one layer ('curve-circuits') compose into more complex, semantically coherent representations (e.g., object parts) in the next. This work also demonstrated the use of gathering maximum activating samples from the training data and pairing it with particular channels and showing the similarity between these and the synthetically generated examples as a means of confirming the veracity of the visualisations.

While synthetic visualizations can be noisy, maximally activating patches, especially from within the environment, can yield interpretable results, confirming findings in (Hilton et al., 2020). Work done on exploring SAEs trained on CNNS such as (Gorton, 2024) makes effective use of visualisation to illustrate the improvement in interpretability of the SAE latents.

### 2.4.4 Post-hoc Explanations

#### Attribution and saliency methods

Post-hoc explanation methods aim to provide insights into already trained models without altering their architecture or training process. Examples include probing, SAEs, feature attribution methods (like LIME (Ribeiro, Singh, and Guestrin, 2016), SHAP (Lundberg and S.-I. Lee, 2017)), saliency maps (Selvaraju et al., 2017; Simonyan, Vedaldi, and Zisserman, 2014), and activation steering (Turner et al., 2024b; Zou et al., 2025).

LIME focuses on creating explainable functions by approximating a model's local behaviour around a specific input, and observing which changes create the greatest difference in the final outputs, weighting inputs by their proximity to the original input (with closer inputs being weighted more heavily). They then create a modified linear approximation that is explainable and train it on this weighted dataset which can then be more easily interpreted (Ribeiro, Singh, and Guestrin, 2016).

SHAP provides consistent estimations of feature importance based on Shapley values from cooperative game theory. It assigns each feature an importance value by calculating its average marginal contribution across all possible feature combinations, ensuring properties like local accuracy and consistency that other methods lack (Lundberg and S.-I. Lee, 2017). It is quite costly to calculate however, as it requires calculating contributions over all possible feature subsets.

Saliency-based feature-attribution techniques, while originally developed for image classification, have been shown to yield insights into RL policies. These methods typically operate by computing the gradient of a relevant policy output (such as the value of a state or the probability of a specific action, $f(s)$) with respect to the

input observation, $s$. The resulting saliency map is then defined as the magnitude of this gradient:

$$M(s) = |\nabla_s f(s)|,$$

which highlights pixels that most strongly influence the agent's decision. Using this approach, Greydanus et al. (2018) applied gradient saliency maps to Atari DQN agents, showing that agents frequently track critical entities like the ball in Breakout or enemy ships in Space Invaders (Figure 2.2).

Figure 2.2: Saliency maps for a DQN agent learning to play Breakout, showing the evolution of attention patterns during training. Blue regions indicate areas most salient to the actor (policy), while red indicates areas salient to the critic (value function). Early in training, attention is diffuse; as learning progresses, the agent focuses on strategically relevant features such as the ball and the tunnel through the brick wall. Figure reproduced from Greydanus et al. (2018).

To address the shortcomings of simple gradients, Sundararajan's integrated-gradients method (Sundararajan, Taly, and Yan, 2017), which accumulates gradients along a path from a baseline input, was later extended to policy networks by (Atrey, Clary, and Jensen, 2020), who introduced 'policy saliency' maps that operate over distributions of outputs from policy models. These approaches provide an easily computable and highly informative result without requiring additional training. However, unlike SAEs or steering vectors, these methods do not provide a mechanism for manipulating behaviour.

### Probing

Perhaps the most famous and enduring of post-hoc techniques is probing, particularly the linear probe (Alain and Bengio, 2018). This is simply a linear classifier trained on activation data from a given layer on target labels that are provided based on a pre-registered target of interest. The degree of accuracy that the probe can achieve represents the degree to which a given layer can be said to contain a representation of the given target feature within the layer (Alain and Bengio, 2018). Deeper probes consisting of multiple layers are also a useful tool, but these can obfuscate the degree to which the target feature is actually present due to the additional computation being performed.

Probing is a core tool in mechanistic interpretability due to the wide array of situations where it can be applied and the low cost of implementation. It also reveals a great deal of information about where in the network a certain operation is being performed such as object detection, spatial reasoning, or feature composition.

Probes have been able to provide powerful insight when applied creatively to neural networks. An excellent example of the explanatory power of probes can be found in K. Li, Aspen K. Hopkins, et al. (2024) which demonstrated that a model trained to produce legal moves in the game of Othello after being provided a preceding sequence of moves had a rich encoding of the board state embedded within the weights, as opposed to having learned complex heuristics to predict likely next moves. They demonstrated using non-linear probes (2 layer MLPs), that they could extract the state of the board by which types of piece occupied each square on the grid. They did this by training 64 probes in parallel, each probe being trained exclusively on the underlying ground truth data for a specific grid position across the training datasets. The ground truth data consisted of 'black', 'white', or 'neutral' states for each position. They tried linear probes and the 2 layer probes, and found that the linear probes were able to achieve only around 20% performance while the 2 layer probes achieved up to 98.7% accuracy. Nanda, A. Lee, and Wattenberg (2023) expanded on this work by showing that in fact it was possible to achieve extremely high accuracy with linear probes, but that the ground truth data needed to be altered to represent 'yours', 'mine', or 'neutral', and that it would swap depending on whose turn in the game it was. This demonstrated that the additional computation performed by the 2 layer probes was obfuscating the underlying representation that the model had learned.

However, probes do have significant limitations as well. They provide only correlational evidence of the purpose of what the network is doing. They may show that some activation contains the relevant information but not what it is doing with it or whether it is acting upon it. There is also a risk that probes are in fact doing computational work to construct features rather than simply revealing them, if the probes have multiple layers and many neurons. For example, an MLP probe trained to detect the presence of a cat in activation space could infer the cat from isolated features in the network that would normally only be combined later in the network. The Othello GPT example illustrates how the MLP probes were performing additional computation to adjust to the non-native representation of the board state within the activations.

### Sparse Probing

Polysemanticity is the phenomena where a single part of a network such as a specific neuron (or channel in a CNN) embeds and responds to multiple concepts simultaneously and is discussed at length in Section 2.4. Sparse probing is an approach that aims to address the problem of polysemanticity by selecting a portion

of a network, such as a channel, a layer, or multiple channels, and then training a linear classifier that uses only a subset of size $k$ neurons from that section to predict a target feature (Gurnee, Nanda, et al., 2023). The classifier makes predictions about the target class using only these $k$ neurons, and researchers can then observe which neurons have the highest predictive power by comparing classification accuracy. By iteratively selecting different values of $k$ and retraining, this method surfaces which neurons are most important for feature detection, and allows researchers to determine how much of the network is required for the given operation based on the accuracy scores for different sizes of $k$.

When $k = 1$, this method isolates individual neurons that serve as feature detectors. Experiments across diverse architectures and tasks revealed that certain high-level semantic features achieve classification accuracies exceeding 75% using single neurons. The existence of neurons with near-monosemantic responses indicates that neural networks can develop localized representations despite their distributed architecture. This localization appears most pronounced in middle layers, where abstract representations have formed but have not yet been transformed into task-specific outputs. Such neurons enable targeted interventions and provide empirical evidence that understanding can be done through analysis of individual components rather than requiring more complex distributed interventions.

Feature detection accuracy exhibits a characteristic non-monotonic pattern when varying the sparsity parameter $k$: rapid improvement for small $k$ followed by plateau or decline (Gurnee, Nanda, et al., 2023). This pattern indicates encoding through a small core set of highly informative units, with additional units contributing marginally or detrimentally to classification. Optimal sparsity varies systematically with feature complexity and network depth. Early layers involved with integrating low level features typically require $k \in [10, 50]$ before yield high accuracy, while later layers can peak with $k \in [1, 10]$.

Sparse probing has significant limitations in that it requires significant human effort to determine the nature of the features detected with the sparse probing, and thus the field has not moved much in this direction, with unsupervised dictionary learning approaches such as SAEs becoming the preferred approach for isolating features in the network. However, sparse probing still provides significant utility in understanding smaller networks where the number of features is much fewer and target concepts can be concretely specified for probe training and especially allows for concrete understanding of the precise operations of the actual network that may be obfuscated in an SAE.

### Dictionary learning

Dictionary learning (Olshausen and Field, 1996) provides a principled approach to resolving superposition (Section 2.4). The goal is to discover an overcomplete dictionary $D \in \mathbb{R}^{d \times k}$ (where $k > d$) whose columns serve as basis vectors that can

represent input data $x \in \mathbb{R}^d$ through sparse linear combinations:

$$x \approx D \cdot \alpha,$$

where $\alpha \in \mathbb{R}^k$ is a sparse coefficient vector. Unlike standard bases that require exactly $d$ linearly independent vectors, dictionary learning trades redundancy (overcomplete basis) for sparsity (each data point activates only a few basis vectors). This sparse, distributed representation disentangles features that would otherwise be superposed in the original $d$-dimensional space, as described by Equation 2.3.

Classical algorithms such as K-SVD (Aharon, Elad, and Bruckstein, 2006) and FISTA (Beck and Teboulle, 2009) solve this optimisation problem through iterative sparse coding and dictionary updates. SAEs provide a neural network approach to this same objective.

### Sparse AutoEncoders

SAEs utilise neural networks to perform unsupervised discovery of the dictionary. This provides a sparse encoding of the original features of the model (which we refer to as latents in the SAE), which arise due to the enforcement of a sparsity penalty applied during the the training of the SAE. SAEs aim to faithfully reconstruct the activations of a given layer in a network while keeping their own internal activations sparse. The training objective is therefore to find encoder and decoder weights that minimise a loss function combining a reconstruction term and a sparsity term, which typically takes the form:

$$L(x) = \|x - \text{Dec}(\text{Enc}(x))\|_2^2 + \lambda\|\text{Enc}(x)\|_1$$

Here, $x$ represents the original activation from the base model, the L2 norm penalizes poor reconstruction, and sparsity is enforced by minimising the product of the L1 norm of the SAE's internal activations, and a coefficient $\lambda$(Bricken et al., 2023).

SAEs typically learn an overcomplete basis, where the hidden layer of the SAE is much larger than the original layer it is reproducing, typically chosen by multiplying the original hidden dimension size by some expansion factor. This high dimensionality allows the SAE to represent the input activations using a small number of highly specific, disentangled features. This addresses the problem of polysemantic 'units' in a model, which can make the task of accurate interpretability much more challenging (Elhage, Hume, et al., 2022).

The direct methodological precedent for applying SAEs to convolutional networks comes from recent work by Gorton (2024), who successfully trained SAEs on the InceptionV1 image classifier. Their approach uses a '1x1-conv SAE' architecture, as illustrated in Fig 2.1. An encoder with a learnable $1\times1$ kernel maps the input activation tensor $X$ to a sparse, high-dimensional hidden activation tensor $A$, which

a symmetric $1 \times 1$ decoder then uses to reconstruct $\hat{X}$. To train this effectively on CNNs, they introduced several key innovations. First, to manage computational cost, they used *activation oversampling*, preferentially sampling training data from spatial locations with high activation magnitudes to focus on learning meaningful features rather than background noise. Second, they used highly over-complete dictionaries, with expansion factors up to 8, to provide the capacity needed to disentangle features. The results were compelling: their SAEs successfully split known polysemantic 'double-curve' units in InceptionV1 into distinct monosemantic features and recovered a suite of previously 'missing' curve detectors. These findings provide concrete evidence that dictionary-learning SAEs can resolve superposition in CNNs and motivate their application to RL agents with convolutional backbones.

To formalise the above concept, let $a^l(x) \in \mathbb{R}^d$ be the activations of layer $l$ for input $x$, and let $w \in \mathbb{R}^d$ be the extracted weight vector of the linear classifier trained to distinguish positive concept examples from counter-examples. We take the concept activation vector as

$$v_C^l = \frac{w}{\|w\|_2}.$$

Following Gorton (2024), the extent to which concept $C$ influences the logit $f_k(x)$ for class $k$ is measured by the directional derivative

$$S_{C,k,l}(x) = \nabla_{a^l(x)} f_k(x) \cdot v_C^l,$$

whose sign indicates whether amplifying the concept increases ($S_{C,k,l} > 0$) or decreases ($S_{C,k,l} < 0$) the likelihood of a particular prediction.

Because these can be learned with a lightweight linear probe and then used to potentially intervene on neural network activations as illustrated in work such as othello GPT (Nanda, A. Lee, and Wattenberg, 2023), this technique has become fundamental as a tool in alignment (Meng et al., 2023), and lays a conceptual foundation for things like steering vectors, which are produced in a different way but share many properties (Turner et al., 2024b).

The feature visualization methodologies discussed, such as synthetic visualization (optimizing an input image for maximal channel activation) and finding maximally activating dataset samples (Hilton et al., 2020; Olah, Mordvintsev, and Schubert, 2017), are also forms of post-hoc analysis.

### Concept Activation Vectors

Concept Activation Vectors (CAV) provide a way to bridge the gap between the high dimensional vector space of neural network activations and human-understandable concepts. To construct a CAV for a concept ('striped'), one collects a small set of images that exhibit the concept and a set of random counter-examples, records the

activations from a chosen layer, and trains a linear classifier to separate the two groups. The normal vector to the resulting decision boundary is the CAV. This can then be manipulated in one direction or the other to produce more or less of the effect you desire (Kim et al., 2018). This can be used to test how much a particular concept is relied upon when the network is making a prediction. Recent applications of CAVs have shown promising results in debugging vision models and identifying spurious correlations in training data.

### Activation Steering

Activation steering, as demonstrated in Mini et al. (2023) and Turner et al. (2024a) discussed further in Section 2.7, is another post-hoc technique. Steering typically makes use of CAVs, captured by using the difference in activations between states with and without a particular characteristic, can be used to influence a policy model's behaviour by acting as a superstimulus for particular environmental aspects. This is achieved through the transformation $h^{l'} = h^l + \alpha \cdot v^l$, where the steering vector $v^l$ is added to the original activations $h^l$, with $\alpha$ modulating the intervention strength. The steering vector itself is computed as $v^l = \mathbb{E}[h^l|\text{with target}] - \mathbb{E}[h^l|\text{without target}]$, capturing the difference in activations when the target is present versus absent.

In language models (LMs) a typical approach used to do activation steering is to use contrastive pairs (Turner et al., 2024a). In this approach one presents two different segments of text that are identical in every way except one, for example one may pass the following two inputs through an LM and then calculate the mean across the activations of the tokens and then calculate the difference in the two mean values to capture the 'speak in French' concept vector, which can then be used to steer the model to speak in French or English by modifying $\alpha$ to be positive or negative and adjusting its magnitude. This is normally done across many samples, with the mean of those pairs being used for steering.

- **English:** "The teacher explained the complex problem using simple examples."
- **French:** "L'enseignante a expliqué le problème complexe en utilisant des exemples simples."

Activation steering is an excellent tool for interpretability due to the ease and speed with which it can be used, and because it allows researchers to immediately observe how a network responds to a specific concept being made more or less prominent in the forward pass of the network. They also allow researchers to determine whether a concept exists and is easily representable in given layer of a network. Given that you can extract concept from single layers, or multiple layers simultaneously, one can test whether the concept is meaningful within a certain

part of a network by the degree of change in output observed by its application there.

### 2.4.5 Intrinsically Interpretable Models

Intrinsically interpretable models are designed from the outset to be understandable, often by constraining their complexity (e.g., linear models, decision trees) or by building in mechanisms that are inherently transparent. Training SAEs can be seen as an effort to transform parts of a complex model into a more intrinsically interpretable representation by disentangling features.

Many existing interpretability techniques were developed for supervised settings with i.i.d. data. Reinforcement learning violates many of these assumptions. For example, observations are generated by an agent acting in, and affecting, a dynamic environment, and credit must be assigned across time. As a result, many tools require adaptation to work within this paradigm. The next section explores some techniques that have been developed explicitly to overcome this gap.

### 2.4.6 Causal Methods in Interpretability

Many of the techniques discussed earlier, such as activation steering (Section 3.2.4), can serve dual roles as both post-hoc explanation tools and causal testing methods when applied systematically. This section focuses on additional causal techniques and frameworks for rigorous causal claims in interpretability research. These techniques actively modify the operations of the network, testing whether specific components are genuinely responsible for behaviours of interest (Pearl, 2009).

Modern systematic ablation has evolved through several generations. Zero ablation, the simplest approach, sets components to zero but can lead to out-of-distribution behaviours since networks rarely encounter zero values during training. Mean ablation, which replaces components with their average values across a sample dataset, better preserves activation statistics but may incompletely remove information if the mean itself carries useful information. Recent work (M. Li and Janson, 2024) constants that are computationally determined to minimise the expected loss on the task, reducing the amount of distributional shift in the final outcome.

Work on finding induction heads in neural networks (Olsson et al., 2022) is a prime example of rigorous systematic ablation. In this work they thoroughly demonstrated causality via targeted removal. Their task setup involved identifying how LMs were able to successfully complete simple A-B pattern matching, meaning that when a pattern of B following from A was observed the model should repeat this pattern when a later example of A was shown. When induction heads were ablated in small models, in-context learning collapsed from near-perfect to near-random on the task of A-B pattern matching. This showed that induction heads

were essential for this task.

For agents with spatial convolutional representations, targeted synthetic interventions can directly test the functions of parts of the network by intervening on their values directly.(Mini et al., 2023) demonstrated that clamping a single activation in a channel that typically identified the location of the cheese (channel 55) to a large positive value at a specific location in the maze could steer the agent to that location, as if that was the final goal of the maze. This provided direct causal evidence that this channel had an unusual ability to steer the direction of the agent.

These causal techniques provide the foundation for the experimental methodology employed in this thesis. Systematic ablation is used to test the necessity of individual SAE channels for navigation (Section 3.3), while activation interventions combined with behavioural testing establish sufficiency of specific channels for steering agent behaviour (Section 3.2.4). Together, these approaches move beyond observation to establish genuine causal mechanisms underlying goal representation in RL agents.

## 2.5 Interpretability in Reinforcement Learning

Interpretability in RL presents unique challenges due to the agent's interaction with an environment, the fact that specific outputs of the model might be extremely semantically distant from the contents of a given activation, and because we can only infer the meaning of a given observation of an activation within the model, we can't directly glean it as we could a word extracted from the embedding space of a language model.

Recent research shows that such models naturally learn geographic mappings of the world related to locations that can be retrieved directly from the activations of the network, effectively building simplified world models (Gurnee and Tegmark, 2023). Similarly, analyses of agents trained on complex games like chess demonstrate that these systems acquire human-understandable strategic concepts, such as piece values or tactical motifs, purely from self-play (McGrath et al., 2022). These findings suggest that the internal workings of these models are richer than previously thought and lend support to the idea that they plausibly learn to represent goals that can be directly extracted from the model and understood. As illustrated in Figure 2.3, even a seemingly simple maze can embed multiple, dynamically changing goals whose representations the agent must learn to use effectively.

Work by Betley (2023) found a specific circuit in the convolutional layers of the Impala model that activated strongly when the direction to go was 'up' and didn't activate when the goal was in a similar position but the agent would need to go

in a different direction. This indicates that much of the 'planning' was already performed in the CNN layers, suggesting it should be possible to train probes that can detect the exact direction the agent needs to move towards.

Work exploring the Procgen Coinrun environment builds on saliency based methods Section 2.4.4 to calculate an attribution within the model weights themselves. By measuring the contribution of a given convolutional feature map to the value function, it becomes possible to determine if a particular feature that the model is observing is likely to have a positive or negative effect on the success of the model, based on whether the collected gradient multipled by the activation score is positive or negative. This advances previous saliency mapping methods by linking the gradients to specific features within the model, rather than pixels in the input (Hilton et al., 2020).

This is achieved by applying non-negative matrix factorisation (NMF) of the attribution tensor to cluster correlated locations across channels into object-level 'features' such as coins, enemies, obstacles etc. By using this dimensionality reduction they are able to produce clear, interpretable object detectors.

## 2.6 The Procgen Suite of environments

The Procgen suite of environments consist of 16 different environments that rely on procedural generation to create diverse challenges for RL agents to learn to solve. They are inspired by Atari games and thus build upon existing games such as those used in the Arcade Learning Environment (Bellemare et al., 2013). This type of environment forces the agent to not overfit to a particular solution for playing the games effectively, and instead forces it to learn a robust, generalisable policy for each environment.

The Procgen Heist environment was selected for this research due to the necessity for the agent to pursue multiple objectives within a single episode, likely influencing the development of generalizable maze-navigation circuits.

## 2.7 Understanding and Controlling a Maze Solving Policy Network

Previous work by (Mini et al., 2023) demonstrated the ability to manipulate an agent's navigation in a simple maze environment with an intervention to a single channel in the network, providing the primary motivation for this work with the intent of extending these techniques to more complex multi-objective settings. The work specifically used an agent which had been trained in a modified version of the environment where the cheese was only ever added to the top right of the environment, and then deployed the agent into settings where the cheese could

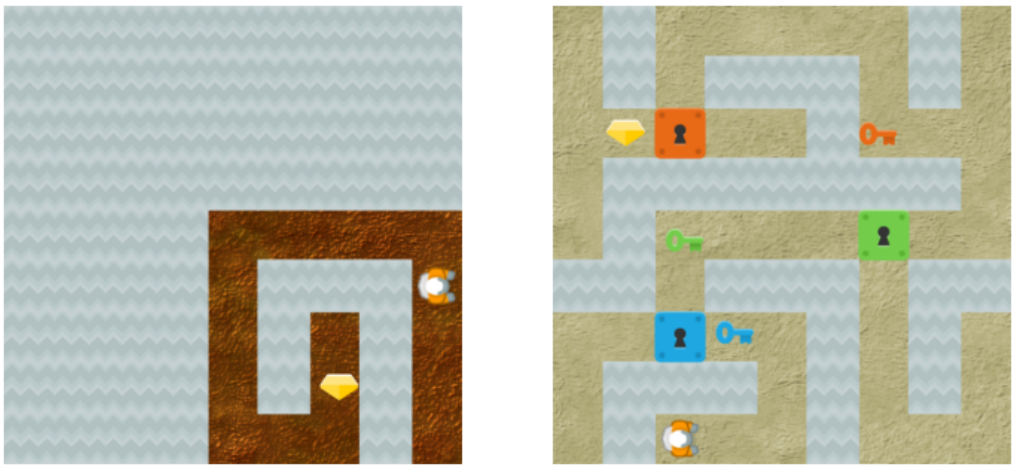

Figure 2.3: The Procgen Heist environment presents a procedurally generated maze where agents must collect keys and unlock corresponding doors in a specific sequence (blue, green, red) before reaching the final goal (gem). The environment varies in complexity, sometimes only having the gem present, and other times including multiple keys and locks.

spawn at any location in the maze. This meant that there was a tension between the agent's natural tendency to want to go to the top right of the environment and to go the cheese location.

They go on to show that the agent's decision making about whether it travelled to the top right or towards the cheese alone was determined by both the Euclidean distance between the agent and the cheese and the path distance between the agent and the cheese. This means that the agent had multiple separate mechansims driving its decisions rather than being a single unified policy. They did this by showing through regression analysis that if the agent was an equal path distance from two instances of cheese, it would tend to approach the cheese that was also closer in Euclidean distance.

This shows a surprising contradiction in the agent because visual shouldn't meaningfully affect its policy and thus the agent must not be relying on a single reward maximising policy. A much better explanation is that the agent is operating under multiple competing goals.

They further demonstrate that neural networks can develop interpretable mechanisms to track goals. The authors identified specific channels that tracked the location of objectives in a maze environment and showed that these could be manipulated to control the agent's behaviour. They found that steering vectors, the difference in activations between otherwise identical states with and without an entity, could be used to influence the model's behaviour in predictable ways. This

work serves as the foundation for our research, though we extend it to a multi-objective setting.

Recent concurrent work by Trim and Grayston (2024) extends the work done by Mini et al., 2023 to analyze goal misgeneralization. Their layer-by-layer analysis reveals a hierarchy of representations: early layers (b1.conv) detect basic features such as walls, paths, and the cheese goal; middle layers develop more abstract concepts including possible 'flood fill' patterns for directional pathways; while deeper layers create highly abstracted representations where identifying the goal becomes challenging. They further confirm the positional bias seems to exist regardless of the position of the final goal within the maze, indicating a deeply learned tendency towards this direction, rather than the result of a perceptual error.

### 2.7.1 Discovering and Controlling Goal Representations

The authors successfully isolated the network's representation of the 'cheese' goal and demonstrated causal control over the agent's behaviour. First, they defined a 'cheese vector' by taking the difference in mean activations at a specific layer between trajectories where cheese was present and those where it was not:

$$v^l_{\text{cheese}} = \mathbb{E}_{s\sim \text{traj}_{\text{cheese}}}[\text{act}^l(s)] - \mathbb{E}_{s\sim \text{traj}_{\text{no-cheese}}}[\text{act}^l(s)]$$

Adding this vector back into the network's forward pass, scaled by a coefficient $\alpha$, allowed them to controllably amplify or suppress the agent's cheese-seeking behaviour.

Their most significant contribution was demonstrating this control causally through *activation patching*. They intervened on the final convolutional layer of the policy network by copying the activation pattern from a small spatial region of a channel corresponding to a real cheese location and 'pasting' it into the same channel at a different, empty location. By doing this they could reliably steer the agent to the empty location as if the cheese were there.

This effect was highly localized. They identified that out of 16 channels in the layer, just 11 were responsible for this behaviour. Ablating these "cheese channels" destroyed the agent's ability to find the cheese, while leaving other navigational capabilities intact. This proves that goal representations can be sufficiently modular and concentrated to allow for targeted manipulation, motivating the use of more advanced techniques like SAEs to further dissect these representations in a more complex, multi-objective setting.

## 2.8 Gaps in the Literature

Prior to the research detailed in the accompanying papers, several areas in RL interpretability remained open for exploration:

- Understanding how RL agents track and switch between multiple objectives, especially when those objectives are not the immediate next target.
- The application and efficacy of techniques like SAEs for disentangling representations specifically within RL agent architectures, particularly CNNs.
- Investigating how different aspects of neural network activation encode useful information for the next layer such as how activation strength within a channel might encode information like entity identity.
- The role and potential utility of 'dead' channels in SAEs for interpretability, which cease to activate during normal operation but might retain learned decoder weights.
- The generalisation of interpretability findings from simpler RL agents to more complex architectures like decision transformers or RNN-based agents, especially for tasks requiring memory.

## 2.9 Summary

Prior work shows huge potential in the applications of RL agents due to their ability to achieve superhuman performance generalise broadly. However, the opacity of how this is achieved represents a persistent problem. A resurgence of interest in this area due to its application in frontier AI models adds urgency to better understand these models and how they operate on a mechanistic level (Hilton et al., 2020; Mini et al., 2023). Techniques such as activation analysis, steering vectors, and feature visualization have provided initial windows into these systems. However, challenges like polysemanticity (where neurons or channels respond to multiple unrelated concepts (Elhage, Hume, et al., 2022; Olah, Cammarata, et al., 2020)) and the distributed nature of representations often make it challenging to make strong conclusions about the operation of the neural network.

The application of tools like Sparse Autoencoders offers a potential solution for being able to make confident statements about whether the nature of how models truly learn to encode concepts (Bricken et al., 2023; Elhage, Hume, et al., 2022), potentially making the functional roles of different network components clearer. While much of the early MI work focused on LLMs, applying and adapting these techniques to RL, with its emphasis on clear goals and environments with fewer components and potentially broadly generalisable solutions offers the ability to make unique insights that may apply to ML models more broadly.

The research presented in the accompanying papers (forming the basis of this thesis) builds directly on this existing work by:

- Extending activation steering and intervention techniques to more complex, multi-objective RL environments like Procgen Heist.
- Applying Sparse Autoencoders to the convolutional layers of an RL agent to disentangle features and investigate the extent to which multiple entities are truly captured within a single layer of the model and through what mechanism this is done.
- Systematically analyzing the role of individual channels in navigation and goal pursuit through methods like ablation studies and retargeting interventions.
- Exploring novel mechanisms of information encoding, such as multiplexing entity information within a single channel through varying activation strengths.

These contributions aim to deepen our understanding of how RL agents represent and pursue goals, and to refine the tools available for mechanistic interpretability for use in this domain.

# Chapter 3

# Methodology

## 3.1 Environment and Model

### 3.1.1 The Procgen Heist Environment

The Procgen Heist environment, from the suite of procedurally generated environments by Cobbe, Hesse, et al. (2020), was chosen for this research. The agent's objective is to navigate a maze to collect a gem, which may require first collecting up to three keys (in the fixed order of blue, green, red) to open corresponding locks. The environment's procedural generation varies the maze layout and the number of key-lock pairs present in each episode, from zero to three.

This environment provides a sparse reward signal, with a reward of +10 for reaching the gem and 0 otherwise. This design, coupled with the variable difficulty, establishes a natural curriculum that allows agents to succeed with random exploration in simpler levels before developing a more sophisticated policy.

The choice of a Procgen environment is motivated by the need to learn generalizable representations. The procedural nature of the mazes discourages the agent from memorizing simple heuristics and instead promotes the development of robust internal models of entities and game mechanics. Furthermore, Heist's multi-objective nature is expected to produce complex internal behaviours, such as general-purpose navigation circuits that can be dynamically retargeted based on the current goal.

For this investigation, the 'easy' distribution of the environment was used, which provides the full 64x64 observation to the agent, making the environment a fully-observable Markov Decision Process (MDP). While memory-based, partially-observable variants exist, they were not used. The distribution of entities is noteworthy: since levels are sampled uniformly from all configurations, the gem and the blue key/lock

appear more frequently than the green and red pairs.

### 3.1.2 Model Architecture

We train a compressed version of the Impala model that uses 5 convolutional layers instead of 15 as used in the original Impala paper. This architecture choice reflects established principles for interpretability research: following the precedent set by (Hilton et al., 2020), simpler architectures often yield more interpretable representations without significant performance loss. The models used are simplified versions of established architectures, chosen for their balance of performance and interpretability. We include the model architecture in full in Appendix .1.

When training our model we use a simple PPO implementation from an open source implementation designed to train Procgen models called Procgen-pfrl. We used the easy distribution of the environment due to compute limitations. We tested model training over a number of different batch sizes and distributions. The model checkpoints used to derive our main findings were trained with standard hyperparameters: learning rate 5e-4, 64 parallel environments collecting 256 steps each (16,384 total steps per update), processed in batches of 8 over 3 epochs for 800 million steps. The training dynamics were significantly impacted by changes to batch size often completely collapsing training performance, but over the course of 5 runs with separate seeds with the parameters above we were able to replicate similar model dynamics in each case.

### 3.1.3 Model Training

The policy models were trained using Proximal Policy Optimization (PPO) with an open-source implementation based on PFRL, specifically designed for Procgen environments. We also made use of Envpool to accelerate the speed of our training while keeping the actual training dynamics unchanged (Weng et al., 2022). Training was conducted until convergence, which in this case was when no statistical improvement was observed for 10000 update steps. The primary model was trained for approximately 800 million environment steps (60,000 update steps).

To explore the effects of training dynamics on the final learned representations, experiments were run with different training configurations. These variations primarily involved altering the batch geometry through four main hyperparameters: the number of parallel environments, the number of steps per rollout, the mini-batch size, and the number of epochs.

In the primary configuration reported in this thesis, we used a setup of ($N_{\text{env}} = 64$, $N_{\text{step}} = 256$, `batch_size` $= 8$, $n_{\text{epoch}} = 3$). In this geometry, the rollout buffer contains $N_{\text{env}} N_{\text{step}} = 16\,384$ transitions. This is reshaped into 8 mini-batches, each containing $B_{\text{mb}} = 2\,048$ samples. The optimizer performs $8 \times 3 = 24$ gradient steps per PPO update, processing a total of $16\,384 \times 3 = 49\,152$ sample-updates

per cycle. This configuration was chosen to balance stable learning from large batches with reasonable training time per update.

We ran 5 sets of training runs for this geometry in order to confirm the repeatability of our results, and ran further training runs with other geometries to observe the extent to which this altered the final structure of the model weights. Namely ($N_{\text{env}} = 128$, $N_{\text{step}} = 128$, `batch_size` $= 8$), ($N_{\text{env}} = 256$, $N_{\text{step}} = 256$, `batch_size` $= 8$), and ($N_{\text{env}} = 256$, $N_{\text{step}} = 128$, `batch_size` $= 16$). These variants were included to probe the sensitivity of the learned representations to rollout length and batch composition. A detailed comparison of the models trained under these settings is provided in Chapter 4.

## 3.2 Interpretability Techniques Employed

### 3.2.1 Feature Visualization

A useful initial approach involved simply creating direct visualisations of the convolutional layers of the model. This involved simply taking a given layer and applying a colour spectrum according to the strength of the activations within that region of the grid. This approach, following (Mini et al., 2023), provided a direct view of which spatial regions triggered strong positive or negative responses in different channels. Figure 3.1 shows an example of this visualisation technique applied to multiple layers.

### 3.2.2 Comparative Activation Analysis

#### Parallel rollout analysis

RQs 1, 2 and 4 aimed to tackle how the network determines which entity to target next, and how to characterise this mechanism, as there are many instances where there are multiple routes leading to different entities in the environment, and the sequence of collection is critical to success. This mechanism addresses these questions by isolating how the network encodes different goal entities and determining whether the model uses entity-specific channels or a general targeting mechanism.

The purpose of this experiment was to determine to what extent the network relies on the mere activation range of the values in the network as part of determining which stage of the maze it is in, and which entities to target.

To directly compare the model activations when targeting different entites, a simple toy maze environment was created that would test some basic capabilities in the model, specifically its ability to turn a corner and travel in a straight line towards a target. The maze took the shape of a T junction, with the agent started at the middle point, and with a blue key placed in the bottom right. A visual of

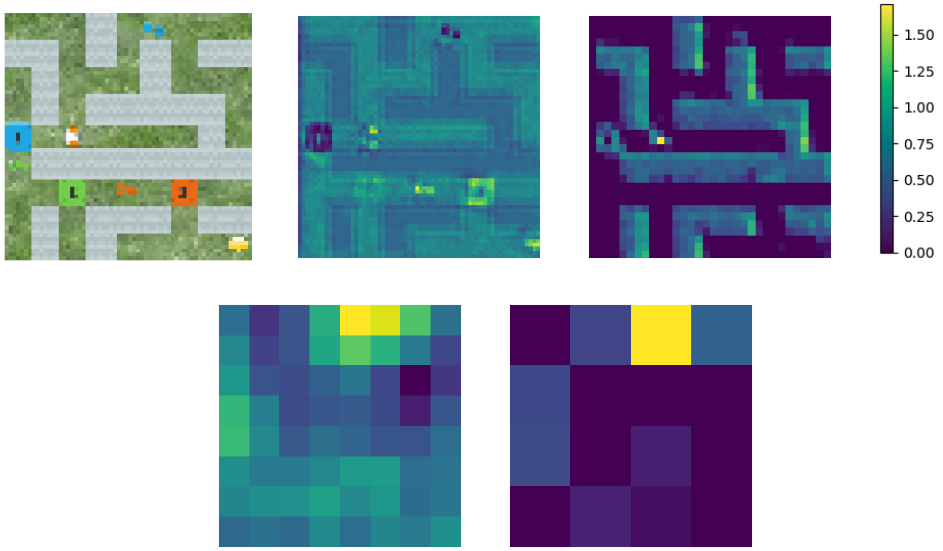


Figure 3.1: Visualization of the maze and neural network activations in different layers. Top row (left to right): observation input, layer 1 channel 10, layer 2 channel 10, and activation colorbar. Bottom row: layer 8 channel 29 and layer 9 channel 29. In the top row filters we observe a minimal activation around the regions of the blue key and lock and in the bottom row we see maximal activations around the blue key. This is a replication of the approach used by (Mini et al., 2023) in order to get an initial sense of the clarity with which entities could be seen in activations.

the maze is provided in Figure 3.2. A rollout of the policy was then performed, but for each step of the rollout, the environmental state was modified to produce an observation with the gem, blue key, green key, and red key. By doing this at every step, 4 parallel rollouts with different entities were produced which allows for a perfectly controlled experiment from which we could observe the variations in activation patterns based only on the nature of the target entity.

With these rollouts we then tracked both the mean centered activation values for each entity, as well as the raw activation magnitudes across all channels in layers conv3a and conv4a. The mean-centering was performed by computing each channel's temporal mean across the entire trajectory and subtracting this mean from the activation at every timestep. This transformation centers each channel's activations around zero, removing baseline offsets between channels and making temporal dynamics comparable across channels with different baseline activation levels. Both raw and mean-centered activations were recorded to enable analysis of absolute activation strengths as well as relative temporal patterns. This design isolates the effect of entity type on the model's internal representations while

holding agent position, maze geometry, and navigational requirements constant across all four parallel trajectories.

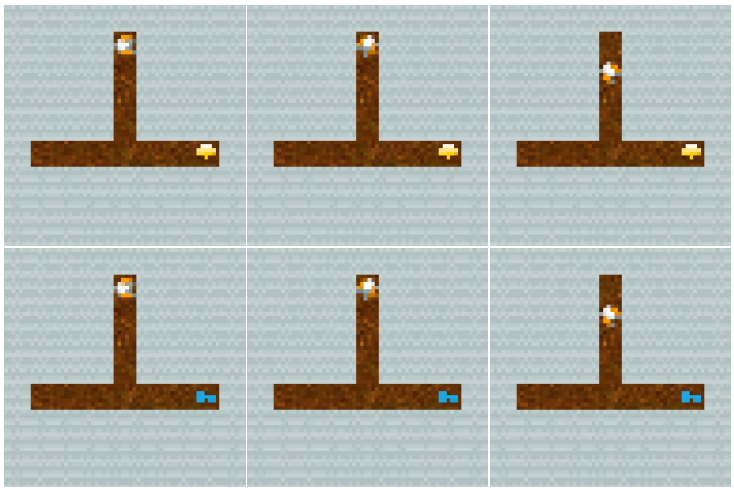

Figure 3.2: This illustrates the parallel rollout of the agent, with each step taken by agent remaining exactly the same while we create an observation with each of the blue key, green key, red key, and gem.

### Analysing activation changes over a rollout

To further investigate RQs 1 and 2, we examine how activation patterns evolve as entities are collected, revealing how goal encoding changes dynamically during task progression. The previous experiment showed how activations vary when entities are changed but all other variables are held constant. However, it is limited in that it only contains a single entity at a time, while normally the environment will have multiple entities present. To observe whether activation ranges were meaningfully related to which entities were present we captured mean activations for given channels in a layer over the course of an episode in an unmodified maze. This allowed us to measure clear patterns would emerge between the activations of different channels, as different entities were collected.

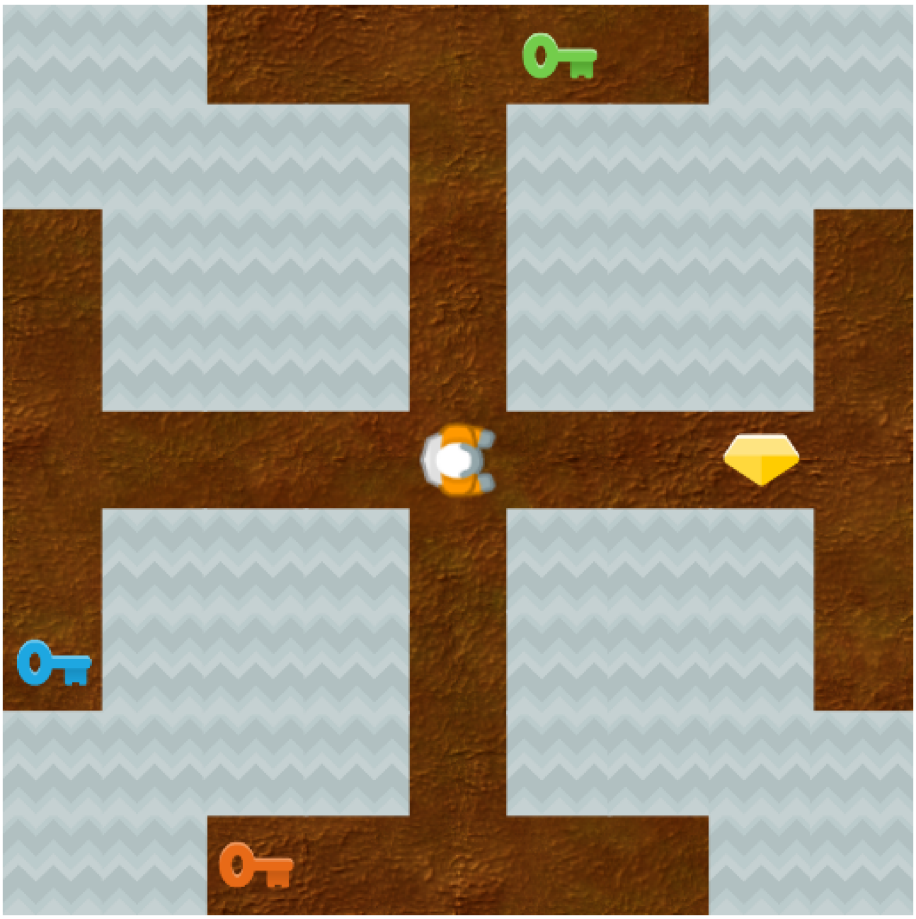

Figure 3.3: This illustrates the 4 directional maze used for analysing which entity would be collected first when different activation offsets were applied. Within a given maze, the positions of the entities within a given arm of the maze would be randomised so as to produce greater permutations to allow meaningful statistical testing.

### Applying offsets to channels for entity steering

The following experiment aims to address RQ2 by testing whether activation magnitude alone can alter entity targeting, and contributes to answering RQ5 by measuring the ability to affect steering across different entities. To do this we swept across a parameter $\alpha$, which represented a constant offset that we would add to all channels in a given layer simultaneously. We would then perform a rollout in a custom four armed maze environment, which can be seen in Figure 3.3. This maze contains the keys and the gem but no locks, and allows for placement of entities at multiple random points within the arms to allow for statistical variance between

runs. We used this environment because it provides some navigational challenges, but also makes it possible for the agent to go towards any of the entities, meaning we can test observe its preferences. We also remove any confounders that may arise from specific maze structures.

To determine the extent to which uniform activation offsets affect target selection, we performed a comprehensive sweep by simply adding a constant offset $\alpha$ to all channels in a given convolutional layer.

To describe this formally: for a given maze seed $s$, channel $c$ in layer $l$, and at each timestep $t$, we modify the channel activation $h_t^{c,l}(s)$ as follows:

$$\tilde{h}_t^{c,l} = h_t^{c,l}(s) + \alpha \tag{3.1}$$

Here, $\alpha$ is the flat scalar offset applied. We performed parameter sweeps over $\alpha$ in the range $[-15, +15]$ in increments of 0.2, with $n = 100$ offset values per increment for conv4a. Some layers exhibited changes over a much narrower activation range such as conv3a and conv2a, and for these we performed sweeps in the range $[-3, +3]$ with increments of 0.03.

### 3.2.3 Sparse Autoencoder (SAE) Training

SAEs are a type of unsupervised learning technique that can be used to break down the most critical functions of a model into a more interpretable set of sparse latents, often by training an overcomplete basis on the original activations of a given layer of the network (Bricken et al., 2023). In this study, we apply this approach to the convolutional neural network (CNN) layers to disentangle model functionality. The training of SAEs directly addresses RQ3 by applying pressure to split polysemantic neurons into interpretable features, if those features are indeed merely polysemantic. Our methodology for adapting SAEs to CNNs builds upon recent work by (Gorton, 2024), and we describe it in later sections.

SAEs were trained on the CNN layers of the compressed IMPALA model. Each model followed the 1×1-conv recipe of (Gorton, 2024) with an L1 warm-up schedule. Training ran for 6 M steps, during which time variance-explained would rapidly exceed 0.99% and policy reward recovered to within 99% of baseline. A compressed variant, with conv4a reduced to 20 channels, was also trained to test robustness under capacity constraints. Table 3.1 lists the expansion factors and L1 coefficients for all layers. 5 SAEs were trained for each layer, as a means of verifying the robustness of our results.

Table 3.1: Expansion factors and L1 Coefficients for the different SAEs

| Layer | Expansion Factor | L1 Coefficient |
|---|---|---|
| conv1a | 2 | 1e-4 |
| conv2a | 4 | 5e-4 |
| conv2b | 4 | 5e-4 |
| conv3a | 4 | 5e-4 |
| conv4a | 4 | 5e-3 |

### 3.2.4 Validating results with SAEs

Our initial steering experiments on the base model revealed that numerous channels responded to multiple, distinct objectives. This prompted further investigation to determine whether this was unstructured polysemanticity (single channels encoding unrelated concepts) or a general encoding strategy for related concepts.

We used SAEs as a validation tool to tease out whether the representations were simply polysemantic encodings (RQ3) or if the model was doing something more sophisticated to encode the different entities in a single channel but still make clear specifications between them (RQ4). The hypothesis was that if the model was learning abstract concepts like 'blue key' or 'green key' an SAE should be able to isolate these concepts into distinct, interpretable features. After training the SAEs, we analysed their latent representations using the steering and ablation techniques described previously. The key objective was to search for individual SAE channels that generalized across a class of related entities—for example, a single channel that could be used to steer the agent towards any of the keys, or if instead entire channels would be dedicated to specific keys and locks, or perhaps only keys. The discovery of channels that encoded all types of entities would provide strong evidence that the model learns general, goal-oriented features, rather than simply having widespread polysemanticity.

By using SAEs in this way, we can be confident in that we have discovered a fundamental mechanism behind the networks operations when examining the functioning of a channel if it seemed to encode multiple entities.

### 3.2.5 Intervention experiments

A critical goal for this work to uncover causal mechanisms behind the model's behaviour and to understand how the network specified which entity to target in a given state of the maze. These causal intervention experiments address RQ2 by identifying which network components determine entity targeting, RQ4 by testing whether specific channels control specific entities, and RQ5 by measuring differential steering effectiveness across entities. While the goals in the environment would

always be sequential, there were still many cases where the model would fail if it tried to target the wrong entity, and so this was a crucial mechanism. Early observational experiments also demonstrated that the preference hierarchy for targeting the blue key, the green key, the red key, and then the gem were strongly encoded in the network.

To demonstrate a true causal mechanism we approached the problem by trying to find parts of the network that seemed to play a key role in deciding model direction. For this we used an activation patching approach. The experimental setup is as follows: We would place a specific entity into the environment and then intervene on a single channel in a single layer by applying a zero mask to the channel and then setting a single neuron of the channel to a specified value. In this experiment, we would do a sweep over a range of values between 0 and the maximum values that the channel would reach during typical functioning as determined by sampling from the environment during operation. By running this sweep, we can get a sense of what the different channels do at different intervention strengths. We kept the length of the episode to 20 steps to ensure that the agent would either be able to reach the target zone, or the actual entity, but not both.

For each entity-channel combination, we ran 500 trials with the activation value incrementing linearly: $v_i = i \cdot \frac{v_{\max}}{N}$, where $i$ is the trial index (1 to $N$), $N = 500$ is the total number of trials, and $v_{\max}$ is the maximum activation range.

We use a 'Q' shaped maze with the design visible in Figure 3.4 because it was challenging enough that the agent would need to retain its navigational abilities to reach either target, while providing a clear decision point where the agent would need to choose between pursuing the original entity or our artificial target zone.

We test a single entity at a time in this way meaning that steering that is effective with a particular value on a given channel may not be effective for redirecting the agent in the presence of another entity.

We scaled up this approach by running a static intervention across every channel in the SAE, and in the base layer of the model. For each entity-channel combination, we performed $N_{\text{trials}} = 500$ trials. In trial $i$, the activation values in a target region of the channel were set to a value $v_i$, calculated as:

$$v_i = i \cdot \frac{V_{\max}}{N_{\text{trials}}}, \quad \text{for } i \in \{1, \ldots, N_{\text{trials}}\}$$

where $V_{\max}$ is the maximum activation observed for the layer. This allowed us to measure the success rate of steering as a function of activation strength and to identify whether channels influence navigation only within specific value ranges.

We test this approach across all layers in the environment. Because conv4a yielded the most promising layer due to its naturally fairly sparse representations we performed additional experiments across 28 model checkpoints over a single base

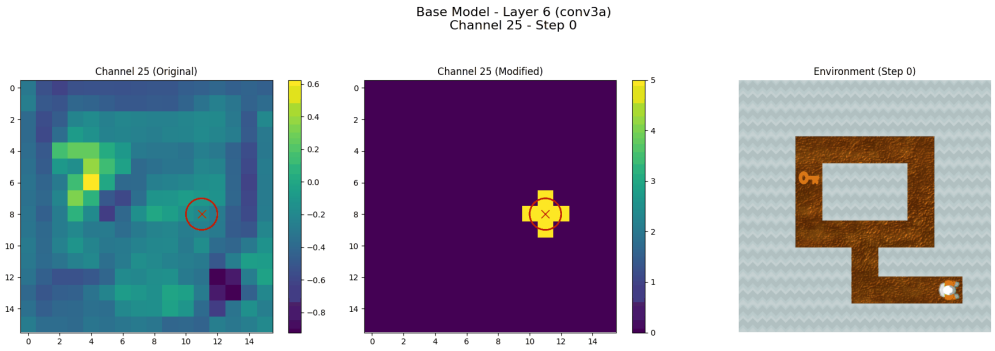


Figure 3.4: Example of the intervention experiment. We test channels to see the extent to which a single channel is capable of steering the agent to another location given the presence of a single entity in the environment. We consider an intervention a success if the agent enters the modified region. The maze is a 9 by 9 grid, while the conv4a channels have dimensions of 8 by 8, leading to a slightly imperfect mapping.

model training run in increments of 3000 update steps from 3000 to 60000. We observe model convergence to maximum rewards at around 20000 steps, and thus many of the later checkpoints are variations of different equally high scoring policies.

To establish a baseline for normal activation ranges, we calculated the 99.9th percentile of activations for each channel over 50 rollouts in unmodified mazes. This threshold is used in later analysis to identify whether successful interventions operate within or outside the network's typical activation distribution.

### 3.2.6 Discovering independent navigation channels

This ablation methodology addresses RQ6 by identifying whether navigational capabilities are localized to specific channels, revealing functional specialization across the network. To determine the role of certain channels in navigational capabilities, to contrast with entity targeting, a comprehensive ablation methodology was developed. For each channel in the network, we ran episodes with only that channel active while masking all other channels to zero. This allowed us to measure the isolated capability of each channel to support navigation to different entities in the environment. We recorded successful entity collection counts for each channel across n=400 rollouts, providing a quantitative measure of each channel's specialization and capability. This methodology allows us identify which channels are sufficient for basic navigation. An example of the maze used is given in 3.5.

Because the agent has a bias towards randomly walking in a given direction depend-

ing on the current policy, it is necessary to rotate the maze so that the direction it's biased towards is the one facing away from the entities. To mitigate this, for every channel a test was carried out to determine the bias direction and rotate the stem of the fork in this direction to avoid contamination of the results.

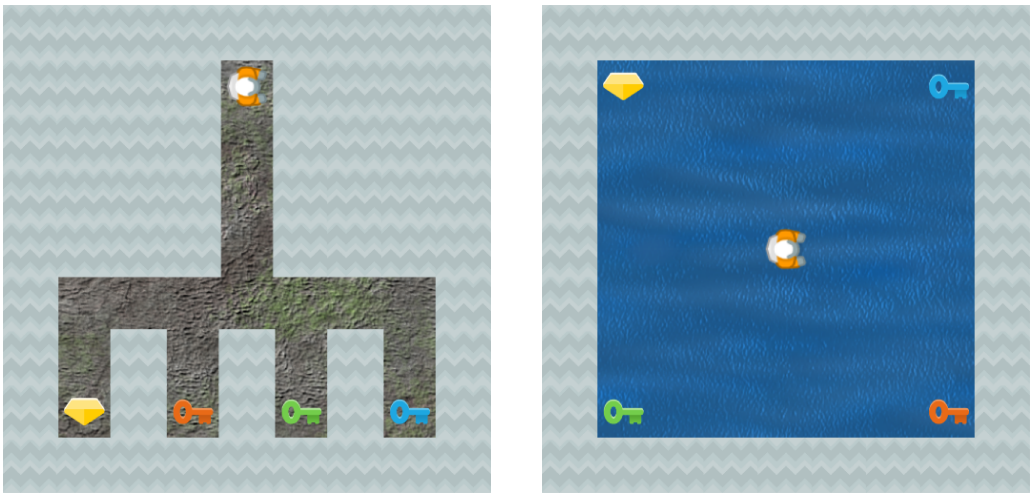

Figure 3.5: Maze configurations for ablation studies. (a) Fork maze tests individual channel navigation capability by requiring explicit path choices towards different entities and providing sufficient complexity that navigational abilities must be preserved. We orient the stem toward the model's inherent bias direction to minimize false positives. (b) Open maze with entities in corners tests the effect of ablating intervention spans without navigational constraints, ensuring only preferences regarding entity collection are observed.

### 3.2.7 Discovering critical infrastructural channels

Complementing the previous analysis, this experiment further addresses RQ6 by distinguishing between channels sufficient for navigation in isolation versus those critical for the network's collective functioning. While the above experiment reveals the degree to which individual channels can retain navigational capabilities, it is not necessarily the case that these individual channels are the most essential to the successful navigation of the network under normal circumstances. Channels which performed badly at independent navigation might simply be highly dependent on other channels in order to functon.

To establish which channels were critical to the overall functioning of the network, we performed a similar experiment to the one described in 3.2.6 but instead of ablating all channels but one, we ablated only one channel to observe the impact

on the collection rates of different entities. We repeated this for every channel in conv4a for n=400 rollouts in the fork maze shown in 3.5. We describe these channels as Infrastucture Channels in later experiments.

The metric by which we ordered these channels was on how great the negative impact was on the collection rates for different entities, because channels that did little to reduce collection rates for a given entity were likely to be less important for that entity.

## 3.3 Expanded ablation experiments

To derive further insights from our earlier results, we ablate regions of the network based on their success in modifying model behaviour in the previous experiments. These targeted ablations address RQs 4, 5, and 6 by systematically testing which network components are necessary for entity-specific targeting, why steering effectiveness varies, and how functionality is distributed across layers. Our experiment uses a modified maze which removes all walls, and has the keys and gem randomly distributed in the corners of a square, shown on the right in Figure 3.5.

In these experiments, the choice of targets for ablation was tailored to the entity in question. For example, in the case of the redundant navigation channels experiments, when we say that the top 10 channels were used for a given entity, we mean that for the blue key results, the 10 channels that had the highest blue key collection rates were chosen. This applies for each of the categories of experiment, though naturally the selection criteria vary according to what was intended with a specific intervention.

We perform six targeted ablations based on our steering and navigation findings:

**Ablation 1 (Intervention spans):** We zero out any activations within the value ranges that successfully steered the agent. Specifically, for entity $e$ and channel $c$, we set $h_{c,i,j} = 0$ whenever the activation falls within the successful steering range $\mathcal{S}_{e,c}$ identified in our intervention experiments.

**Ablation 2 (Redundant Navigation channels):** We ablate the top 10 channels that had the highest collection rates for a given entity in our independent navigation study, setting these channels to zero while leaving the rest unchanged.

**Ablation 3 (Infrastructure channel Ablation):** We ablate the top 10 infrastructure channels that, when ablated, caused the lowest collection rates for a given entity, setting these channels to zero while leaving the rest unchanged.

**Ablation 4 (Combined):** We ablate both the navigation channels that had the best collection rates for a given entity and the intervention spans for a given entity simultaneously.

**Ablation 5 (Inverse):** We preserve only the intervention spans that successfully steered the agent in the presence of entity $e$, ablating everything else. This tests whether these spans alone are sufficient for navigation.

**Ablation 6 (Random):** We randomly select 10 channels (equal to the number of navigation channels ablated) and completely ablate them as a baseline.

## 3.4 Probing for Abstractions

To determine at which layer in the network different concepts were processed, we performed two different probing experiments, one which targeted the concept of which entity was currently the current correct target of the agent, and which direction did the agent needs to move in to reach its goal. These probing experiments address RQ1 by localizing where goal information is encoded, and RQ6 by revealing which layers specialize in processing target identity versus navigational planning.

The key question we wanted to answer with this experiment was to determine at which point the greatest activity on processing the current next target was performed. Probes are an imperfect mechanism for this because they provide only correlational evidence, and there are ways that the information about which entity is the current target may be present without actually being involved in further processing of that information. They do at least show whether there are sufficient features that a distinction between the target classes can be made however, and this can inform important intuitions about the network.

### 3.4.1 Next Entity Probes

This probing approach addresses RQs 1 and 2 by identifying which layers encode the next target entity, and RQ6 by comparing representational clarity across network depth. To investigate where the network encodes information about which entity to target next, we trained multi-class linear probes to predict the next target entity from layer activations.

For determining the next entity, we gathered a large number of observations from many unmodified maze rollouts, capturing activations every 7 steps to ensure sufficient diversity was achieved. For each of seven entity types (blue key, green key, red key, gem, blue lock, green lock, red lock), we gathered 2000 observations where that entity was the agent's next target, yielding 14,000 total samples balanced across entity types.

We trained 7-way classifiers to predict which specific entity was the next target. We used a 90/10 train-test split.

We trained two types of probes:

- **Per-channel probes**: For each channel in layers conv1a-conv4a, we flattened the spatial activations and trained linear classifiers on the flattened activations.
- **Whole-layer probes**: For full layers (conv1a, conv2a, conv3a, conv4a, fc1, fc2, fc3), we flattened all activations across all channels/neurons and trained a linear classifier for each layer

All probes were linear classifiers trained with the Adam optimizer (learning rate 0.01) for 200 epochs.

### 3.4.2 Direction probes

To establish which parts of the network were more involved with navigation, we trained multi-class linear probes to predict which direction the agent would need to move to reach the next target entity. This is a complex task because often the direction needed to travel in was significantly different than the position of the entity relative to the player. By probing for directional information separately from entity identity, this experiment addresses RQ6 by testing whether navigation processing is functionally separated from goal representation in the network architecture.

Building the training dataset involved using Dijkstra's algorithm to calculate the most efficient route to the next entity and using the grid state to calculate which path would need to be taken, and the optimal direction to take (which the policy would reliably choose). For each of the four cardinal directions we gathered 2000 observations and extracted model activations for each, excluding examples where the agent was directly next to the target entity, or where the agent was too far from the edge of an grid square in the maze, ensuring that there was no ambiguity in the direction that needed to be taken.

We trained 4-way classifiers with the same architecture and training procedure as the entity probes.

## 3.5 Summary

This chapter detailed the methodologies employed to investigate goal representation and control in a deep reinforcement learning agent. It began with a description of the Procgen Heist environment and the model architectures used. Key interpretability techniques were then outlined, including the training of SAEs, feature visualisation methods, the calculation and application of steering vectors for activation interventions, and systematic ablation studies. Finally, the experimental design and the quantitative and qualitative metrics used for evaluation were

presented, providing a framework for understanding the results discussed in the subsequent chapter.

# Chapter 4

# Results

All results presented are from checkpoint 35001 (35k update steps), past convergence which occurred around 30k, unless otherwise noted.

## 4.1 Parallel Rollout Activation Analysis

In performing parallel rollouts targeting the different entities, as shown in Figure 3.2, we observe that there are distinct parallel trajectories in the mean activations across different channels in the network for each entity.

By observing which channels had the lowest and greatest offset between runs targeting different entities we could construct clear visual comparisons to determine whether the primary mechanism changing between entities was simply the mean activation, or if there was significant variance in which channels were activating. This would compare if the entity signal was captured via specific channels that would uniquely fire given the presence of an entity, or if the activation offset itself would carry the signal.

We found that in most cases we could clearly observe clean parallel trajectories with significant gaps in the values between them, indicating strong value encoding for entity targeting. For the channels with the lowest offset, we see similar parallelism, though often certain entities will be very close in their values such as red key or blue key overlapping, meaning there is little distinguishing between the entities in these channels under this experimental setup. We note that given more natural environment setups with more entities present that the activation dynamics are likely to change significantly.

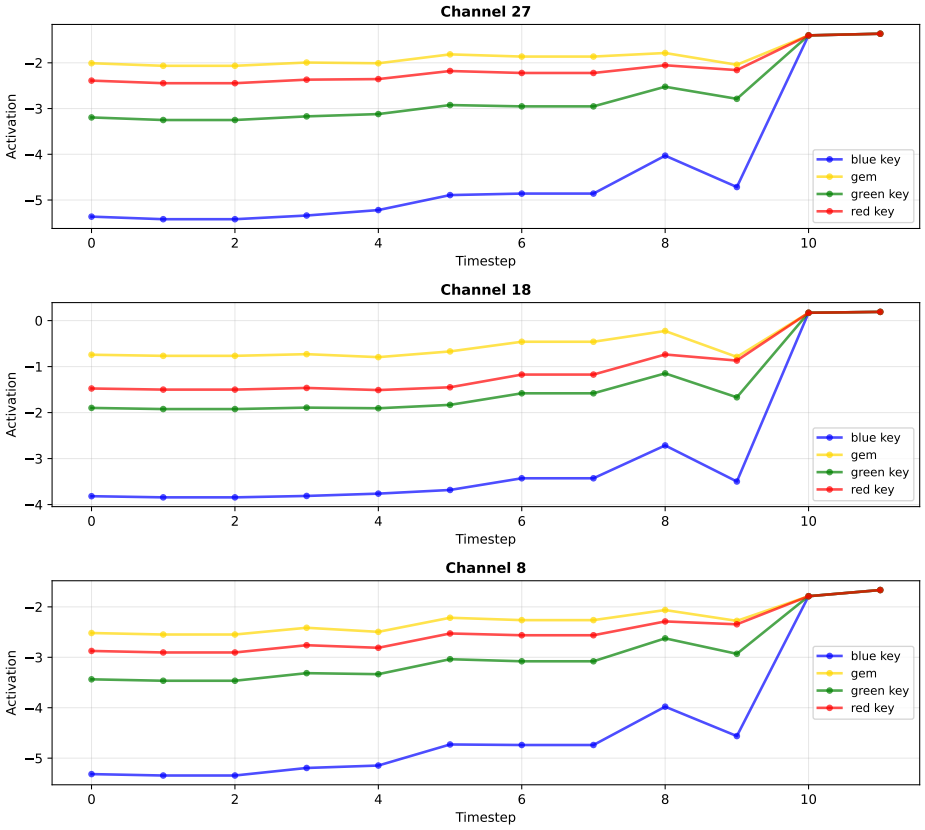


Figure 4.1: Trajectory patterns showing distinct activation patterns across different entities in layer conv4a at checkpoint 35k.

We observe that extremely high correlation between activation patterns exists in both conv3a and conv4a, with conv3a showing an average correlation of 0.956 (range: 0.837-0.995). However, while conv4a has comparable average correlation (0.931), it contains a subset of channels with substantially lower correlations (range: 0.401-0.995), suggesting these channels encode temporal dynamics rather than simple offset patterns and play an unusual role in entity targeting. The full results are shown in the mean inter-entity correlations averaged over 10 rollouts in Figure 4.2.

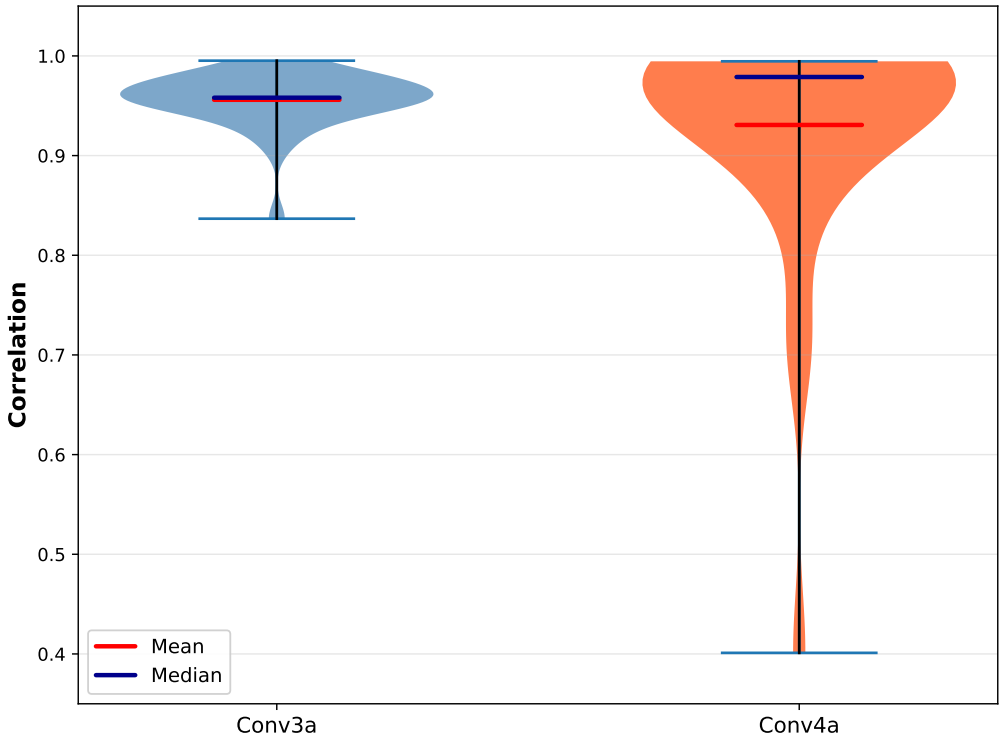


Figure 4.2: Violin plots comparing the distribution of inter-entity correlations between conv3a and conv4a layers. Each violin shows the distribution of mean correlations within channels for that layer. For each channel, correlations are computed by: (1) calculating pairwise correlations between entity trajectories within a rollout, (2) averaging across all entity pairs, and (3) averaging over 10 rollouts.

### 4.1.1 Activation patterns over rollouts

We carried out further experiments to observe how activation patterns evolved over the course of a rollout, as different entities were collected. To do this we simply captured activations over the course of a rollout in both the custom maze environments, and unmodified mazes. We then took the mean of the activations in a given channel and tracked it over the timesteps in the rollout. This gave rise to clear patterns where activations would typically jump upwards as each key was collected, with the size of the jump decreasing as each one was collected. These were particularly obvious with the most high variance channels, with the effect being present not ubiquitous amongst the other channels. Results of a randomly selected rollout shown in Figure 4.3.

Another recurring pattern in the activation trajectory was the pinching effect seen

in Figure 4.3, wherein the activations suddenly converge to a similar value across entities. This is likely because the network has learned that regardless of the entity involved, if the player is one step away from the entity then it always makes sense to go directly to that entity, as in the normal course of a rollout there is no scenario in which it can be directly beside an entity while it would be incorrect to collect that entity, except in the case of locks.

Interestingly, in the cross maze where it was possible to access all the keys from the beginning the model would choose to target the green key first in about 12% of cases. In these instances we would still see the positive jump in activations as we saw in the case of the blue key, indicating that this phenomena is primarily tied to the number of entities observed, and not necessarily targeting a specific entity.

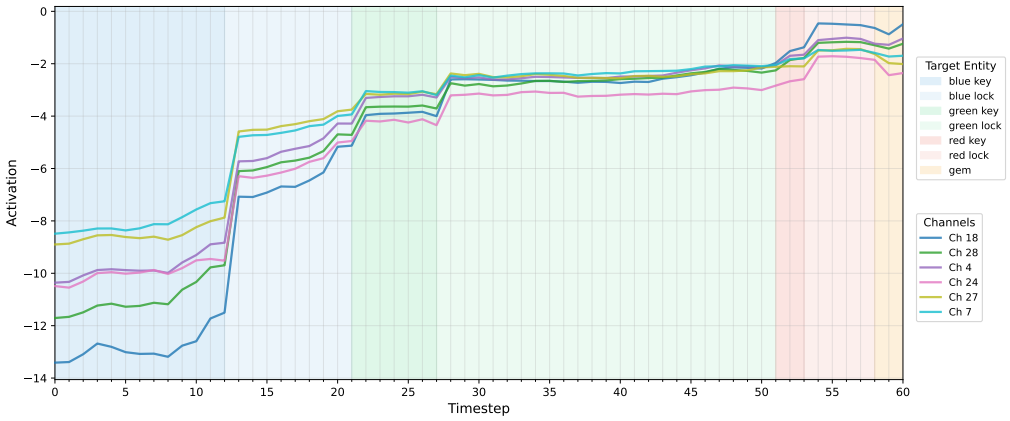


Figure 4.3: Plot showing the mean activation values captured across all timesteps in an unmodified rollout in the 6 channels that had the greatest variance over the course of the rollout. The shaded regions indicate the current 'next target', the entity that it would logically pursue next in solving the maze. Note the upward jumps in activations at the points between shaded regions, indicating contact with an entity.

## 4.2 Activation Offsets Achieve Steering

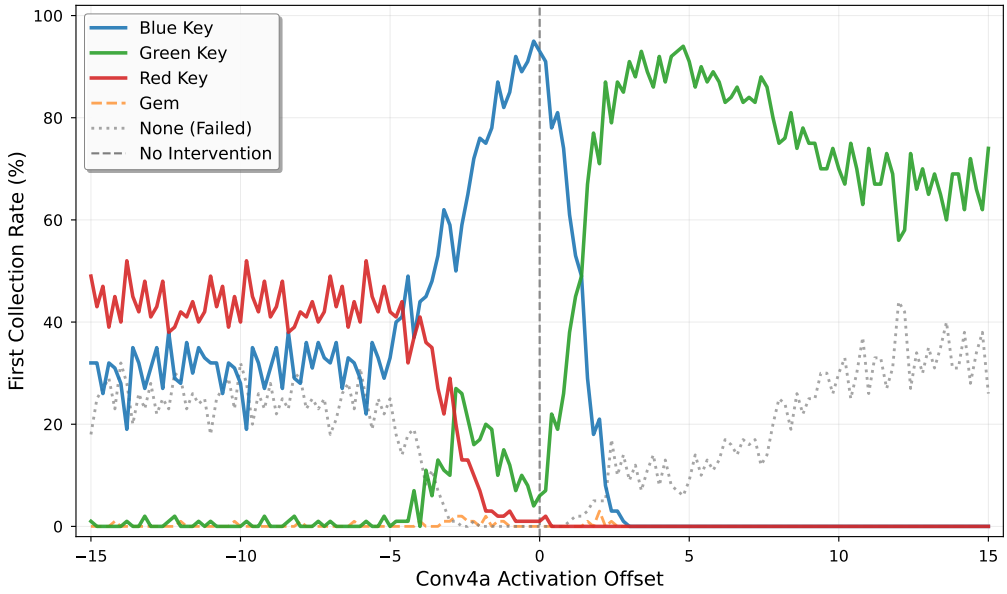


Figure 4.4: Activation offset experiment results. This shows the degree to which modifying the activations by a simple offset value would induce steering towards different entities in the environment, with different offset range providing reliable if imperfect redirection towards the green key and red key. A run ends immediately upon contact with an entity, and is considered 'none' if no entity is collected over 100 timesteps.

A surprising finding was that merely by adjusting the activation ranges within channels by a static offset we could reliably achieve coherent steering towards different entities than what the agent would target by default. Results of rates of initial pickup over different rollouts are presented in Figure 4.4. We also observe this result on different checkpoints, with different success rates for offset values but with similar patterns. We find that steeering towards different entities by adjusting mean offset works across layers shows that it is endemic to the structure of the network, and not simply an isolated phenomena.

We observe complete shifts in entity targeting. Baseline rates show 93% ± 6% blue key targeting at offset 0.0. At positive offsets, the agent switches to targeting the green key, peaking at 94% ± 5% at offset +4.8. At negative offsets, we observe peak red key collection of 52% ± 10% at offset −5.8. Rates of not collecting any entity increase at the extremes, but overall collection remains fairly high, meaning important navigational functions continue to function.

Conv2a (shown in Figure 4.5) shows particularly amenable offset steering with fine

grained control. Red key steering peaks at 75% ± 5% at offset +0.30, illustrating clear and strong steering. There is also a surprising surge in gem collection (12.2% ± 3.2% at offset +0.40). The precision of the technique is notably high in conv2a, likely due to it being the first layer where the next target is being established in the hierarchy as illustrated by Table 4.1.

Conv1a, despite having minimal entity probe accuracy (20.0% overall), shows strong steering capabilities with a different distribution than that observed in the other layers. Unlike the overlapping, nuanced patterns in later layers, conv1a exhibits discrete zones: green dominates the negative range (60-73% from -3.0 to -0.5), blue shows a sharp peak near the baseline (92% at +0.1), and red controls the positive range (50-75% from +0.5 to +3.0). Surprisingly we find the best gem steering in conv1a with 57% gem collection at +1.9 offset—far exceeding conv2a (12%) and conv4a (3%). This suggests conv1a interventions is able to shift the normal bias towards collecting keys first, before target integration occurs.

We also investigate whether the colours in the background have a significant impact on which entities are selected for targeting, and we find in an n=1000 sample that there is no statistically significant relationship between the colours of the background and the choice of entity (p=0.21). Figures for conv3a, conv2a, and conv1a, including more precise sweeps over smaller regions are included in Appendix .2.

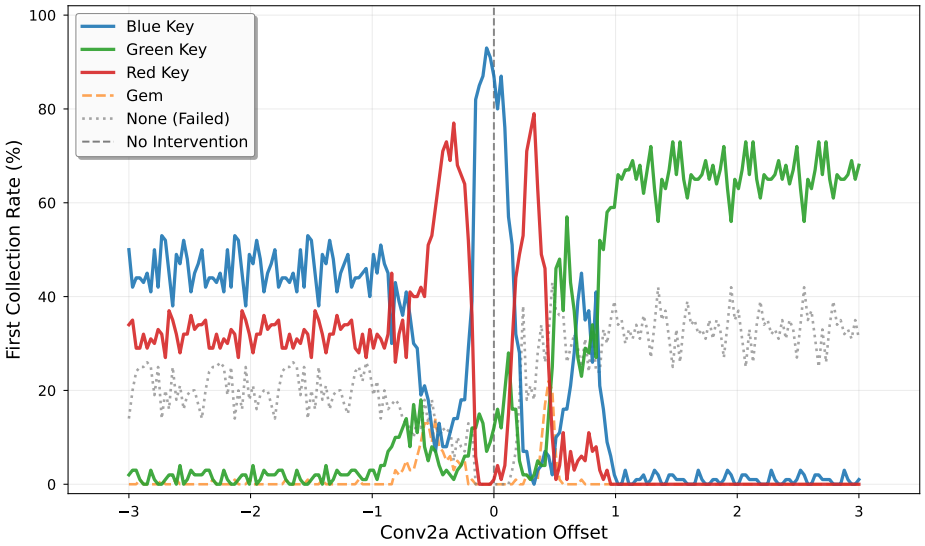


Figure 4.5: Activation offset experiment results. This shows the degree to which modifying the activations by a simple offset value would induce steering towards different entities in the environment, with different offset range providing reliable if imperfect redirection towards the green key and red key. A run ends immediately upon contact with an entity, and is considered 'none' if no entity is collected over 100 timesteps.

### 4.2.1 Quantitative Steering Results

We apply our incremental steering experiments across all channels in conv3a and conv4a and find a large number of channels which successfully steer the agent away from a given entity. We present our results from layer conv4a in Figure 4.6. For conv3a a similar phenomena is observed but with far more channels and many more effective steering values. We add black dotted lines to demarcate the 99.9th percentile activation threshold. This demonstrates that for many channels, the steering is far outside of their normal activation range, indicating that the steering functions as a kind of superstimulus and is likely not part of normal functioning for the network.

A possible explanation for why the ranges have an upper limit beyond which they stop working is that the activation is so strong that it disrupts normal functioning of the network, meaning that the agent fails to reach the artificially activated region.

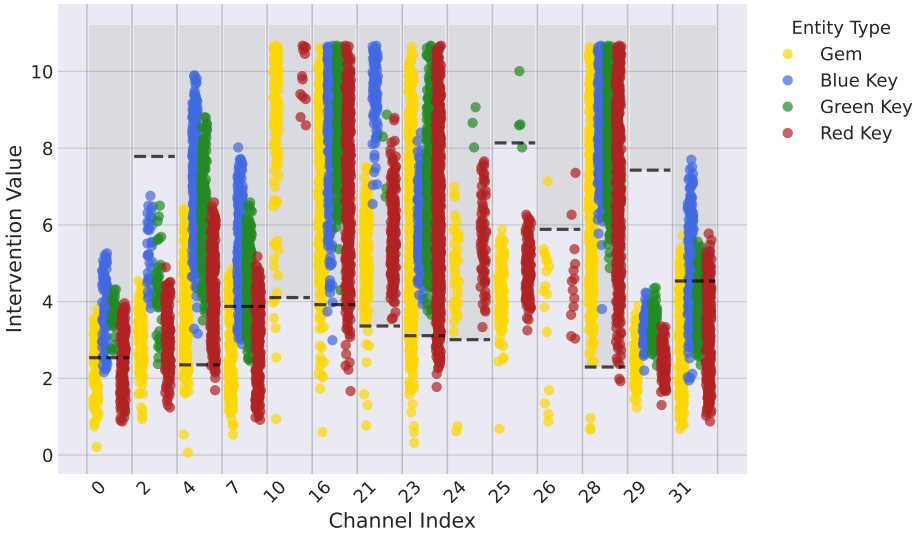


Figure 4.6: To determine the frequency with which a given channel can be used to steer the model away from a given entity, we create an artificial maze in a 'Q' shape and place an artificial activation at a point to the right, and a given entity from the maze on the left. We declare an intervention to be successful based on whether the agent makes contact with the region around the artificial activation. Each colored dot in the above plot represents a successful suppression of the policy's normal target. Intervention success is partially localized to specific activation value ranges that vary based on the entity. Black dotted lines indicate the 99.9th percentile activation threshold, showing where interventions are outside the normal distribution of activations. This sample is from channel conv4a.

The most striking aspect of our results is the fact that particular value ranges worked for specific entities, illustrating how the channel seemed to successfully distract the agent when in the presence of a particular entity through the amplitude of the activation within a channel. The successful distraction may mean that channel is particularly attuned to that entity, and having it activate elsewhere overwrites its typical normal function, but we believe that jumping to that conclusion may be be pre-emptive.

The primary value of this finding is in providing further support for the way in which individual channels seem to multiplex the targeting of specific entities with varying activation strengths. We trained SAEs on these layers and found they exhibit similar multiplexing structures, confirming this is a robust strategy rather than simply the result of polysemanticity.

## 4.3 Using SAEs to Validate Activation Encoding

### 4.3.1 SAE Performance

We found that the SAEs tended to be extremely effective at recovering performance on the original task, with the conv4a SAEs recovering 99.6% of performance and the conv3a SAEs recovering 102% of performance (likely due to random variance in the seeds and evaluations) as measured by average reward attained in Procgen heist mazes. This recovery would typically occur within 40000 steps of training. The SAEs were also able to rapidly achieve variance explained scores approaching 100% (indicating near-perfect reconstruction of the original activations) within 120000 steps. The SAEs were typically trained for 2 million steps to ensure convergence. This success provides good evidence that the SAEs worked as intended and could be relied upon to validate our other results.

### 4.3.2 SAEs Preserve Base Model Structure

Training SAEs on conv3a and conv4a did not reveal additional interpretable structure beyond what we observed in the base model. SAE latents exhibited the same activation magnitude encoding for entity differentiation, with variations in the intervention spans. This preservation across radically different representation schemes suggests activation-level encoding was fundamental to the operations of the network and not simply the result of compression into a sparse basis.

Our primary method for using SAEs to ensure that we hadn't simply found polysemanticity in the network was to use the same quantitative steering approach described in 4.2.1 applied to all of the latents in an SAE. If it was in fact natural to encode the different entities into different channels then we would likely have

observed a different mechanism emerging where each channel had a specific entity for which it was responsible. Instead, we observe broadly the same patterns with channels successfully steering away from many entities, with a similar offset pattern. Figures illusrating this can be observed for conv3a and conv4a SAEs in Appendix .6.

#### Steering with Dead Channels (SAE)

One of the most surprising findings from the SAE intervention experiments was that a large number of the channels that were effective for steering the agent were 'dead channels'. These are channels that exhibited no activation under normal operating conditions of the model. It was confirmed that these channels remained inactive over 100 rollouts.

## 4.4 Probing for Entity Representations

To determine where the network encodes information about which entity to target next, we trained linear probes on activations from each layer. We gathered 14,000 balanced samples across seven entity types (blue/green/red keys and locks, plus gem) and trained multi-class classifiers with a 90/10 train-test split. This approach reveals at which layer the network most explicitly represents the current goal.

### 4.4.1 Convolutional Layers Encode Entities

The changing accuracies in probe accuracy across layers reveal that the convolutional layers seem to dominate in the selection of which entity to target, while the fully connected layers rapidly transition to translating information into actions. Table 4.1 shows whole-layer probe accuracies from the input through to the final hidden layer.

The probe accuracies illustrate two phases of information processing:

**Phase 1 - Emergence (conv1a → conv3a):** The whole layer probe of Conv1a performs barely above random baseline (20.0% ± 2.1% vs 14.3%), on everything except the green key (99.4%). However, examining individual channels reveals that conv1a contains highly specialised entity detectors: five channels achieve >90% accuracy on blue keys (with Ch11 reaching 100%), five channels exceed 90% on red keys (Ch9 and Ch14 both at 100%), and three channels on the gem (with Ch0 at 97.3%). This shows that conv1a is detecting specific entities, but naturally cannot synthesise a hierarchy between them, due no processing having occurred in the network. In contrast, conv2a achieves 96.6% ± 1.0% accuracy across all entity types, representing a 76.6 percentage point jump. This dramatic difference shows that conv2a integrates entity information into a structure from the signals in

Table 4.1: Whole-layer probe accuracy for predicting next target entity across network layers. Random baseline is 14.3%. Overall accuracy shown with 95% confidence intervals (n=1,400 test samples).

| Layer | Overall | Blue Key | Green Key | Red Key | Gem | $\Delta \rightarrow$ Previous |
|---|---|---|---|---|---|---|
| conv1a | 20.0% ± 2.1% | 0.0% | 99.4% | 0.0% | 0.0% | — |
| conv2a | 96.6% ± 1.0% | 97.7% | 97.7% | 85.6% | 99.5% | +76.6% |
| conv3a | 99.3% ± 0.4% | 99.1% | 99.4% | 98.6% | 100.0% | +2.7% |
| conv4a | 97.6% ± 0.8% | 99.1% | 98.3% | 97.6% | 100.0% | -1.6% |
| fc1 | 87.5% ± 1.7% | 95.4% | 80.2% | 80.3% | 96.2% | -10.1% |
| fc2 | 76.8% ± 2.2% | 92.6% | 55.8% | 75.0% | 90.8% | -10.7% |
| fc3 | 20.4% ± 2.1% | 23.3% | 16.9% | 20.2% | 35.9% | -56.4% |

conv1a. Performance improves slightly in conv3a (99.3% ± 0.4% accuracy) before showing a slight decline at conv4a (97.6% ± 0.8%).

**Phase 2 - Compression (fc1 → fc3):** The clarity of entity representation decreases progressively through the fully-connected layers, dropping from 87.5% ± 1.7% (fc1) to 76.8% ± 2.2% (fc2) and finally collapsing to 20.4% ± 2.1% in fc3—barely above random. To confirm whether the information was simply being represented non-linearly, we trained MLP probes on these layers, which performed very similarly to the linear probes.

This marks the point at which the network transforms explicit entity representations into action-oriented features. This makes sense as in final layer it has to produce only the action distribution and value estimation, which would naturally be quite distant from the entity representations themselves.

### 4.4.2 Directional Navigation Probes

To understand which points of the network are most involved with navigation, we trained linear probes to predict the optimal movement direction (up, right, down, left) from layer activations. We gathered 8,000 samples (2,000 per direction) using Dijkstra's algorithm to compute optimal paths, filtering out examples where the agent was not centered in a grid cell to avoid ambiguity, and also removing examples where the agent was directly next to the target entity in order to enhance the difficulty of the problem. Results are shown in Table 4.2.

The directional probes reveal a significantly different structure than the next target entity probes. Conv2a achieves only 42.7% ± 2.4% accuracy (barely above the 25% baseline), with highly unbalanced performance, achieving near-zero accuracy on downward movement (5.5%) despite reasonable performance on horizontal di-

Table 4.2: Whole-layer probe accuracy for predicting optimal movement direction across network layers. Random baseline is 25%. Overall accuracy shown with 95% confidence intervals (n=1,600 test samples).

| Layer | Overall | Up | Right | Down | Left | $\Delta \rightarrow$ Previous |
|---|---|---|---|---|---|---|
| conv1a | 25.1% ± 2.1% | 100.0% | 0.3% | 0.0% | 0.0% | — |
| conv2a | 42.7% ± 2.4% | 44.5% | 61.5% | 5.5% | 59.2% | +17.6% |
| conv3a | 63.6% ± 2.4% | 61.0% | 66.8% | 69.0% | 57.8% | +20.9% |
| conv4a | 51.7% ± 2.5% | 51.2% | 48.8% | 51.0% | 55.8% | -11.9% |
| fc1 | 62.9% ± 2.4% | 60.5% | 63.5% | 61.5% | 66.2% | +11.2% |
| fc2 | 67.3% ± 2.3% | 64.5% | 66.2% | 69.2% | 69.2% | +4.4% |
| fc3 | 21.1% ± 2.0% | 37.0% | 16.8% | 21.0% | 9.8% | -46.2% |

rections. The best convolutional layer performance is observed at conv3a (63.6% ± 2.4%), before showing a somewhat surprising drop at conv4a to 51.7% ± 2.5%.

Crucially, unlike entity information which degrades in fully-connected layers, directional information *recovers and improves* through the FC layers. FC1 recovers to 62.9% ± 2.4%, and fc2 achieves the highest accuracy overall at 67.3% ± 2.3%, with balanced performance across all four directions. This suggests that spatial navigation information is actively refined and strengthened as the network prepares for action selection, rather than being compressed away.

The surprising conv4a performance drop (63.6% $\rightarrow$ 51.7%) diverges from the pattern observed in the next target entity probes, which retain most of their accuracy at this layer (99.3% $\rightarrow$ 97.6%). This suggests conv4a functions as a bridge in converting the high level spatial information into a structure coherent to the fully connected layers that temporarily disrupts the linear separability of directional information.

### 4.4.3 Comparing Entity and Directional Information Flow

The contrasting progressions reveal a two-stage processing architecture. Entity information contains specialist detectors even at conv1a and peaks at conv3a (99.3% ± 0.4%), while directional information is entirely absent at conv1a (25.1% ± 2.1%, at random baseline) and peaks much later at fc2 (67.3% ± 2.3%). This indicates that the network implements a hierarchical processing strategy:

**Stage 1 - Goal Identification (conv2a–conv4a):** Entity information emerges sharply from conv1a (which contains specialist entity detectors despite poor whole-layer performance) through conv2a (96.6% ± 1.0%) and peaks at conv3a (99.3% ±

0.4%), remaining extremely high at conv4a at 97.6% ± 0.8%. At this stage, directional information remains relatively weak (63.6% ± 2.4% at conv3a), suggesting the network prioritizes identifying the correct goal before planning how to reach it. Notably, conv1a contains no directional information whatsoever (25.1% ± 2.1%, at random baseline), contrasting with its entity specialist channels—indicating directional processing emerges later in the network.

**Stage 2 - Navigation Planning (fc1–fc2):** As entity information compresses (99.3% → 20.4%), directional information strengthens (63.6% → 67.3%). The network transforms explicit entity representations into spatial navigation commands, with peak directional accuracy at fc2 (67.3% ± 2.3%) suggesting motion-oriented features directly relevant to action selection. Both types of information then collapse at fc3 (entity: 20.4% ± 2.1%, direction: 21.1% ± 2.0%), as they are compressed into the final action distribution and value estimates.

This functional separation explains key findings from our intervention experiments. The near-perfect entity encoding at conv3a explains why steering and ablation interventions were most effective at this depth—we modified explicit goal representations before their transformation into motor commands. Conversely, the distributed nature of directional information across multiple layers explains why navigation remained largely functional even under heavy ablation, making it more robust to localized disruption than the concentrated entity representations.

## 4.5 Expanded ablation studies

### 4.5.1 Channel Ablation Studies

Our ablation analysis reveals that the ability to navigate is surprisingly redundantly encoded throughout the network. The heatmap in Figure 4.7 shows the results in full. We observe that all channels have some success with reaching the entities some of the time with the best performing channels achieving success rates greater than 40% for certain entities, while others have lower than 10% success rates between entities. We also see that some channels show similar performance across entities, while others are significantly better at particular entities.

### 4.5.2 Testing Ablation Impacts

Our ablation studies reveal two counterintuitive findings.

First, ablating the gem's steering channels improved collection rates from 74% to 93.2%, while ablating the key related spans reduced key collection by 47% on average. This is likely a result of the distribution of steering spans: gem-related spans appear across 31/32 channels, whereas key-related spans appear in only 14 channels. In addition, key steering spans typically appear as regions within

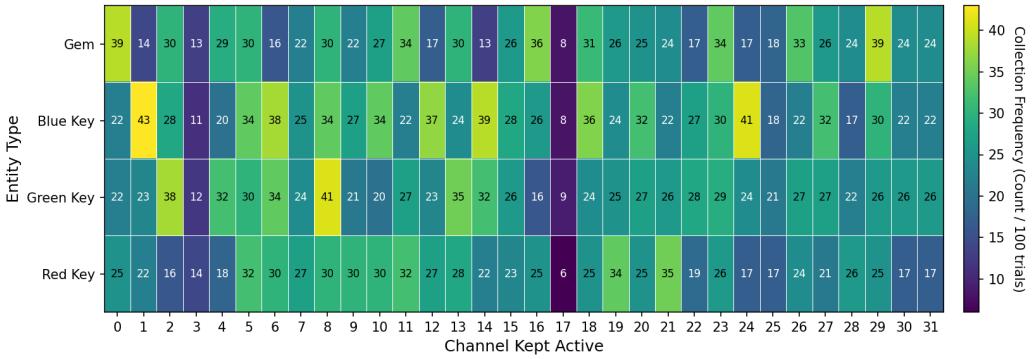


Figure 4.7: Heatmap showing entity collection counts when only a single channel is active (all others ablated). The vertical axis shows different entity types (gem, blue key, green key, red key), while the horizontal axis shows channel indices. Brighter colors (yellow) indicate higher collection success rates.

larger gem spans. When we ablate gem-associated regions, we remove most key steering functionality along with the gem signals. The remaining channels, which still encode some weak gem targeting and are now free of interference from key signals, actually perform better, explaining the improved collection rate.

Second, we discovered that the intervention signals actually interfere with the collection of keys. Ablating navigation channels alone devastated key collection (45% average), but additionally ablating intervention spans partially restored performance (53% average). This suggests that the intervention spans, when present alongside disabled navigation channels, actively interfere with key collection. Removing both eliminates this interference. Ablating intervention spans alone had minimal impact on keys (even causing a slight improvement), indicating that these spans are not necessary for key collection and, in fact, become actively harmful when navigation is impaired.

To better understand the role of the intervention spans in explaining network performance, we performed inverse ablations where we preserved only the intervention spans for each entity. With the updated measurements, inverse performance is 31.8% (gem), 61.8% (blue), 43.6% (green), and 32.5% (red), all below the corresponding random-control rates of 44.1%, 74.9%, 63.1%, and 48.9%. Intervention spans alone are not sufficient to drive collection and do not outperform a random-control baseline. The intervention spans appear to encode some entity information but rely on other navigation circuitry to be effective.

Table 4.3: Effect of targeted ablations on entity collection rates (n=400 trials per condition). Bold values indicate performance improvements or minimal degradation compared to baseline. Values shown with 95% confidence intervals.

| Condition | Gem | Blue Key | Green Key | Red Key |
|---|---|---|---|---|
| Baseline (No Ablation) | **74.5% ± 4.3%** | 98.5% ± 1.2% | 95.0% ± 2.1% | 80.0% ± 3.9% |
| Redundant Navigation Channels | 45.0% ± 4.9% | 69.6% ± 4.5% | 55.9% ± 4.9% | 43.1% ± 4.9% |
| Infrastructure Channels | 27.3% ± 4.4% | 70.2% ± 4.5% | 34.5% ± 4.7% | 26.5% ± 4.3% |
| Intervention Spans | 64.8% ± 4.7% | **99.8% ± 0.4%** | **97.8% ± 1.4%** | **83.2% ± 3.7%** |
| Navigation + Intervention Spans | 32.1% ± 4.6% | 71.9% ± 4.4% | 50.7% ± 4.9% | 37.1% ± 4.7% |
| Preserve Only Intervention Spans | 31.8% ± 4.6% | 61.8% ± 4.8% | 43.6% ± 4.9% | 32.5% ± 4.6% |
| Random Ablation (Control) | 44.1% ± 4.9% | 74.9% ± 4.3% | 63.1% ± 4.7% | 48.9% ± 4.9% |

### 4.5.3 Infrastucture Channels for Each Entity

We additionally share our findings on the different collection rates in the empty environment used in the ablation study as a result of ablating single channels. We ablated the resulting top 10 infrastructure channels for each entity based on the results of the single channel intervention experiment described in 3.2.7 and found that certain important Infrastucture Channels were shared between entities, but also found some specialisation amongst them.

In particular, channel 25 was a critical channel which had a significant impact on collection rates for all entities in the fork maze experiments, reducing collection rates by 18% on average.

The results shown in 4.4 demonstrate the results of ablating the top 10 infrastucture channels for each entity type in the empty maze environment. We observe that that ablating the channels identified as critical for gem collection had a far greater impact on the collection rates of other entities than removing their respective critical top 10 infrastucture channels.

This may be the result of a greater share of the core navigational functionality being encoded in the gem infrastructure channels than in the key infrastructure channels. Interestingly collection of the blue key was more greatly affected by the ablation of the red key's channels than by the ablation of its own infrastucture channels.

Table 4.4: Effect of ablating top 10 infrastructure channels for each entity on collection rates (n=400 trials per condition). Bold values indicate the largest performance decreases for each entity type. Values shown with 95% confidence intervals.

| Condition | Gem | Blue Key | Green Key | Red Key |
|---|---|---|---|---|
| Baseline | 74.5% ± 4.3% | 98.5% ± 1.2% | 95.0% ± 2.1% | 80.0% ± 3.9% |
| Single Channel (Gem) | **27.3% ± 4.4%** | 70.2% ± 4.5% | 34.5% ± 4.7% | 26.5% ± 4.3% |
| Single Channel (Blue Key) | 32.5% ± 4.6% | **80.2% ± 3.9%** | 52.8% ± 4.9% | 38.0% ± 4.8% |
| Single Channel (Green Key) | 23.2% ± 4.1% | 97.5% ± 1.5% | **90.5% ± 2.9%** | 51.7% ± 4.9% |
| Single Channel (Red Key) | 49.8% ± 4.9% | 55.2% ± 4.9% | 49.2% ± 4.9% | **46.0% ± 4.9%** |

*Note: The diagonal shows on-target effects: Gem (-47.2%), Blue Key (-18.3%), Green Key (-4.5%), Red Key (-34.0%). Gem infrastructure channels affect all entities catastrophically, while key channels often have surprising and counter-intuitive impacts reducing their respective entities collection rates less than other entities.*

## 4.6 Summary of Key Findings

Our investigation reveals three fundamental insights into how neural networks implement goal-directed behaviour in complex environments:

**1. Activation Magnitude Encodes Entity Identity.** Our heist network learns to encode targeting information through systematic variations in activation strength rather than dedicating separate channels to each entity. Parallel rollouts show consistent activation offsets between entities (correlation 0.931 in conv4a), with complete behavioural redirection achievable through uniform offset adjustments—shifting from 93% blue key collection at baseline to 94% green key at +4.8 offset. The fact that the network has this strong baseline preference for the blue key, with occasional deviations to the green key validates our hypothesis regarding a deeply learned hierarchical structure. This magnitude-based encoding persists across layers and even through SAE decomposition, confirming it represents a fundamental computational strategy rather than an architectural artifact.

**2. Two-Phase Processing Architecture Separates Goal Selection from Navigation.** Probe experiments reveal distinct information processing stages: entity determination occurs in convolutional layers (peaking at 99.3% accuracy in conv3a) while navigation planning happens in fully-connected layers (peaking at 67.3% directional accuracy in fc2). Conv4a serves as a critical bridge, maintaining entity information while reformatting spatial representations. This separation explains why steering interventions succeed at conv3a/4a—we intervene precisely where goals crystallize, before transformation into distributed action plans.

**3. Layer Depth Determines Intervention Character.** Despite minimal probe accuracy (20.0%), conv1a shows strong but chaotic steering with discrete entity zones and uniquely achieves 57% gem collection—far exceeding later layers. Conv2a offers the finest control with $\pm 0.5$ offset precision, representing the optimal intervention point where representations first integrate. Conv3a/4a provide stable but coarser manipulation of crystallized representations. This progression from raw detection through integration to crystallization defines both the network's computational strategy and our ability to control it.

These findings demonstrate that understanding where and how information is encoded—not just what is encoded—is crucial for interpreting and controlling neural network behaviour.

# Chapter 5

# Discussion

## 5.1 Addressing Research Questions and Hypotheses

Before interpreting our findings in detail, we first highlight the original research questions and hypotheses from Chapter 1 and the extent to which they have been answered.

### 5.1.1 Hypothesis Testing

Table 5.1: Hypothesis outcomes with key supporting evidence.

| H# | Hypothesis | Outcome | Key Evidence |
|---|---|---|---|
| H1 | Dedicated entity channels | **Partial** | Conv1a specialised, conv2-4a multiplex |
| H2 | SAEs disentangle features | **Rejected** | SAEs preserve multiplexing structure |
| H3 | Hierarchical preferences | **Confirmed** | Pickup rates for the blue key of 94% and green key 6% in cross maze |
| H4 | SAEs successfully recover performance | **Confirmed** | 99.6-102% recovery |
| H5 | Layer specialization | **Confirmed** | Entity → direction transition proven |

While some of our initial hypotheses proved correct, we were surprised by the discovery of the activation magnitude mechanism as a means of encoding goals.

We also found that SAEs did not work as expected in this case, but as a result were able to validate some of our new results. We were also able to better characterise some of our initial ideas about how the network might work.

### 5.1.2 Research Questions

Table 5.2: Research questions and their answers based on experimental evidence.

| RQ | Question | Answer |
|---|---|---|
| RQ1 | How are goals encoded? | Via activation magnitude multiplexing across many channels (Section 4.1, 4.2) |
| RQ2 | How is the current target determined? | Two-phase: detection at conv1a → integration at conv2a-conv4a (Table 4.1) |
| RQ3 | Can SAEs clarify feature roles? | Yes, but only providing validation (Section 4.3.2) |
| RQ4 | Dedicated vs general channels? | Conv1a specialised conv2a-conv4a generalised (Appendix .4) |
| RQ5 | Why different steering success rates across entities? | Redundant gem encodings allowed greater steering(Table 4.3) |
| RQ6 | Do layers have specialised roles? | Yes: entity detection in conv layer, navigation in fc2 (Tables 4.1, 4.2) |

## 5.2 Interpretation of Results

### 5.2.1 Entity Encoding as A Core Mechanism

Our findings in Section 4.1 and 4.2 show that rather than than requiring separate channels for each entity type, the model has learned to use activation magnitude as an additional dimension for encoding information.

The ability to completely redirect agent behaviour through uniform activation offsets (93% blue key collection shifting to 94% green key collection in conv4) demonstrates this encoding is fundamental to the network's learned algorithm. This suggests that interventions on neural networks need to consider not just which channels to modify, but also to consider how activation strengths may flip conditions in the network, and in particular if different tasks might cause the

network to shift into different regions of activation space by a constant offset in portions of the network.

Due to the similarity of the task of tracking each key, it proves efficient to repurpose channels already capable of entity detection and allows the reuse of computational machinery with different 'intensity settings', having them target each entity sequentially while indicating the specific entity through activation strength. This strategy likely emerges from efficiency pressures during training wherein the core navigation task is exactly the same, but can simply be repurposed to focus on different entities.

The activation jumps observed regardless of collection order—occurring even when the green key is collected first (12% of cases)—reveal that the network tracks collection progress through entity count, potentially in addition to the specific entity, though they seem to be separate mechanisms responsible for each in the network, since there are clearly parts of the network specifically detecting certain entities as shown by the conv1a probe results presented in Appendix .4.

One challenge in making this discovery was that many of the tools typically used in mechanistic interpretability were fundamentally unsuited to exposing this mechanism in the network, for example SAEs and probes provided no real insight into the levels of activations in response to different parts of the network, and instead appeared to encode many things simultaneously. What appeared to be polysemanticity was instead a different strategy for encoding the information.

### 5.2.2 Two-Phase Information Processing Architecture

The probe results reveal why CNN-FC hybrid architectures are well suited for goal-directed navigation. CNNs capture and classify spatial relationships necessary for identifying 'what is where,' while fully-connected layers performed the task of integrating this information into coherent actions.

The inverse relationship between entity and directional information across layers suggests the network performs active transformation rather than passive feature extraction. Conv4a's role as a bridge between the spatial and fully-connected layers is illustrated by the non-monotonic increase in the linear separability of the direction in which to go from conv3a to fc2. This implies that the linear separability drops due to an active transformation taking place in the structure of the information in conv4a. Future work in understanding this transformation would be valuable in establishing how RL agents transform visual information into motor output.

This occurs at conv3a/conv4a. We're targeting the precise point where the network crystallizes its goal commitment, before that commitment disperses into the distributed representations necessary for action selection. Interventions earlier would

affect raw perception; interventions later would fight against already-distributed action plans.

### 5.2.3 SAE Validation and Dead Channels

The majority of steerable SAE channels were 'dead channels', channels that had 0 activations during rollouts. We hypothesise that when a hidden channel dies during training, gradient flow stops, freezing decoder weights at their last values. This would explain why these frozen weights can still respond to artificial activation during interventions, making steering possible using otherwise silent channels. More importantly, SAEs preserved the same magnitude-based encoding found in the base model rather than decomposing it into cleaner features, suggesting this encoding strategy is fundamental rather than an artifact of polysemanticity.

### 5.2.4 Solution Multiplicity and Representational Drift

A striking observation from our experiments was that intervention positions changed completely across different checkpoints even after convergence. Despite maintaining similar performance, the network continued exploring different encoding schemes within the solution space. This is largely due to the fact that we trained the model with an endless variety of maze variations rather than a fixed number, meaning it continued to adapt and change over time. This configuration was also the only one that led to successful training of our smaller model.

This variety of solutions suggests that amplitude-based multiplexing is not a unique solution but rather one of many equivalent representations the network can adopt, and a pattern which is likely to emerge over the course of training. The continuous drift between encoding schemes post-convergence indicates the network exists on a 'valley floor' of equally viable solutions, constantly reorganizing its internal representations while preserving behavioural performance.

We find a notable checkpoint at update step 30001 where no steerable channels at all are found for any entity, though we only found 1 example of this in the 60 checkpoints we tested, suggesting it might be less optimal than those with the stronger intervention configuration.

### 5.2.5 Conditions Where Activation Multiplexing Is Effective

This finding has particular relevance to RL agents trained in environments that rely on specific state variables in order to operate. For problems like this, two features of the environment are particularly important. One is that the same mechanism will be used repeatedly in each stage, meaning sharing resources wherever possible is optimal. The second is that there is strong pressure to correctly distinguish

between these entities. For example, in the case of Heist it will always use the same circuitry to navigate to the next entity, regardless of its nature, and if it targets the wrong entity it will almost certainly fail to solve the maze, meaning the costs for incorrect targeting is very high. This means taking radical measures to delineate the target by shifting the activation space entirely of many of the channels is an effective strategy.

A similar phenomena could occur in other networks when distinct modes of operation may appear, for example in the case of reasoning LLMs, there may be benefit to multiplexing activations in certain channels to indicate whether it is currently in the reasoning stage of output or in the actual ouptut step. By doing this, certain signals could uniquely activate words such that they have different meanings in the output compared to when they are used in the chain of thought. Further work in this area could be highly instructive.

### 5.2.6 How The Network Selects Targets

We were surprised by the extreme degree of redundancy of steering that could be found in this network, whereby modifying a single channel in a single layer was sufficient to completely alter behaviour. We speculate that a reason for this is due to the network being strongly biased towards targeting via negative activations rather than positive activations. Evidence of this is provided in the mean activations generally being slightly below 0 for all channels during a standard rollout, as seen in Figure 7.

This would mean that positive signals are taken as very strong signals regarding where to go, and thus easily induce a change in actions. This would make sense given that much of the problem involves removing unhelpful noise regarding the positions of the other entities, and thus would require downweighting regions associated with the entities that are not the current targets. This would also explain why becoming more positive over time is effective, as there are less things to downweight as entities are collected and removed from the environment. Proving that the negative activations play this role would be a fruitful direction for future work.

It also seems to be the case that past conv1a, the network generally does not rely heavily on specialised channels for selecting the next target, but rather that many of the channels in conv2a, conv3a, and conv4a play critical roles for multiple or all entities, and that which entity they are currently attending to is largely determined by the overall activation range in the inputs from previous layers. We do see small amounts of specialisation in conv4a with certain channels being quite focused on one or two entities, such as channels 24, 25, and 26 in Figure 4.6 primarily being responsible for the red key and gem.

## 5.3 Theoretical Implications

The findings from this research contribute to the theoretical understanding of mechanistic interpretability in deep reinforcement learning in several ways. The demonstration of activation-level encoding suggests that neural networks may employ more nuanced information encoding strategies than simple feature presence/absence, utilizing the analog nature of activations as an additional information dimension. This challenges some simpler models of feature representation and highlights the potential for information multiplexing as a core principle of neural computation, especially when under pressure for representational efficiency.

The dead channel phenomenon in SAEs, and its mechanistic explanation, provides a concrete example of how network components that appear non-functional during standard inference can retain latent capabilities and learned knowledge. This has implications for how we assess model capacity and robustness, and for understanding the learning dynamics that lead to such states.

One observation from carrying out this work is that uncovering the role of activation magnitude across entire layers as a selection mechanism was difficult to uncover using typical mechanistic interpretability tools such as probes and SAEs. The most important discovery regarding how the network functioned (activation level entity encoding) turned out to be explicable purely by analysing the mean activations of different channels. It is possible that there may be further interesting discoveries to be made using similar approaches, particularly in networks that would likely benefit from shared functionality for similar tasks that require clear delineation.

For RL interpretability specifically, this work underscores that techniques successful in one domain (e.g., LLMs) or simpler environments may need adaptation or re-evaluation in more complex, interactive settings. The interplay between learned environmental structure (like sequential objectives in Heist) and emergent network representations is a key area for future theoretical development.

## 5.4 Practical Applications

The discovery that this network uses activation magnitude for entity encoding may have implications if the effect is observed to generalise to other models. Given the level of that was possible steering through small offset adjustments ($\pm 0.5$ at conv2a) the technique has the potential to offer a lightweight control mechanism that doesn't require architectural changes or retraining. While it remains to be seen to what extent other neural networks have similar encodings, the potential upside of the flexibility of the mechanism and ease of determining whether it is present means that determining if models do work in this way could have immediate practical benefits. It could even be worth structuring environments and training

in such a way as to encourage the development of such a structure in order to allow for greater control via precise intervention, particularly if the mechanism is better understood.

As discussed in Section 5.2, the two-phase processing architecture suggests that interventions should target specific layers depending on the desired effect: earlier convolutional layers for goal selection, FC layers for navigation and planning. This layer-specific approach could make it simpler to debug models by focusing attention on the relevant portion of the network. However, there are still significant challenges in understanding the mechanisms at play in the FC layers of this network.

Most importantly, understanding that representations are most malleable early in the network where they first integrate low level information provides an optimal control point, rather than trying shift the course of the model after entity selection has already occurred.

## 5.5 Limitations of the Study

This research, while providing several insights, is subject to certain limitations:

Our work is currently limited by the fact that we test only on a single environment, as there are not many environments that have similar qualities such as the extreme generalization encouraged by Procgen, and the staged multi-goal configuration in Heist. To address this, future work could attempt this with MiniGrid environments, though a custom configuration that can establish similar diversity may be needed.

It is also unclear to what extent the intervention channel multiplexing phenomena generalises to other architectures, and we plan to test this in future work.

Our training methodology differs from standard practice in using unlimited procedurally generated environments rather than the typical 200-500 set of environments one would set. While this was necessary for successful training with our compressed architecture, it may affect the generalization of our findings to models trained under standard Procgen protocols. The unlimited environment diversity likely also contribute to the representational drift we observe, as the network continually trains on new environmental distributions.

We also do not yet fully characterise the function of the steering spans, or if their steering translates to an actual unique role within the network's natural functioning. We plan to address this directly through a thorough application of probes that are trained to detect which entity is currently the next entity in order of pursuit by the network. Early work shows that the later convolutional layers do this very successfully, as well as the first fully connected layer. However, to truly

understand the role of the intervention spans and navigation channels, it may be necessary to train probes on every channel and every row in the fully connected layer to determine their function.

Intervention experiments, by their nature, can push the network into off-distribution activation states. While this is useful for causal testing, the agent's behaviour under such strong artificial interventions do not represent its typical behaviour. We do not yet have a clear picture of precise circuitry under normal operating conditions. Additionally, the steering interventions were typically only applied with positive activations, but the mean channel activation analysis shown in Figure 4.3 reveals that for many channels the activations of the network are mostly negative. This means that a large portion of their activity is not being captured accurately in the interventions. We believe future work in understanding the role of the negative activations in the convolutional channels would lead to fruitful insights.

Our approach of simply intervening in every channel with 400 different activation strengths is also an approach that would need to be approached more carefully, due to cost constraints, in larger models.

This research explored only a single environment (Procgen Heist) and architecture (5-conv, 2 FC layer Impala model) under specific training conditions. Whether activation-level encoding generalises to other architectures, environments, and training regimes remains open. Our interventions often pushed networks into off-distribution states beyond normal ranges, and used positive activations despite many channels operating mostly in negative ranges, leaving many of the specific mechanisms of the network uncharacterised.

## 5.6 Future Work

This research opens several promising avenues for future investigation. The discovery of activation-magnitude encoding suggests examining whether similar mechanisms exist in other architectures and domains, particularly in transformer-based models where activation magnitudes might encode different modes of operation.

Testing this mechanism across different RL environments—especially those with hierarchical goal structures or mode-switching requirements—could reveal whether activation multiplexing is a general principle of efficient neural computation under resource constraints.

The methodological insight that simple mean activation analysis revealed what sophisticated tools missed suggests revisiting other 'well-understood' models with basic statistical approaches. This could uncover overlooked organizational principles, particularly in systems previously deemed too polysemantic for interpretation.

The precise control achievable through small activation offsets ($\pm 0.5$ at conv2a) warrants investigation into whether training procedures could be designed to encourage such structures, potentially yielding models that are interpretable and controllable by design rather than post-hoc analysis.

Finally, the two-phase architecture's clear separation between 'what to target' and 'how to navigate' suggests applications in hybrid systems where interpretable goal selection could be combined with powerful but opaque execution modules, potentially offering a path toward more trustworthy autonomous systems.

## 5.7 Reflections on the Research Process

This research undertook many iterations, with the most important insight only occurring very close to the end, discovering the importance of the activation values as being key for entity targeting. Early hints of this were present in the form of visualising the activations and seeing the shift in activations as different entities were collected, but rigorously showing this with a simple experiment was a significant breakthrough that led to real insight. The key insight that led to this was from the steering activation ranges, and their multiplexed structure. Testing this result rigorously for validity led to the observation of the stronger activation offset phenomenon.

One of the most important reflections of this was how relying on existing MI techniques failed to provide a good way of discovering this. A great deal of our effort and research time went towards the SAEs and implementing them, but the majority of real insights came from simple techniques such as probes, and our mean activation offset analysis.

If our contribution of simply observing mean activations across different parts of the network can be transferred to other models and contexts, this would be very encouraging and may lead to new ways of thinking about models. The simplicity of the technique means that it theoretically can be applied to almost any model, though it will likely be highly variable in its usefulness. The abstraction of channels in our convolutional layers were useful high levels units to examine, and which are not present in many model architectures.

Another insight was that the idea of the network being too polysemantic to be a good target for analysis also led to discouragement on the viability of the project as a whole, and led to the implementation of the SAEs. The later insight was the network was not truly polysemantic, but that it recycled circuits by creating unique conditions through the activation ranges being used at any given time. The 'polysemanticity' was an important hint of this mechanism at play.

For people who aim to do similar work in future, a critical learning we would want to share would be the need for simple experiments, such as the structure

of the T-maze in 3.2 which allowed us to isolate the key variables. It was also essential to focus on the structure of activations over the entire course of a rollout, which provided far more information than what could be gleaned from analysing individual observations and their respective activations. The context provided by the full rollout was essential to understanding the network.

# Chapter 6

# Conclusion

## 6.1 Summary of the Research

This work investigated how deep reinforcement learning agents internally represent and pursue sequential goals in the procedurally generated Procgen Heist environment. Using mechanistic interpretability techniques such as linear probes, activation interventions, parallel rollout analysis, and SAEs, we investigated the core mechanism behind how our network learned to encode goal structures in this multi-objective task and whether interpretability tools developed for language models translate to RL agents.

## 6.2 Key Findings and Contributions

We primarily demonstrate the unusual and previously undocumented phenomena of the network encoding specific entities using activation ranges to track state variables. We show this through the clear parallel trajectories of activations as demonstrated by the parallel rollout experiments that have an extremely high correlation (r=0.931 in conv4a), demonstrating that the process being used to target each entity remains constant while the value strength shifts significantly. Most surprisingly we are able to get clear and consistent shifts in behaviour that maintain core network functions with only a simple constant offset to all channel weights, a remarkably strong result for such a simple intervention. This highlights a significiant gap in current approaches to MI research, which focuses primarily on identifying which parts of the network are responsible for a certain activation, but fails to consider how varying activation strengths may shift network behaviour.

We further identified a bimodal structure in the network regarding target entity identification early in the network, and a surge in correctly identifying the correct direction to travel later in the network. Entity information crystallizes to 99.3%

probe accuracy in conv3a before collapsing to 20.4% in fc3, while directional information is not as strong in early layers but strengthens to 67.3% in fc2. This reveals that convolutional layers determine where things are in the maze before the FC layers determine how to arrive there. This explains why our interventions in the convolutional layers are able to succeed before targets have crystallized.

We also found that SAEs, while of some use in validating our results, failed to provide a great deal of additional insight into the network that would have not been gained otherwise. This highlights the potential pitfalls of trying to apply promising tools before considering simple interventions and experiments that may lead to more insight.

## 6.3 Final Remarks

This thesis demonstrates the practicality of understanding such neural networks, though it also highlights some key bottlenecks in doing this at scale. In particular, many of the tools that have been developed to interpret such models were not able to capture key parts of the networks functioning. This highlights that there are still significant gaps in the way that we approach thinking about neural networks, and in particular networks trained with RL. It also highlights the importance of examining fundamental structures such as activation patterns and high level statistics as a simple first step in approaching such problems.

The two-phase processing architecture, with convolutional layers determining the structure of the maze and positions of entities and fully-connected layers determining 'how to get there', while not surprising as a structure matches what we might expect from such a network and provides tangible evidence of theoretical predictions. This provides the opportunity for hands on debugging and potential pathways to debugging issues and achieving precise and simple control of such models.

Looking forward, activation-level encoding may extend beyond this specific environment and architecture. The conditions that encourage the development of these structures: repeated use of similar circuitry for structurally similar subtasks that require clear delineation, appear across many domains. If activation magnitude proves to be a general mechanism for information multiplexing under efficiency constraints, it would significantly impact how we build, interpret, and control neural networks. The simplicity and cheapness of applying this techniques makes it particularly appealing. More work is required to verify this phenomena across other models, and to determine how different types of models and domains change the way in which it might manifest.

Given this potential, we hope that understanding this particular toy model may alter the way the reader thinks about neural networks and approaching under-

standing them, and that cracking the problem of the inner workings of such models may happen may be easier as a result.

# Bibliography


Aharon, M., M. Elad, and A. Bruckstein (Nov. 2006). "K-SVD: An Algorithm for Designing Overcomplete Dictionaries for Sparse Representation". In: *IEEE Transactions on Signal Processing* 54.11, pp. 4311–4322. ISSN: 1941-0476. DOI: 10.1109/TSP.2006.881199. (Visited on 10/06/2025).

Alain, Guillaume and Yoshua Bengio (Nov. 2018). *Understanding Intermediate Layers Using Linear Classifier Probes*. DOI: 10.48550/arXiv.1610.01644. arXiv: 1610.01644 [stat]. (Visited on 09/13/2025).

Amodei, Dario, Chris Olah, Jacob Steinhardt, Paul Christiano, John Schulman, and Dan Mané (2016). "Concrete Problems in AI Safety". In: *arXiv preprint arXiv:1606.06565*. arXiv: 1606.06565.

Atrey, Akanksha, Kaleigh Clary, and David Jensen (Feb. 2020). *Exploratory Not Explanatory: Counterfactual Analysis of Saliency Maps for Deep Reinforcement Learning*. arXiv: 1912.05743 [cs]. (Visited on 08/18/2024).

Bau, David, Bolei Zhou, Aditya Khosla, Aude Oliva, and Antonio Torralba (Apr. 2017). *Network Dissection: Quantifying Interpretability of Deep Visual Representations*. DOI: 10.48550/arXiv.1704.05796. arXiv: 1704.05796 [cs]. (Visited on 08/22/2025).

Beck, Amir and Marc Teboulle (2009). "A Fast Iterative Shrinkage-Thresholding Algorithm for Linear Inverse Problems". In: *SIAM Journal on Imaging Sciences* 2.1, pp. 183–202. DOI: 10.1137/080716542. eprint: https://doi.org/10.1137/080716542.

Bellemare, Marc G., Yavar Naddaf, Joel Veness, and Michael Bowling (2013). "The Arcade Learning Environment: An Evaluation Platform for General Agents". In: *Journal of Artificial Intelligence Research* 47, pp. 253–279.

Betley, Jan (July 2023). "Localizing Goal Misgeneralization in a Maze-Solving Policy Network". In: (visited on 03/10/2025).

Bostrom, Nick and Nick Bostrom (July 2014). *Superintelligence: Paths, Dangers, Strategies*. Oxford, New York: Oxford University Press. ISBN: 978-0-19-967811-2.

Bostrom, Nick and Eliezer Yudkowsky (2014). "The Ethics of Artificial Intelligence". In: *The Cambridge Handbook of Artificial Intelligence*. Ed. by Keith Frankish and William M. Ramsey. Cambridge: Cambridge University Press, pp. 316–334. ISBN: 978-0-521-87142-6. DOI: 10.1017/CBO9781139046855.020. (Visited on 09/13/2025).

Bricken, Trenton, Adly Templeton, Joshua Batson, Brian Chen, Adam Jermyn, Tom Conerly, Nick Turner, Cem Anil, Carson Denison, Amanda Askell, Robert Lasenby, Yifan Wu, Shauna Kravec, Nicholas Schiefer, Tim Maxwell, Nicholas Joseph, Zac Hatfield-Dodds, Alex Tamkin, Karina Nguyen, Brayden McLean, Josiah E Burke, Tristan Hume, Shan Carter, Tom Henighan, and Christopher Olah (2023). "Towards Monosemanticity: Decomposing Language Models with Dictionary Learning". In: *Transformer Circuits Thread*.

Cammarata, Nick, Gabriel Goh, Shan Carter, Ludwig Schubert, Michael Petrov, and Chris Olah (2020). "Curve Detectors". In: *Distill*. DOI: 10.23915/distill.00024.003.

Casademunt, Helena, Caden Juang, Adam Karvonen, Samuel Marks, Senthooran Rajamanoharan, and Neel Nanda (July 2025). *Steering Out-of-Distribution Generalization with Concept Ablation Fine-Tuning*. DOI: 10.48550/arXiv.2507.16795. arXiv: 2507.16795 [cs]. (Visited on 09/13/2025).

Cobbe, Karl, Christopher Hesse, Jacob Hilton, and John Schulman (July 2020). *Leveraging Procedural Generation to Benchmark Reinforcement Learning*. DOI: 10.48550/arXiv.1912.01588. arXiv: 1912.01588 [cs, stat]. (Visited on 08/08/2024).

Cobbe, Karl, Oleg Klimov, Chris Hesse, Taehoon Kim, and John Schulman (June 2019). "Quantifying Generalization in Reinforcement Learning". In: *Proceedings of the 36th International Conference on Machine Learning*. Ed. by Kamalika Chaudhuri and Ruslan Salakhutdinov. Vol. 97. Proceedings of Machine Learning Research. PMLR, pp. 1282–1289.

Conerly, Tom, Adly Templeton, Trenton Bricken, Jonathan Marcus, and Tom Henighan (Apr. 2024). *Update on How We Train SAEs*. (Visited on 04/13/2025).

Cunningham, Hoagy, Alwin Peng, Jerry Wei, Euan Ong, Fabien Roger, Linda Petrini, Misha Wagner, Vladimir Mikulik, and Mrinank Sharma (Jan. 2025). "Cost-Effective Constitutional Classifiers via Representation Re-Use". In: *Anthropic Alignment Science Blog*.

Dalvi, Fahim, Nadir Durrani, Hassan Sajjad, and Preslav Nakov (2021). "On the Effect of Dropping Layers of Pre-Trained Transformer Models". In: *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing (EMNLP)*.

Degrave, J., F. Felici, J. Buchli, M. Neunert, B. Strouse, T. Hottiger, J. Byrnes, R. de Folter, A. Laterre, M. van de Walle, J. van Dorland, T. Laan, L. Ledsgaard, G. Sartoretti, K. Sünderhauf, A. Bridgwater, A. Ecuyer-Wallaert, N. Heess, Y. Tassa, R. Hafner, M. Riedmiller, and T. Lillicrap (Feb. 2022). "Magnetic Control of Tokamak Plasmas through Deep Reinforcement Learning". In: *Nature* 602.7897, pp. 414–419. DOI: 10.1038/s41586-021-04301-9.

Elhage, Nelson, Tristan Hume, Catherine Olsson, Nicholas Schiefer, Tom Henighan, Shauna Kravec, Zac Hatfield-Dodds, Robert Lasenby, Dawn Drain, Carol Chen, Roger Grosse, Sam McCandlish, Jared Kaplan, Dario Amodei, Martin Wattenberg, and Christopher Olah (Sept. 2022). "Toy Models of Superposition". In: *Transformer Circuits Thread*. (Visited on 04/13/2025).

Elhage, Nelson, Neel Nanda, Catherine Olsson, Tom Henighan, Nicholas Joseph, Ben Mann, Amanda Askell, Yuntao Bai, Anna Chen, Tom Conerly, Nova DasSarma, Dawn Drain, Deep Ganguli, Zac Hatfield-Dodds, Danny Hernandez, Andy Jones, Jackson Kernion, Liane Lovitt, Kamal Ndousse, Dario Amodei, Tom Brown, Jack Clark, Jared Kaplan, Sam McCandlish, and Chris Olah (2021). "A Mathematical Framework for Transformer Circuits". In: *Transformer Circuits Thread*.

Engstrom, Logan, Andrew Ilyas, Shibani Santurkar, Dimitris Tsipras, Firdaus Janoos, Larry Rudolph, and Aleksander Madry (2020). "Implementation Matters in Deep Policy Gradients: A Case Study on PPO and TRPO". In: *arXiv preprint arXiv:2005.12729*. arXiv: 2005.12729.

Espeholt, Lasse, Hubert Soyer, Remi Munos, Karen Simonyan, Vlad Mnih, Tom Ward, Yotam Doron, Vlad Firoiu, Tim Harley, Iain Dunning, Shane Legg, and Koray Kavukcuoglu (July 2018). "IMPALA: Scalable Distributed Deep-RL with Importance Weighted Actor-Learner Architectures". In: *Proceedings of the 35th International Conference on Machine Learning*. Ed. by Jennifer Dy and Andreas Krause. Vol. 80. Proceedings of Machine Learning Research. PMLR, pp. 1407–1416.

García, Javier and Fernando Fernández (2015). "A Comprehensive Survey on Safe Reinforcement Learning". In: *Journal of Machine Learning Research* 16.1, pp. 1437–1480.

Gorton, Liv (June 2024). *The Missing Curve Detectors of InceptionV1: Applying Sparse Autoencoders to InceptionV1 Early Vision*. DOI: 10.48550/arXiv.2406.03662. arXiv: 2406.03662 [cs]. (Visited on 04/12/2025).

Greydanus, Samuel, Anurag Koul, Jonathan Dodge, and Alan Fern (July 2018). "Visualizing and Understanding Atari Agents". In: *Proceedings of the 35th International Conference on Machine Learning*. PMLR, pp. 1792–1801. (Visited on 06/19/2025).

Gurnee, Wes, Neel Nanda, Matthew Pauly, Katherine Harvey, Dmitrii Troitskii, and Dimitris Bertsimas (June 2023). *Finding Neurons in a Haystack: Case Studies with Sparse Probing*. DOI: 10.48550/arXiv.2305.01610. arXiv: 2305.01610 [cs]. (Visited on 09/04/2025).

Gurnee, Wes and Max Tegmark (2023). "Language Models Represent Space and Time". In: *The Eleventh International Conference on Learning Representations (ICLR)*.

He, Kaiming, Xiangyu Zhang, Shaoqing Ren, and Jian Sun (Dec. 2015). *Deep Residual Learning for Image Recognition*. DOI: 10.48550/arXiv.1512.03385. arXiv: 1512.03385 [cs]. (Visited on 09/06/2025).

— (July 2016). *Identity Mappings in Deep Residual Networks*. DOI: 10.48550/arXiv.1603.05027. arXiv: 1603.05027 [cs]. (Visited on 09/06/2025).

Hendrycks, Dan, Mantas Mazeika, and Thomas Woodside (Oct. 2023). *An Overview of Catastrophic AI Risks*. DOI: 10.48550/arXiv.2306.12001. arXiv: 2306.12001 [cs]. (Visited on 09/12/2025).

Hilton, Jacob, Nick Cammarata, Shan Carter, Gabriel Goh, and Chris Olah (Nov. 2020). "Understanding RL Vision". In: *Distill* 5.11, e29. ISSN: 2476-0757. DOI: 10.23915/distill.00029. (Visited on 04/11/2025).

Hubinger, Evan, Richard van der-Merwe, Vladimir Mikulik, Adam Lan, Sebastian Farquhar, Mark Whalen, John Bottomley, Lorand Nemeth, Zachary Kenton, Andrew Lohn, et al. (2019). "Risks from Learned Optimization in Advanced Machine Learning Systems". In: *arXiv preprint arXiv:1906.01820*. arXiv: 1906.01820.

Kim, Been, Martin Wattenberg, Justin Gilmer, Carrie Cai, James Wexler, Fernanda Viegas, and Rory Sayres (June 2018). *Interpretability Beyond Feature Attribution: Quantitative Testing with Concept Activation Vectors (TCAV)*. DOI: 10.48550/arXiv.1711.11279. arXiv: 1711.11279 [stat]. (Visited on 06/18/2025).

Koch, Jack, Lauro Langosco, Jacob Pfau, James Le, and Lee Sharkey (2021). "Objective Robustness in Deep Reinforcement Learning". In: *arXiv preprint arXiv:2105.14111* 2. arXiv: 2105.14111.

Krakovna, Victoria (Apr. 2020). *Specification Gaming: The Flip Side of AI Ingenuity*. https://deepmind.google/discover/blog/specification-gaming-the-flip-side-of-ai-ingenuity/. (Visited on 09/13/2025).

— (n.d.). *Specification Gaming Examples in AI*.

Langosco, Lorenzo, Rohin Shah, Victoria Krakovna, Gillian Hadfield, Dario Amodei, et al. (2022). "Goal Misgeneralization in Deep Reinforcement Learning". In: *NeurIPS*.

Li, Kenneth, Aspen K Hopkins, David Bau, Fernanda Viégas, Hanspeter Pfister, and Martin Wattenberg (2022). "Emergent World Representa-

tions: Exploring a Sequence Model Trained on Othello". In: *arXiv preprint arXiv:2210.13382*. arXiv: 2210.13382.

Li, Kenneth, Aspen K. Hopkins, David Bau, Fernanda Viégas, Hanspeter Pfister, and Martin Wattenberg (June 2024). *Emergent World Representations: Exploring a Sequence Model Trained on a Synthetic Task*. DOI: 10.48550/arXiv.2210.13382. arXiv: 2210.13382 [cs]. (Visited on 09/19/2025).

Li, Maximilian and Lucas Janson (Sept. 2024). *Optimal Ablation for Interpretability*. DOI: 10.48550/arXiv.2409.09951. arXiv: 2409.09951 [cs]. (Visited on 09/25/2025).

Lundberg, Scott M. and Su-In Lee (Dec. 2017). "A Unified Approach to Interpreting Model Predictions". In: *Proceedings of the 31st International Conference on Neural Information Processing Systems*. NIPS'17. Red Hook, NY, USA: Curran Associates Inc., pp. 4768–4777. ISBN: 978-1-5108-6096-4. (Visited on 09/14/2025).

Marks, Samuel, Johannes Treutlein, Trenton Bricken, Jack Lindsey, Jonathan Marcus, Siddharth Mishra-Sharma, Daniel Ziegler, Emmanuel Ameisen, Joshua Batson, Tim Belonax, Samuel R. Bowman, Shan Carter, Brian Chen, Hoagy Cunningham, Carson Denison, Florian Dietz, Satvik Golechha, Akbir Khan, Jan Kirchner, Jan Leike, Austin Meek, Kei Nishimura-Gasparian, Euan Ong, Christopher Olah, Adam Pearce, Fabien Roger, Jeanne Salle, Andy Shih, Meg Tong, Drake Thomas, Kelley Rivoire, Adam Jermyn, Monte MacDiarmid, Tom Henighan, and Evan Hubinger (Mar. 2025). *Auditing Language Models for Hidden Objectives*. DOI: 10.48550/arXiv.2503.10965. arXiv: 2503.10965 [cs]. (Visited on 09/13/2025).

McDougall, Callum, Arthur Conmy, Cody Rushing, Thomas McGrath, and Neel Nanda (Oct. 2023). *Copy Suppression: Comprehensively Understanding an Attention Head*. DOI: 10.48550/arXiv.2310.04625. arXiv: 2310.04625 [cs]. (Visited on 10/28/2025).

McGrath, Thomas, Andrei Kapishnikov, Nenad Tomašev, Adam Pearce, Demis Hassabis, Been Kim, Ulrich Paquet, and Vladimir Kramnik (Nov. 2022). "Acquisition of Chess Knowledge in AlphaZero". In: *Proceedings of the National Academy of Sciences* 119.47, e2206625119. ISSN: 0027-8424, 1091-6490. DOI: 10.1073/pnas.2206625119. arXiv: 2111.09259 [cs, stat]. (Visited on 07/22/2024).

Meng, Kevin, David Bau, Alex Andonian, and Yonatan Belinkov (Jan. 2023). *Locating and Editing Factual Associations in GPT*. arXiv: 2202.05262 [cs]. (Visited on 01/22/2024).

Mini, Ulisse, Peli Grietzer, Mrinank Sharma, Austin Meek, Monte MacDiarmid, and Alexander Matt Turner (Oct. 2023). *Understanding and Con-*

*trolling a Maze-Solving Policy Network*. DOI: 10.48550/arXiv.2310.08043. arXiv: 2310.08043 [cs]. (Visited on 04/13/2025).

Mnih, Volodymyr, Adria Puigdomenech Badia, Mehdi Mirza, Alex Graves, Timothy Lillicrap, Tim Harley, David Silver, and Koray Kavukcuoglu (June 2016). "Asynchronous Methods for Deep Reinforcement Learning". In: *Proceedings of The 33rd International Conference on Machine Learning*. PMLR, pp. 1928–1937. (Visited on 09/05/2025).

Mnih, Volodymyr, Koray Kavukcuoglu, David Silver, Alex Graves, Ioannis Antonoglou, Daan Wierstra, and Martin Riedmiller (Dec. 2013). *Playing Atari with Deep Reinforcement Learning*. DOI: 10.48550/arXiv.1312.5602. arXiv: 1312.5602 [cs]. (Visited on 05/18/2023).

Mnih, Volodymyr, Koray Kavukcuoglu, David Silver, Andrei A. Rusu, Joel Veness, Marc G. Bellemare, Alex Graves, Martin Riedmiller, Andreas K. Fidjeland, Georg Ostrovski, Stig Petersen, Charles Beattie, Amir Sadik, Ioannis Antonoglou, Helen King, Dharshan Kumaran, Daan Wierstra, Shane Legg, and Demis Hassabis (Feb. 2015). "Human-Level Control through Deep Reinforcement Learning". In: *Nature* 518.7540, pp. 529–533. ISSN: 1476-4687. DOI: 10.1038/nature14236. (Visited on 09/05/2025).

Nakabi, Taha Abdelhalim and Pekka Toivanen (Mar. 2021). "Deep Reinforcement Learning for Energy Management in a Microgrid with Flexible Demand". In: *Sustainable Energy, Grids and Networks* 25, p. 100413. ISSN: 2352-4677. DOI: 10.1016/j.segan.2020.100413. (Visited on 10/05/2025).

Nanda, Neel, Andrew Lee, and Martin Wattenberg (Sept. 2023). *Emergent Linear Representations in World Models of Self-Supervised Sequence Models*. DOI: 10.48550/arXiv.2309.00941. arXiv: 2309.00941 [cs]. (Visited on 06/18/2025).

Olah, Chris, Nick Cammarata, Ludwig Schubert, Gabriel Goh, Michael Petrov, and Shan Carter (Mar. 2020). "Zoom In: An Introduction to Circuits". In: *Distill* 5.3, 10.23915/distill.00024.001. ISSN: 2476-0757. DOI: 10.23915/distill.00024.001. (Visited on 03/07/2025).

Olah, Chris, Alexander Mordvintsev, and Ludwig Schubert (Nov. 2017). *Feature Visualization*. DOI: 10.23915/distill.00007. (Visited on 04/13/2025).

Olah, Chris, Arvind Satyanarayan, Ian Johnson, Shan Carter, Ludwig Schubert, Katherine Ye, and Alexander Mordvintsev (2018). "The Building Blocks of Interpretability". In: *Distill*. DOI: 10.23915/distill.00010.

Olshausen, Bruno A. and David J. Field (June 1996). "Emergence of Simple-Cell Receptive Field Properties by Learning a Sparse Code for Natural Images". In: *Nature* 381.6583, pp. 607–609. ISSN: 1476-4687. DOI: 10.1038/381607a0. (Visited on 10/06/2025).

Olsson, Catherine, Nelson Elhage, Neel Nanda, Nicholas Joseph, Nova DasSarma, Tom Henighan, Ben Mann, Amanda Askell, Yuntao Bai, Anna Chen, Tom Conerly, Dawn Drain, Deep Ganguli, Zac Hatfield-Dodds, Danny Hernandez, Scott Johnston, Andy Jones, Jackson Kernion, Liane Lovitt, Kamal Ndousse, Dario Amodei, Tom Brown, Jack Clark, Jared Kaplan, Sam McCandlish, and Chris Olah (Sept. 2022). *In-Context Learning and Induction Heads*. DOI: 10.48550/arXiv.2209.11895. arXiv: 2209.11895 [cs]. (Visited on 09/25/2025).

Omohundro, Stephen M. (June 2008). "The Basic AI Drives". In: *Proceedings of the 2008 Conference on Artificial General Intelligence 2008: Proceedings of the First AGI Conference*. NLD: IOS Press, pp. 483–492. ISBN: 978-1-58603-833-5. (Visited on 04/14/2025).

Pearl, Judea (2009). *Causality: Models, Reasoning and Inference*. 2nd ed. USA: Cambridge University Press. ISBN: 0-521-89560-X.

Ribeiro, Marco Tulio, Sameer Singh, and Carlos Guestrin (Aug. 2016). *"Why Should I Trust You?": Explaining the Predictions of Any Classifier*. DOI: 10.48550/arXiv.1602.04938. arXiv: 1602.04938 [cs]. (Visited on 09/14/2025).

Schulman, John, Sergey Levine, Philipp Moritz, Michael I. Jordan, and Pieter Abbeel (Apr. 2017). *Trust Region Policy Optimization*. DOI: 10.48550/arXiv.1502.05477. arXiv: 1502.05477 [cs]. (Visited on 09/04/2025).

Schulman, John, Philipp Moritz, Sergey Levine, Michael Jordan, and Pieter Abbeel (Oct. 2018). *High-Dimensional Continuous Control Using Generalized Advantage Estimation*. DOI: 10.48550/arXiv.1506.02438. arXiv: 1506.02438 [cs]. (Visited on 09/03/2025).

Schulman, John, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov (Aug. 2017). *Proximal Policy Optimization Algorithms*. DOI: 10.48550/arXiv.1707.06347. arXiv: 1707.06347 [cs]. (Visited on 09/04/2025).

Selvaraju, Ramprasaath R., Michael Cogswell, Abhishek Das, Ramakrishna Vedantam, Devi Parikh, and Dhruv Batra (Oct. 2017). "Grad-CAM: Visual Explanations from Deep Networks via Gradient-Based Localization". In: *2017 IEEE International Conference on Computer Vision (ICCV)*, pp. 618–626. DOI: 10.1109/ICCV.2017.74. (Visited on 09/14/2025).

Shihab, Ibne Farabi, Sanjeda Akter, and Anuj Sharma (July 2025). *Detecting and Mitigating Reward Hacking in Reinforcement Learning Systems: A Comprehensive Empirical Study*. DOI: 10.48550/arXiv.2507.05619. arXiv: 2507.05619 [cs]. (Visited on 09/13/2025).

Silver, David, Aja Huang, Chris J. Maddison, Arthur Guez, Laurent Sifre, George van den Driessche, Julian Schrittwieser, Ioannis Antonoglou, Veda Panneershelvam, Marc Lanctot, Sander Dieleman, Dominik Grewe, John Nham, Nal Kalchbrenner, Ilya Sutskever, Timothy Lillicrap, Madeleine

Leach, Koray Kavukcuoglu, Thore Graepel, and Demis Hassabis (Jan. 2016). "Mastering the Game of Go with Deep Neural Networks and Tree Search". In: *Nature* 529.7587, pp. 484–489. ISSN: 1476-4687. DOI: 10.1038/nature16961.

Simonyan, Karen, Andrea Vedaldi, and Andrew Zisserman (2014). "Deep inside Convolutional Networks: Visualising Image Classification Models and Saliency Maps". In: *Workshop at International Conference on Learning Representations*.

Sundararajan, Mukund, Ankur Taly, and Qiqi Yan (July 2017). "Axiomatic Attribution for Deep Networks". In: *Proceedings of the 34th International Conference on Machine Learning*. PMLR, pp. 3319–3328. (Visited on 06/19/2025).

Sutton, R.S. and A.G. Barto (Sept. 1998). "Reinforcement Learning: An Introduction". In: *IEEE Transactions on Neural Networks* 9.5, pp. 1054–1054. ISSN: 1941-0093. DOI: 10.1109/TNN.1998.712192. (Visited on 09/05/2025).

Sutton, Richard S and Andrew G Barto (2018). *Reinforcement Learning: An Introduction*. MIT press.

Sutton, Richard S, David McAllester, Satinder Singh, and Yishay Mansour (1999). "Policy Gradient Methods for Reinforcement Learning with Function Approximation". In: *Advances in Neural Information Processing Systems*. Vol. 12. MIT Press. (Visited on 09/03/2025).

Templeton, Adly, Tom Conerly, Jonathan Marcus, Jack Lindsey, Trenton Bricken, Brian Chen, Adam Pearce, Craig Citro, Emmanuel Ameisen, Andy Jones, Hoagy Cunningham, Nicholas L Turner, Callum McDougall, Monte MacDiarmid, Alex Tamkin, Esin Durmus, Tristan Hume, Francesco Mosconi, C. Daniel Freeman, Theodore R. Sumers, Edward Rees, Joshua Batson, Adam Jermyn, Shan Carter, Chris Olah, and Tom Henighan (May 2024). *Scaling Monosemanticity: Extracting Interpretable Features from Claude 3 Sonnet*. (Visited on 04/12/2025).

Thatte, Akshay, Anirudh G. Mini, Jacob Hilton, Aditi Khanna, Pieter Abbeel, and Rohan Taori Krishna (2022). "Insights into Representations Learned by Reinforcement Learning Agents". In: *NeurIPS 2022 Workshop on Visually Grounded Reinforcement Learning*.

Trim, Tristan and Triston Grayston (Oct. 2024). *Mechanistic Interpretability of Reinforcement Learning Agents*. DOI: 10.48550/arXiv.2411.00867. arXiv: 2411.00867 [cs]. (Visited on 11/22/2025).

Turner, Alexander Matt, Lisa Thiergart, Gavin Leech, David Udell, Juan J. Vazquez, Ulisse Mini, and Monte MacDiarmid (June 2024a). *Activation Addition: Steering Language Models Without Optimization*. DOI:

10.48550/arXiv.2308.10248. arXiv: 2308.10248 [cs]. (Visited on 04/13/2025).

Turner, Alexander Matt, Lisa Thiergart, Gavin Leech, David Udell, Juan J. Vazquez, Ulisse Mini, and Monte MacDiarmid (Oct. 2024b). *Steering Language Models With Activation Engineering*. DOI: 10.48550/arXiv.2308.10248. arXiv: 2308.10248 [cs]. (Visited on 12/07/2024).

Wang, Kevin, Alexandre Variengien, Arthur Conmy, Buck Shlegeris, and Jacob Steinhardt (Nov. 2022). *Interpretability in the Wild: A Circuit for Indirect Object Identification in GPT-2 Small*. DOI: 10.48550/arXiv.2211.00593. arXiv: 2211.00593 [cs]. (Visited on 10/28/2025).

Weng, Jiayi, Min Lin, Shengyi Huang, Bo Liu, Denys Makoviichuk, Viktor Makoviychuk, Zichen Liu, Yufan Song, Ting Luo, Yukun Jiang, Zhongwen Xu, and Shuicheng Yan (2022). "EnvPool: A Highly Parallel Reinforcement Learning Environment Execution Engine". In: *Advances in Neural Information Processing Systems 35*. Ed. by Sameer Koyejo, Shakir Mohamed, A. Agarwal, D. Belgrave, K. Cho, and A. Oh. Curran Associates, Inc., pp. 22409–22421.

Yudkowsky, Eliezer and Eliezer Yudkowsky (July 2008). "Artificial Intelligence as a Positive and Negative Factor in Global Risk". In: *Global Catastrophic Risks*. Oxford University Press. ISBN: 978-0-19-857050-9 978-0-19-191810-0. DOI: 10.1093/oso/9780198570509.003.0021. (Visited on 09/13/2025).

Zeiler, Matthew D. and Rob Fergus (Nov. 2013). *Visualizing and Understanding Convolutional Networks*. DOI: 10.48550/arXiv.1311.2901. arXiv: 1311.2901 [cs]. (Visited on 08/22/2025).

Zou, Andy, Long Phan, Sarah Chen, James Campbell, Phillip Guo, Richard Ren, Alexander Pan, Xuwang Yin, Mantas Mazeika, Ann-Kathrin Dombrowski, Shashwat Goel, Nathaniel Li, Michael J. Byun, Zifan Wang, Alex Mallen, Steven Basart, Sanmi Koyejo, Dawn Song, Matt Fredrikson, J. Zico Kolter, and Dan Hendrycks (Mar. 2025). *Representation Engineering: A Top-Down Approach to AI Transparency*. DOI: 10.48550/arXiv.2310.01405. arXiv: 2310.01405 [cs]. (Visited on 09/14/2025).

# Appendix A

# Appendix Title

## .1 Technical Appendices

## .1 Model Architecture Details

ImpalaCNN architecture:

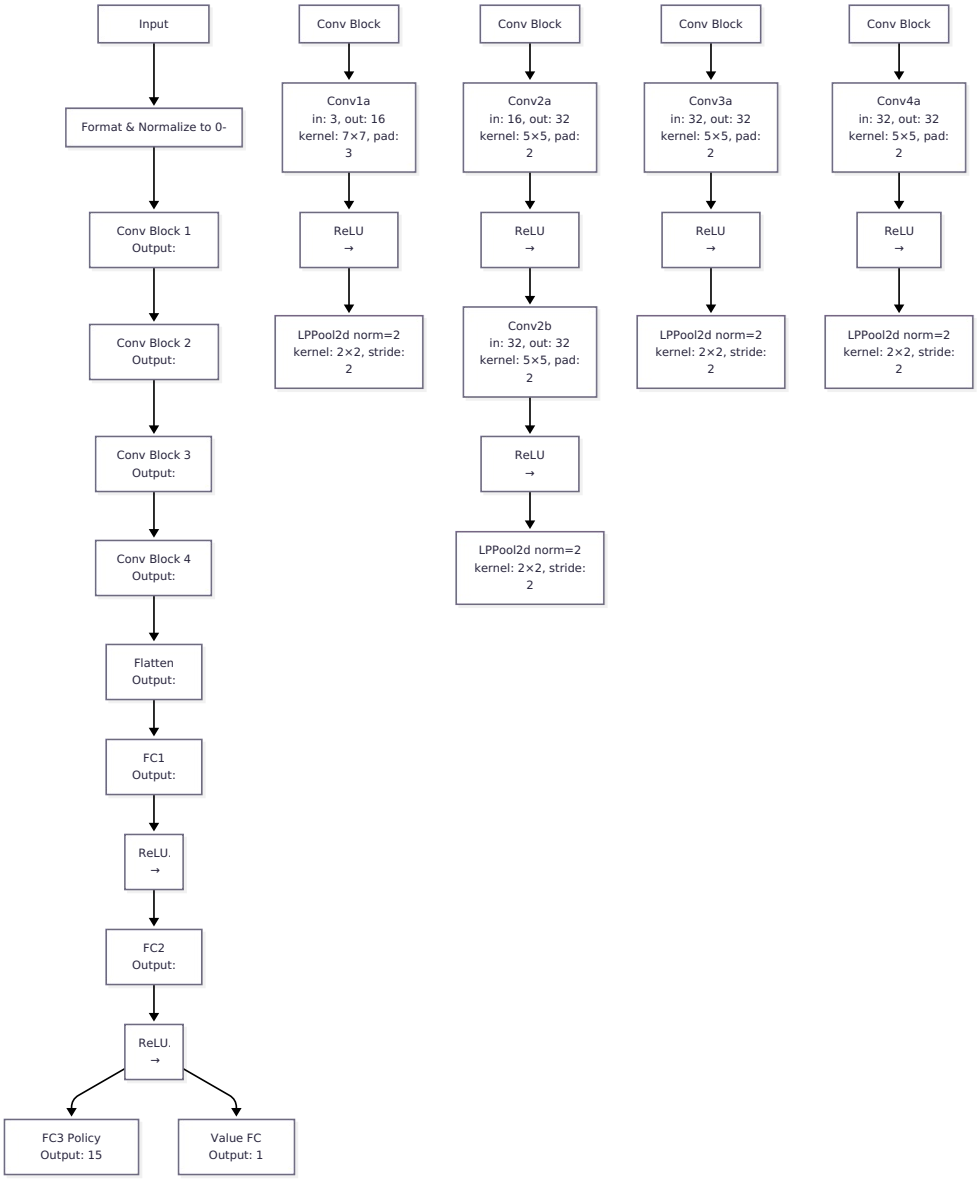


Figure 1: The convolutional neural network architecture used in our experiments. We use a modified version of the Impala CNN with 5 convolutional layers rather than the original 15 layers, which provides better interpretability without compromising performance.

# .2 Activation offset results

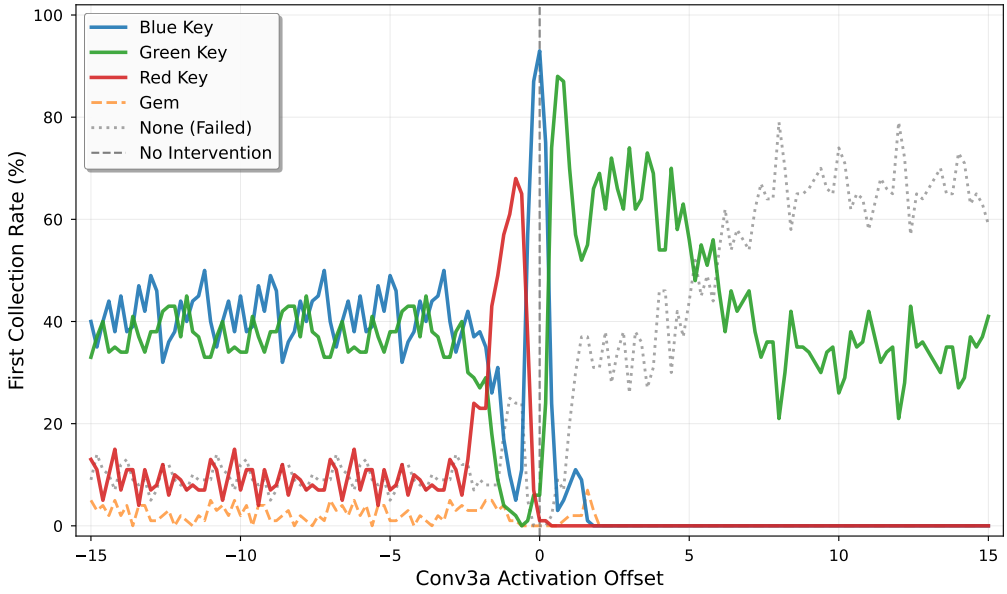


Figure 2: Full range activation offset sweep for conv3a layer at checkpoint 35001. Each subplot shows entity collection success rates across different activation offset values for a specific channel. This wide-range sweep reveals the broad activation ranges over which different channels can successfully steer the agent toward specific entities.

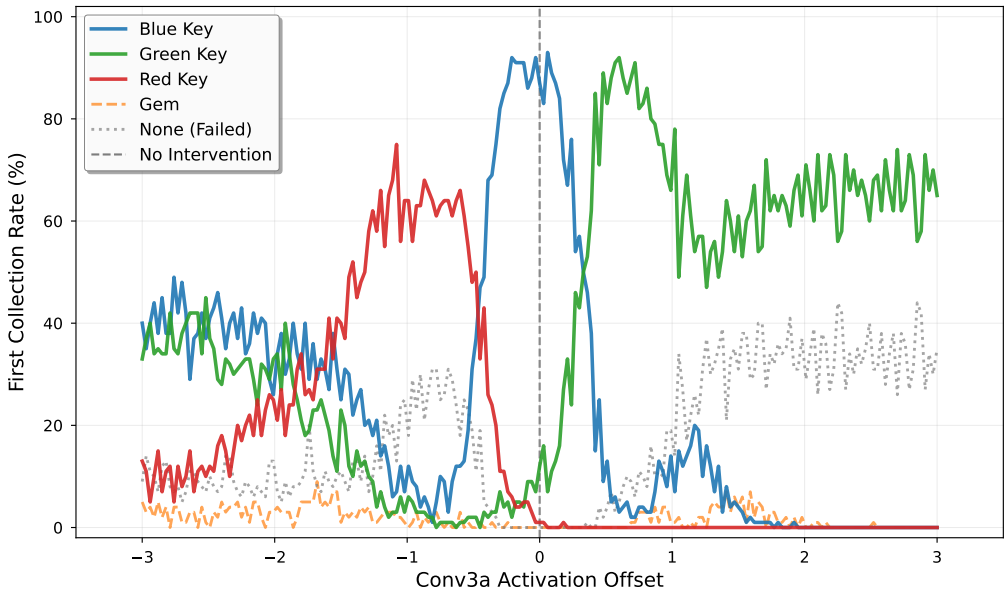


Figure 3: Narrow range activation offset sweep for conv3a layer at checkpoint 35001, focusing on offset values from -3 to +3 with step size 0.03. This fine-grained sweep provides higher resolution detail of entity collection rates in the critical activation range where steering behavior transitions occur.

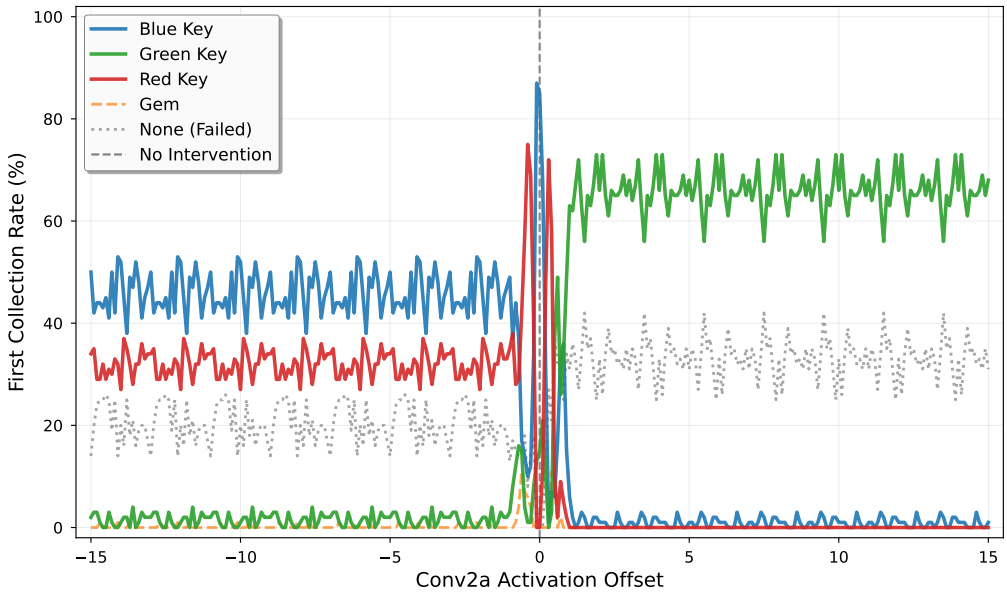


Figure 4: Wide range activation offset sweep for conv2a layer at checkpoint 35001, testing offset values from -15 to +15 with step size 0.1. This broader sweep examines the extent to which earlier convolutional layers can be steered across a wide range of activation offsets.

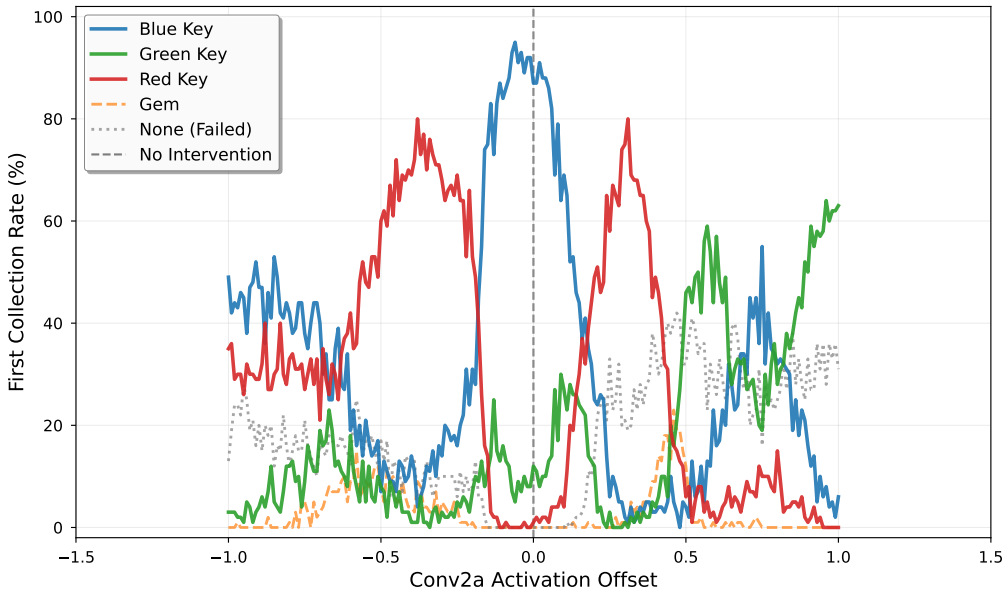


Figure 5: Narrow range activation offset sweep for conv2a layer at checkpoint 35001, focusing on offset values from -1 to +1 with step size 0.01. This high-resolution sweep examines fine-grained steering behavior in the conv2a layer's most sensitive activation range.

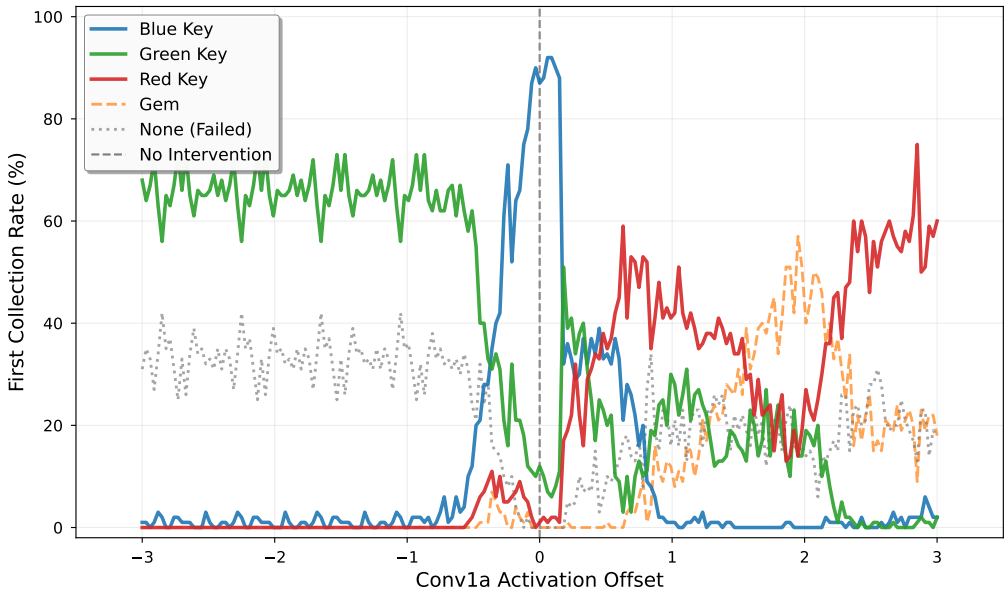


Figure 6: Activation offset sweep for conv1a layer at checkpoint 35001, testing offset values from -3 to +3 with step size 0.03. This sweep examines steering capabilities in the earliest convolutional layer, which operates closest to the raw input observations.

# .3 Mean Activation Per Channel During Rollout

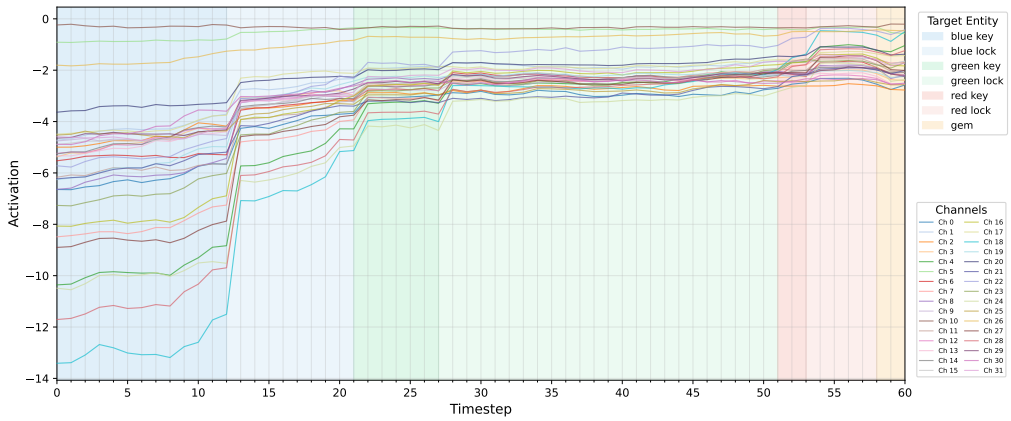


Figure 7: Figure showing the mean activation per channel across all channels in the same rollout as presented in Figure 4.3. We see that the pattern of increasing activation strength over the course of an episode with sharp jumps at the collection of different entities is shared across many but not all channels, with a few showing very consistent activation strengths from beginning to end. In general mean activations remain below 0 for all channels, indicating that a great deal of the neurons across channels are negative a majority of the time.

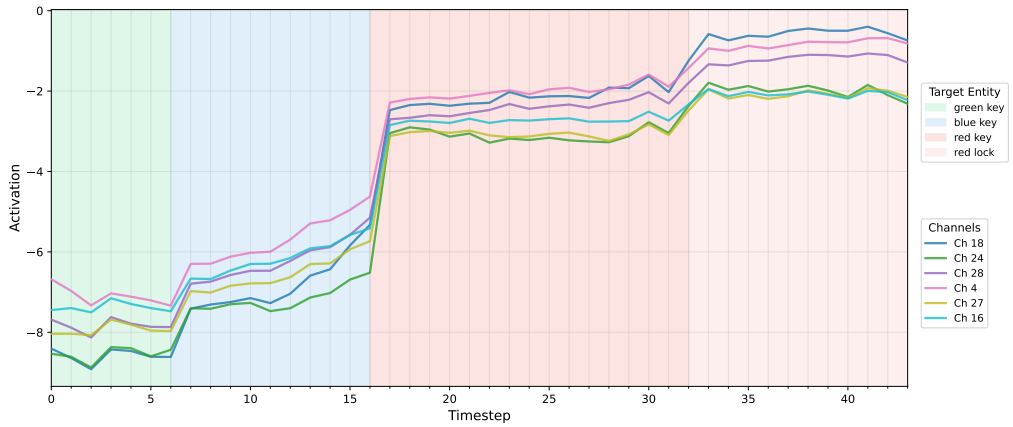


Figure 8: Plot showing the mean activation values captured across all timesteps in an unmodified rollout in the 6 channels that had the greatest variance over the course of the rollout. The shaded regions indicate the current 'next target', the entity that it would logically pursue next in solving the maze. In this rollout in the cross maze environment, the agent targeted the green key first. A similar uptick in positive values can be observed at the moment of green key collection, though the larger uptick in mean activations typically associated with the blue key can be observed at the moment of blue key collection as well.

# .4 Channel-Level Probe Results

This section presents detailed per-channel probe accuracies for multi-class entity classification across convolutional layers. All probes were trained using logistic regression on individual channel activations to predict the next entity the agent will navigate toward (7-way classification: blue key, green key, red key, blue lock, green lock, red lock, gem). Lock entities are omitted from these tables for brevity, though they were included in the overall accuracy calculations. Random baseline performance is 14.3%.

## .4.1 Conv1a Layer - All Channels

Table 1 shows probe results for all 16 channels in the Conv1a layer. This layer exhibits the highest variance in channel performance, with the best channel (Ch0) achieving 97.1% overall accuracy while the worst (Ch5) achieves only 14.7%. Three channels (Ch0, Ch11, Ch12) demonstrate strong generalist performance across all entity types, suggesting they encode rich multi-entity representations.

Table 1: Conv1a layer per-channel probe accuracies. All 16 channels shown. Bold indicates highest accuracy in each column.

| Channel | Overall | Blue Key | Green Key | Red Key | Gem |
|---|---|---|---|---|---|
| 0 | **97.1%** | 97.7% | 94.8% | 98.1% | **97.3%** |
| 1 | 17.7% | 0.0% | 0.0% | 79.3% | 45.1% |
| 2 | 65.1% | 90.7% | 14.5% | 94.7% | 90.2% |
| 3 | 20.3% | 0.0% | 3.5% | 0.0% | 50.5% |
| 4 | 17.2% | 98.1% | 0.0% | 0.0% | 4.4% |
| 5 | 14.7% | 0.0% | 0.0% | 0.0% | 0.0% |
| 6 | 21.9% | 28.4% | 0.0% | 54.8% | 0.0% |
| 7 | 16.1% | 0.0% | 0.0% | 94.7% | 0.0% |
| 8 | 21.1% | 0.0% | 61.6% | 0.0% | 53.8% |
| 9 | 14.9% | 0.0% | 0.0% | **100.0%** | 0.0% |
| 10 | 17.1% | 0.0% | 43.6% | 76.0% | 0.0% |
| 11 | 94.7% | **100.0%** | **98.3%** | 90.4% | 88.6% |
| 12 | 91.6% | 99.5% | 94.8% | 88.5% | 89.7% |
| 13 | 20.3% | 0.0% | 0.0% | 54.3% | 0.0% |
| 14 | 15.2% | 2.3% | 0.0% | **100.0%** | 0.0% |
| 15 | 17.7% | 0.0% | 0.0% | 0.0% | 48.4% |

## .4.2 Conv2a Layer - Top 10 Channels

Table 2 presents the top 10 performing channels in Conv2a (out of 32 total). This layer shows dramatically improved performance compared to Conv1a, with the best channel (Ch4) achieving near-perfect 99.4% accuracy. The top channel achieves perfect accuracy on three entity types (blue lock, green lock, red lock, gem), suggesting highly disentangled representations.

Table 2: Conv2a layer top 10 channels by overall accuracy (out of 32 total channels). Bold indicates highest accuracy in each column.

| Channel | Overall | Blue Key | Green Key | Red Key | Gem |
|---|---|---|---|---|---|
| 4 | **99.4%** | 98.1% | 98.8% | **99.0%** | **100.0%** |
| 27 | 98.9% | 98.6% | **99.4%** | 96.6% | **100.0%** |
| 1 | 95.2% | 98.6% | 96.5% | 94.2% | 98.9% |
| 25 | 93.9% | 97.2% | 94.8% | 88.0% | 96.2% |
| 14 | 90.4% | 88.8% | 94.2% | 83.2% | **100.0%** |
| 31 | 90.2% | 92.6% | 88.4% | 86.1% | 95.1% |
| 5 | 88.4% | 92.1% | 86.1% | 75.0% | 96.7% |
| 29 | 87.6% | 95.4% | 89.5% | 77.9% | 96.2% |
| 18 | 86.4% | 88.4% | 84.3% | 77.4% | **100.0%** |
| 16 | 85.5% | **95.8%** | 77.9% | 70.7% | 91.9% |

### .4.3 Conv3a Layer - Top 10 Channels

Table 3 shows the top 10 channels in Conv3a (out of 32 total). Interestingly, this layer shows lower per-channel performance than Conv2a (best: 82.9% vs 99.4%), despite whole-layer probes achieving 99.3% accuracy. This suggests Conv3a relies more heavily on distributed representations across multiple channels rather than individual specialist channels.

Table 3: Conv3a layer top 10 channels by overall accuracy (out of 32 total channels). Bold indicates highest accuracy in each column.

| Channel | Overall | Blue Key | Green Key | Red Key | Gem |
|---|---|---|---|---|---|
| 19 | **82.9%** | **89.8%** | **79.1%** | **80.8%** | 90.2% |
| 20 | 82.7% | 77.2% | **79.1%** | 80.3% | 88.6% |
| 9 | 78.6% | 77.7% | **79.1%** | 76.4% | 90.2% |
| 4 | 78.4% | 83.7% | 75.0% | 72.1% | 83.7% |
| 6 | 76.4% | 84.7% | 72.1% | 73.1% | 85.3% |
| 25 | 76.2% | 79.1% | 72.1% | 62.5% | **97.8%** |
| 23 | 75.6% | 87.9% | 66.9% | 71.2% | 85.3% |
| 2 | 74.4% | 77.7% | 66.3% | 74.0% | 86.4% |
| 3 | 74.1% | 72.1% | 69.2% | 70.2% | 89.7% |
| 21 | 73.9% | 79.5% | 61.6% | 70.7% | 87.5% |

### .4.4 Conv4a Layer - Top 10 Channels

Table 4 presents the top 10 channels in Conv4a (out of 32 total). This is the final convolutional layer before the fully connected layers. Notably, even the best channel (Ch18, 68.1%) shows substantially lower performance than individual channels in earlier layers, despite whole-layer probes achieving 97.6% accuracy. This indicates that Conv4a representations are highly distributed and no single channel carries sufficient information for accurate classification.

Table 4: Conv4a layer top 10 channels by overall accuracy (out of 32 total channels). Bold indicates highest accuracy in each column.

| Channel | Overall | Blue Key | Green Key | Red Key | Gem |
|---|---|---|---|---|---|
| 18 | **68.1%** | 83.3% | 45.4% | **70.7%** | **88.0%** |
| 8 | 67.1% | **86.5%** | 50.0% | 59.1% | 73.4% |
| 17 | 64.9% | 83.3% | 55.2% | 57.7% | 78.3% |
| 28 | 63.0% | 82.3% | 51.2% | 52.4% | 79.9% |
| 10 | 62.0% | 68.8% | 55.2% | 58.2% | 78.8% |
| 0 | 61.9% | 78.6% | **56.4%** | 55.3% | 66.3% |
| 21 | 61.9% | 71.2% | 44.8% | 61.5% | 69.0% |
| 4 | 61.8% | 74.4% | 47.1% | 52.4% | 81.5% |
| 26 | 60.4% | 74.0% | 39.5% | 51.4% | 69.0% |
| 16 | 59.4% | 82.3% | 45.4% | 56.3% | 64.7% |

### .4.5 Summary of Channel-Level Findings

Several patterns emerge from the channel-level probe analysis:

1. **Layer hierarchy**: Conv1a contains a small number of highly informative generalist channels (Ch0, Ch11, Ch12) alongside many specialist channels that encode only specific entities. Conv2a shows improved overall performance with multiple high-performing generalist channels. Conv3a and Conv4a show increasingly distributed representations where individual channels carry less information.

2. **Distributed vs sparse coding**: The gap between whole-layer and best single-channel performance grows substantially in deeper layers (Conv3a: 99.3% whole vs 82.9% best channel; Conv4a: 97.6% whole vs 68.1% best channel), indicating a shift from sparse coding in early layers to distributed coding in later layers.

3. **Entity-specific patterns**: Some channels in Conv1a show extreme specialization (e.g., Ch9 and Ch14 achieve 100% on red key but near 0% on other

entities), while Conv2a channels tend toward more balanced multi-entity performance.

## .5 Intervention spans across checkpoints

Figures 9–13 illustrate the representational drift observed across checkpoints 40k-60k. Despite stable task performance, the specific channels exhibiting steering behavior and their associated activation ranges shift dramatically between checkpoints, demonstrating that the network continuously explores equivalent solutions within a broad basin of the loss landscape.

## .6 SAE Intervention Results

The following figures show steering intervention success rates for SAE latents both trained on the 35000 step checkpoint. Unlike the base model channels (which have 32 dimensions per layer), the SAE uses a 4x expansion factor, resulting in 128 latents per layer. These figures demonstrate that sparse autoencoder latents can also be successfully steered to influence agent behavior toward specific entities. Colored dots represent successful steering interventions for different entities (blue key, green key, red key, gem), while black lines indicate the 99.9th percentile of normal activations during standard gameplay. Figures shown have a threshold for inclusion of $\geq 200$ successes for readability.

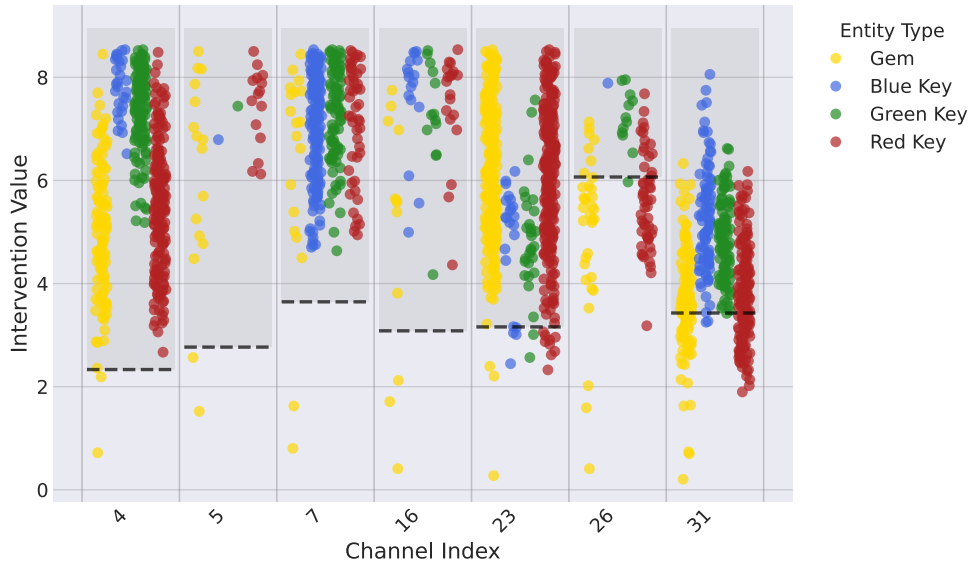


Figure 9: Steering intervention success rates for conv4a layer at checkpoint 40k. Colored dots represent successful steering interventions for different entities (blue key, green key, red key, gem), while black lines indicate the 99.9th percentile of normal activations. Each panel shows a different channel exhibiting steering behavior.

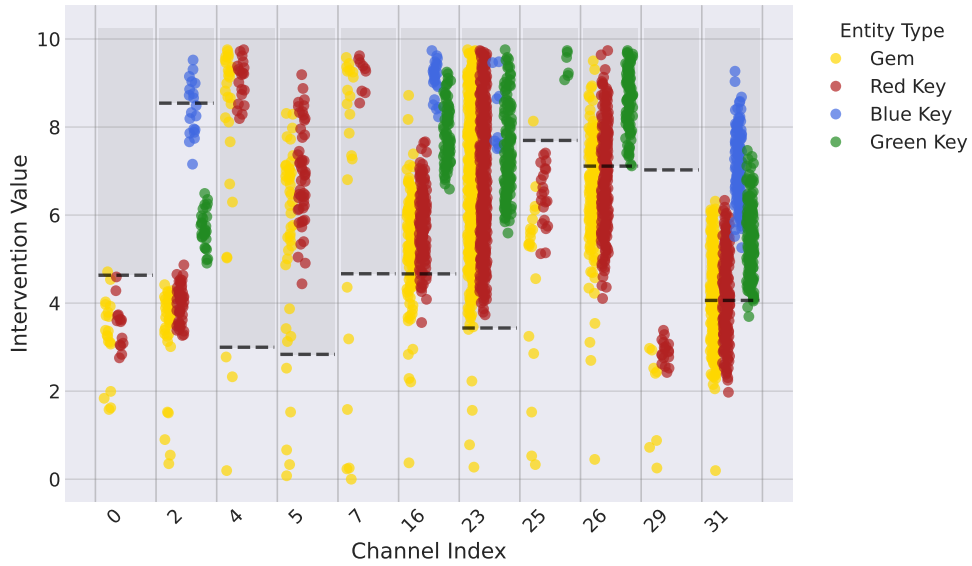


Figure 10: Steering intervention success rates for conv4a layer at checkpoint 45k. Colored dots represent successful steering interventions for different entities (blue key, green key, red key, gem), while black lines indicate the 99.9th percentile of normal activations. Each panel shows a different channel exhibiting steering behavior.

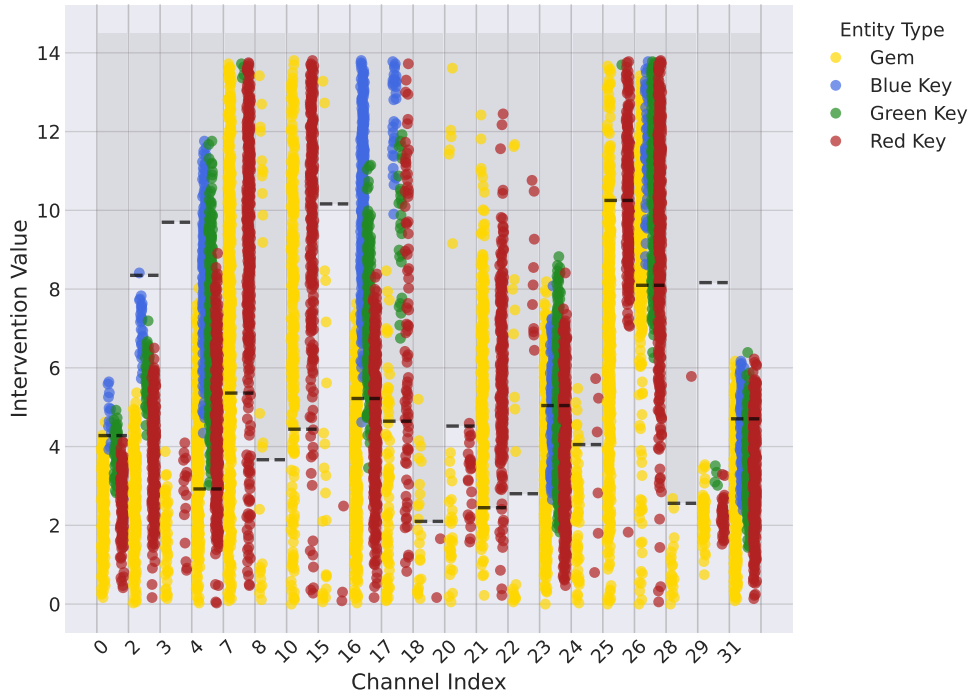


Figure 11: Steering intervention success rates for conv4a layer at checkpoint 50k. Colored dots represent successful steering interventions for different entities (blue key, green key, red key, gem), while black lines indicate the 99.9th percentile of normal activations. Each panel shows a different channel exhibiting steering behavior.

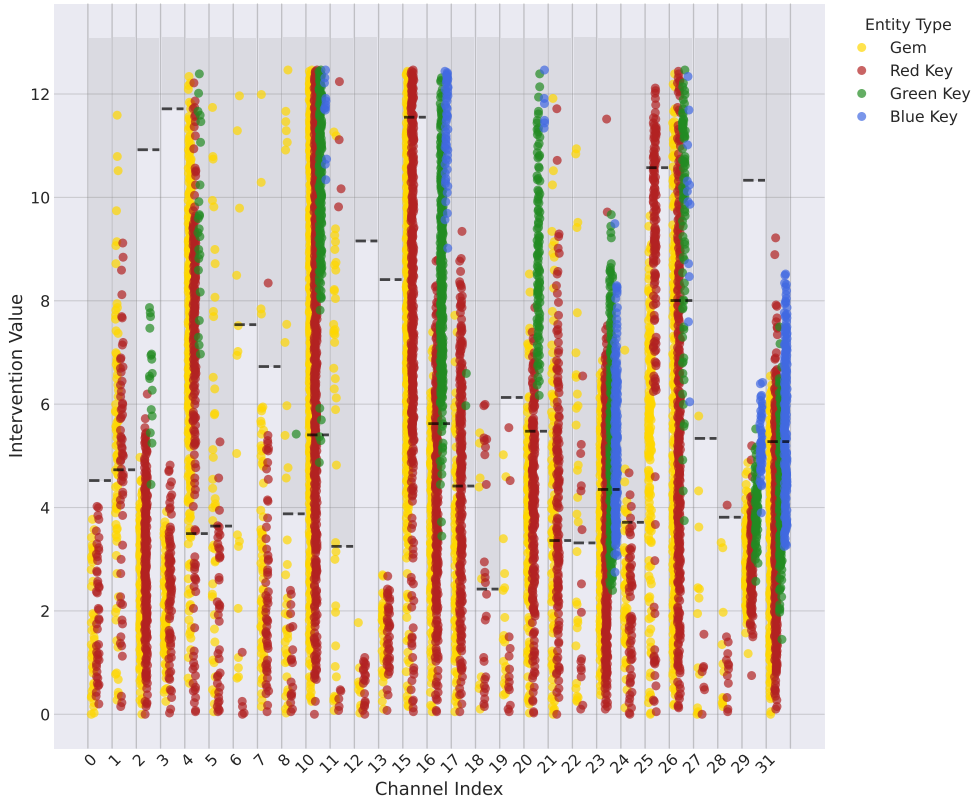


Figure 12: Steering intervention success rates for conv4a layer at checkpoint 55k. Colored dots represent successful steering interventions for different entities (blue key, green key, red key, gem), while black lines indicate the 99.9th percentile of normal activations. Each panel shows a different channel exhibiting steering behavior.

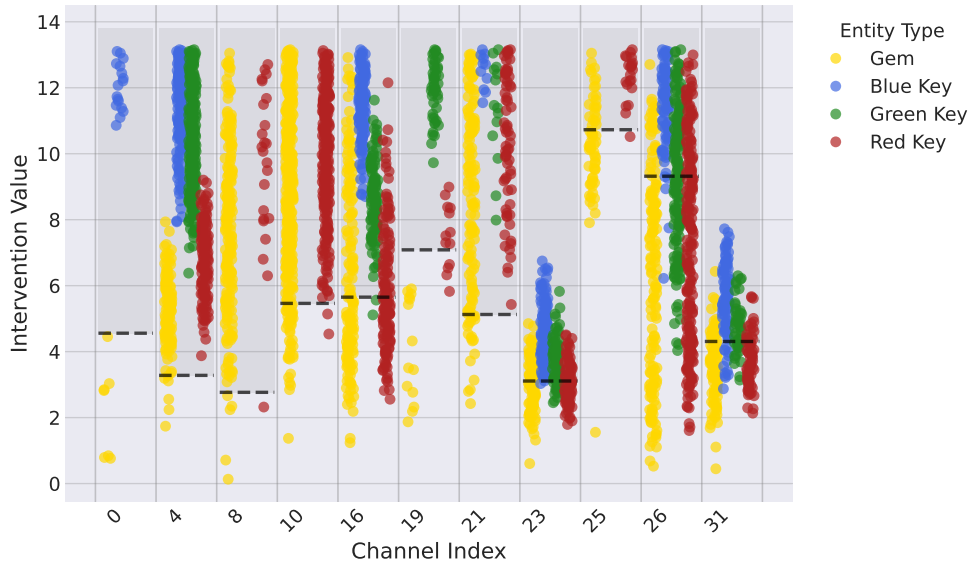


Figure 13: Steering intervention success rates for conv4a layer at checkpoint 60k. Colored dots represent successful steering interventions for different entities (blue key, green key, red key, gem), while black lines indicate the 99.9th percentile of normal activations. Each panel shows a different channel exhibiting steering behavior.

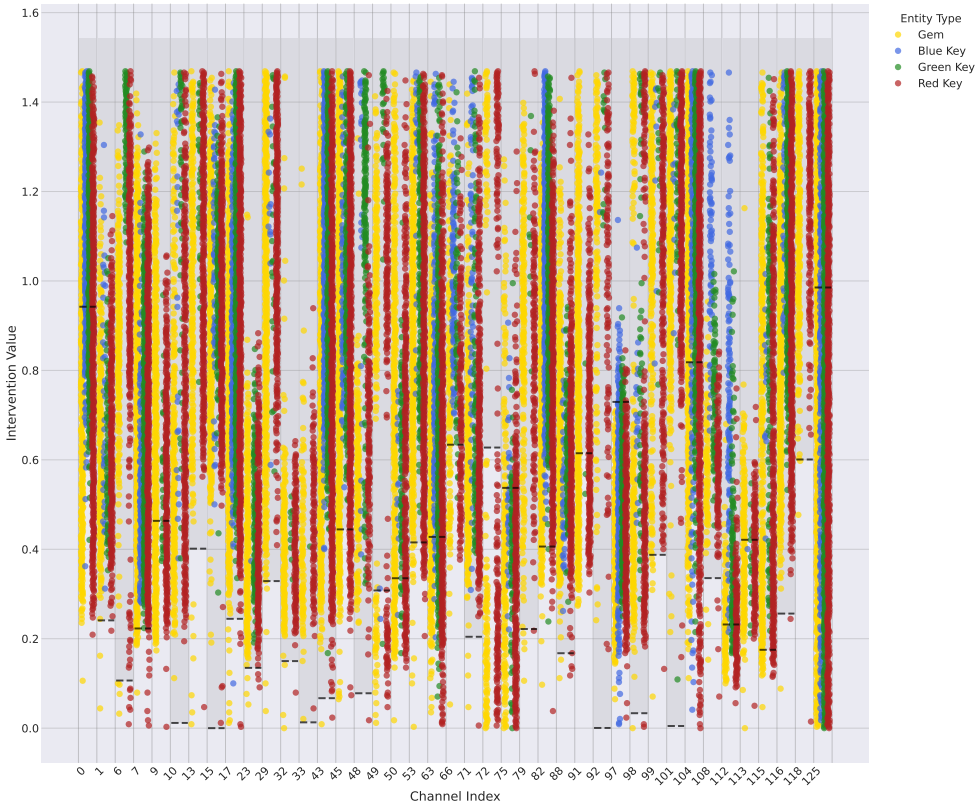


Figure 14: SAE latent steering intervention success rates for conv3a layer, showing the top 41 most successful latents with a cutoff threshold of $\geq 200$ successes. Of particular note is the large number of steering successes in general in conv3a and also the significant overlap between different sections, including within similar activation regions, displaying relatively less of the multiplexing phenomena relative to elsewhere.

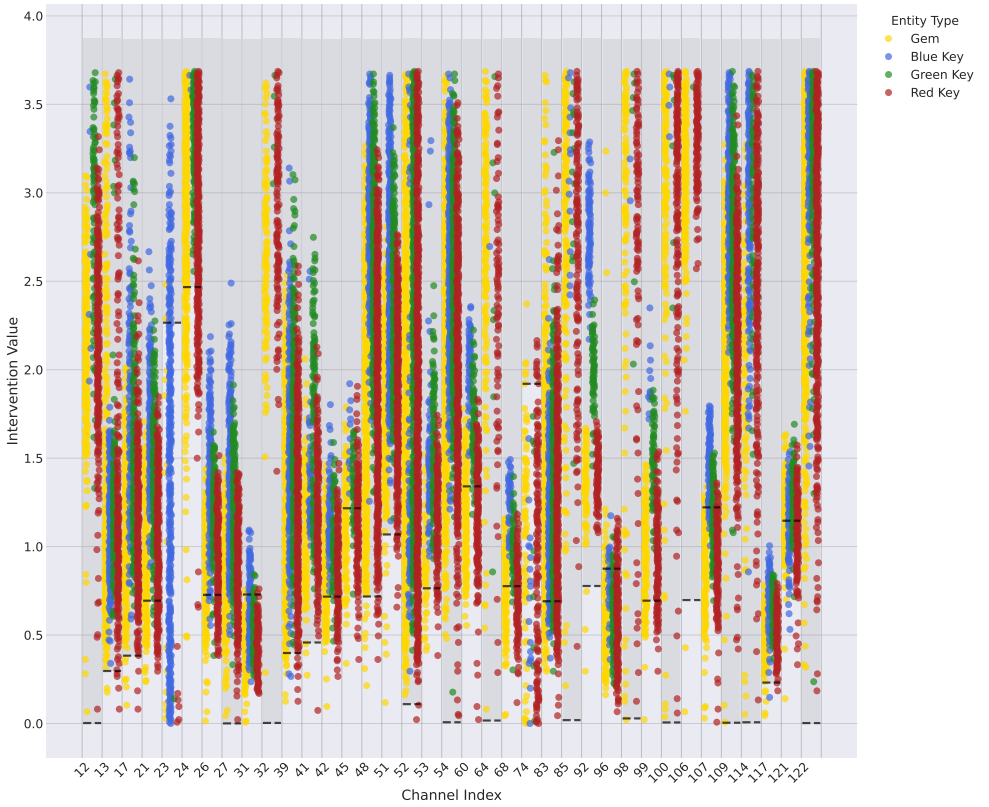


Figure 15: SAE latent steering intervention success rates for conv4a layer, showing the top 37 most successful latents with a cutoff threshold $\geq 200$ successes. Conv4a SAE shows significantly sharper multiplexing than any other example, in particular latent 92 shows almost perfect delineation of entities. Latent 23 also contains an unusual capacity for steering the blue key in particular, indicating that some blue key specialisation may have been learned, though we caution against making strong conclusions from the steering alone due to the complex nature of the network.